Statistical physics approaches to optimal learning protocols

Francesca Mignacco

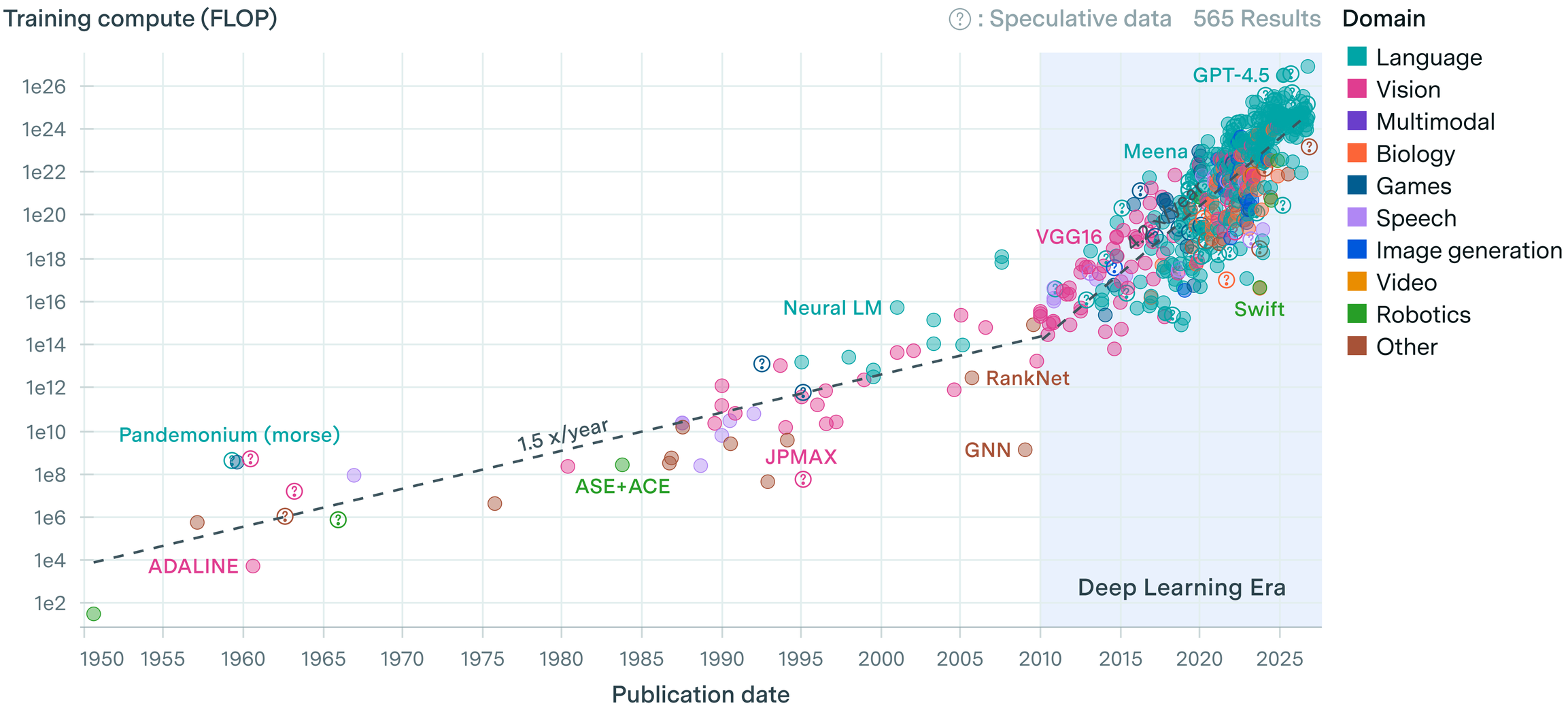
Princeton University & CUNY Graduate Center

Machine Learning in Chemical Physics @ Princeton

27th October 2026

The empirical successes of modern ML

Notable AI models



EPOCH AI | CC-BY

Questions in ML theory

For instance, there are many important questions regarding neural networks which are largely unanswered. There seem to be conflicting stories regarding the following issues:

- Why don't heavily parameterized neural networks overfit the data?
- What is the effective number of parameters?
- Why doesn't backpropagation head for a poor local minima?
- When should one stop the backpropagation and use the current parameters?

Questions in ML theory

… since 30 years !

For instance, there are many important questions regarding neural networks which are largely unanswered. There seem to be conflicting stories regarding the following issues:

- Why don't heavily parameterized neural networks overfit the data?
- What is the effective number of parameters?
- Why doesn't backpropagation head for a poor local minima?
- When should one stop the backpropagation and use the current parameters?

"Reflections after refereeing papers for NIPS", Leo Breiman, **1995**

Questions in ML theory

… since 30 years !

For instance, there are many important questions regarding neural networks which are largely unanswered. There seem to be conflicting stories regarding the following issues:

- Why don't heavily parameterized neural networks overfit the data?
- What is the effective number of parameters?
- Why doesn't backpropagation head for a poor local minima?
- When should one stop the backpropagation and use the current parameters?

INQUIRY = sensible and intelligent efforts to understand what is going on. For example:

- mathematical heuristics
- simplified analogies (like the Ising Model)
- simulations
- comparisons of methodologies
- devising new tools
- theorems where useful (rare!)
- shunning panaceas

"Reflections after refereeing papers for NIPS", Leo Breiman, **1995**

Questions in ML theory

For instance, there are many important questions regarding neural networks which are largely unanswered. There seem to be conflicting stories regarding the following issues:

1. Why don't heavily parameterized neural networks overfit the data?
2. What is the effective number of parameters?
3. Why doesn't backpropagation head for a poor local minima?
4. When should one stop the backpropagation and use the current parameters?

1 Understanding the **statics**

"Reflections after refereeing papers for NIPS", Leo Breiman, **1995**

Questions in ML theory

For instance, there are many important questions regarding neural networks which are largely unanswered. There seem to be conflicting stories regarding the following issues:

- Why don't heavily parameterized neural networks overfit the data?
- What is the effective number of parameters?
- Why doesn't backpropagation head for a poor local minima?
- When should one stop the backpropagation and use the current parameters?

2

1 Understanding the **statics**

2 Understanding the **dynamics**

"Reflections after refereeing papers for NIPS", Leo Breiman, **1995**

Questions in ML theory

For instance, there are many important questions regarding neural networks which are largely unanswered. There seem to be conflicting stories regarding the following issues:

- Why don't heavily parameterized neural networks overfit the data?
- What is the effective number of parameters?
- Why doesn't backpropagation head for a poor local minima?
- When should one stop the backpropagation and use the current parameters?

2

1 Understanding the **statics**

2 Understanding the **dynamics**

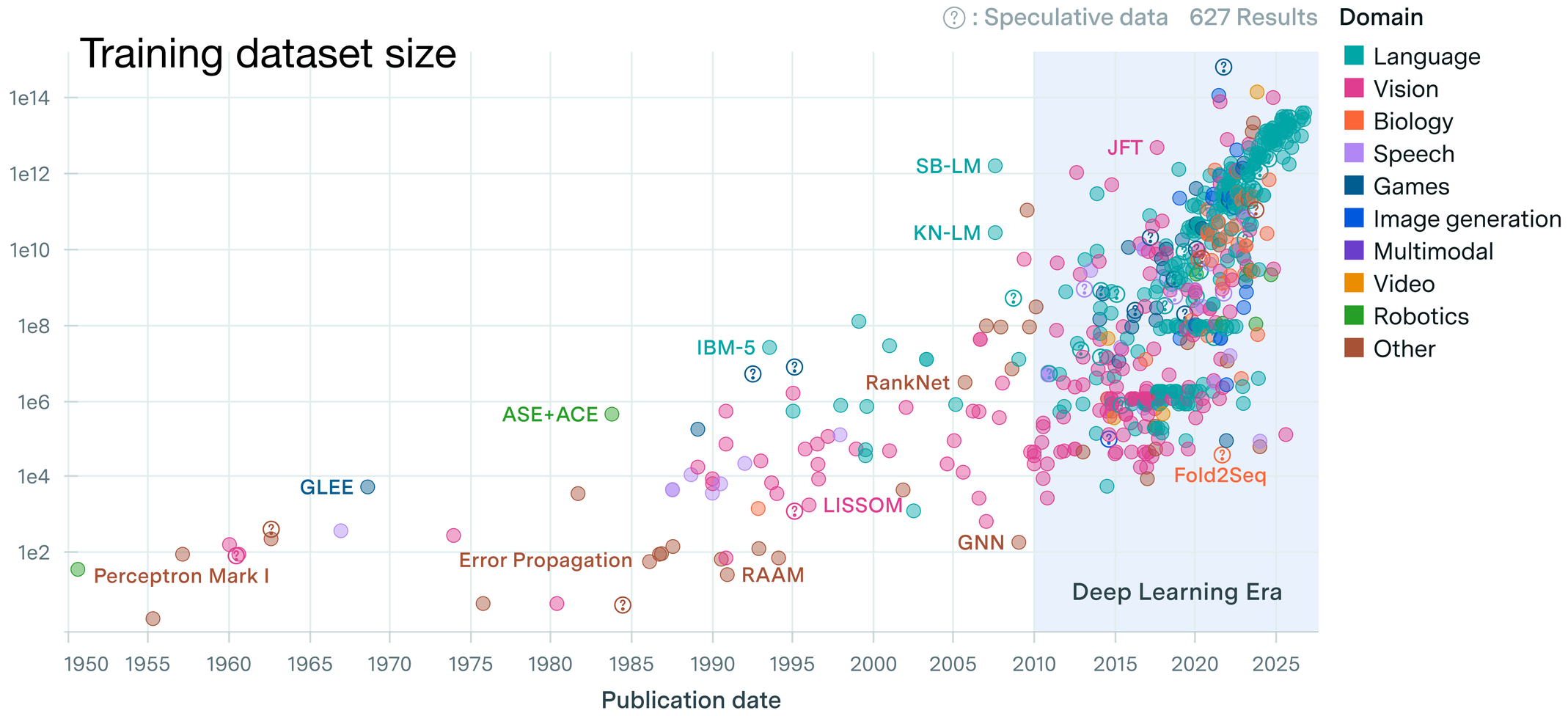
Focus of **this talk**

"Reflections after refereeing papers for NIPS", Leo Breiman, **1995**

Three main ingredients

Three main ingredients

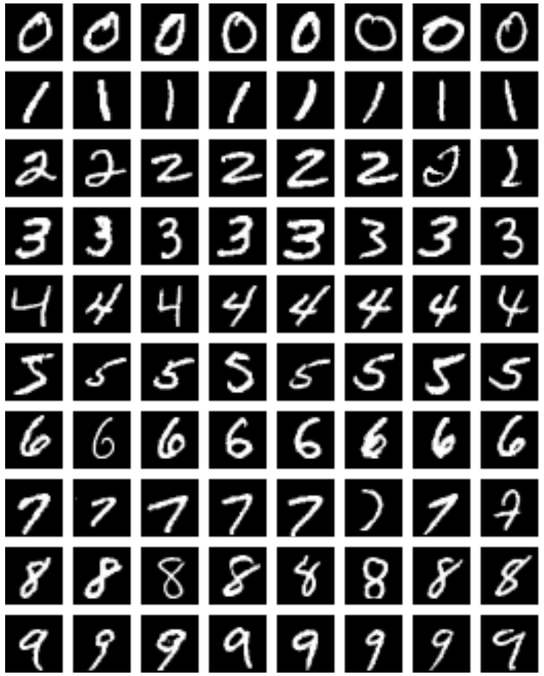
① Data



Three main ingredients

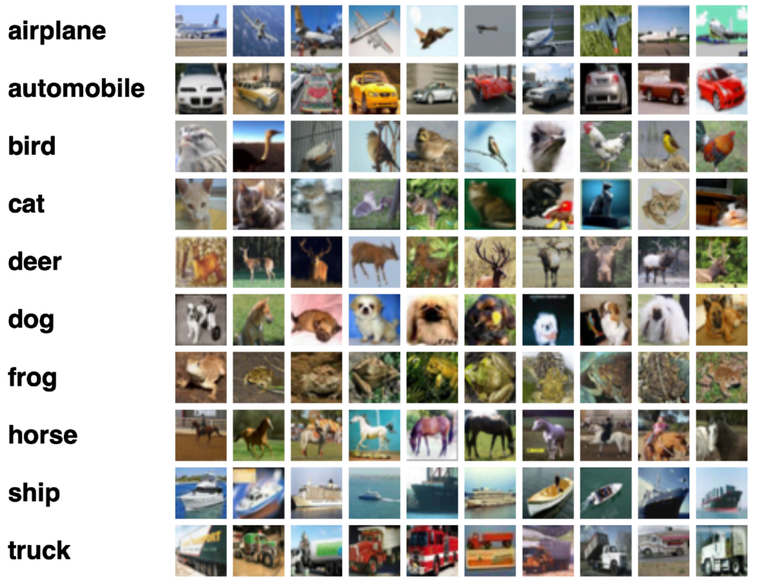
① Data

e.g., images



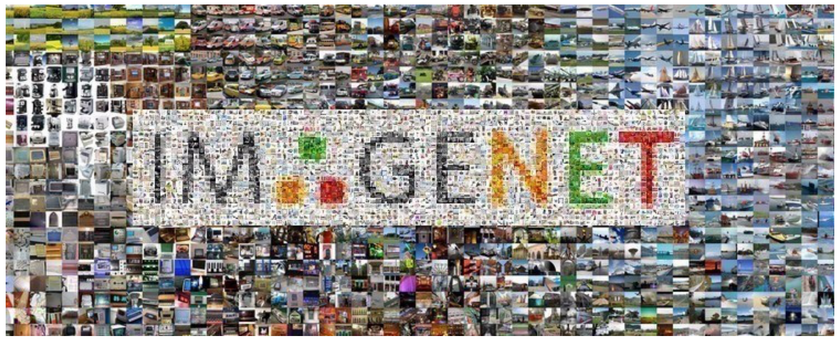
MNIST, 1998

28 × 28 grayscale, ~70k images



CIFAR10, 2009

32 × 32 RGB, ~60k images



ImageNet, 2009

14M images, ~20k categories



LAION-5B, 2022

~5.8B of image-text pairs

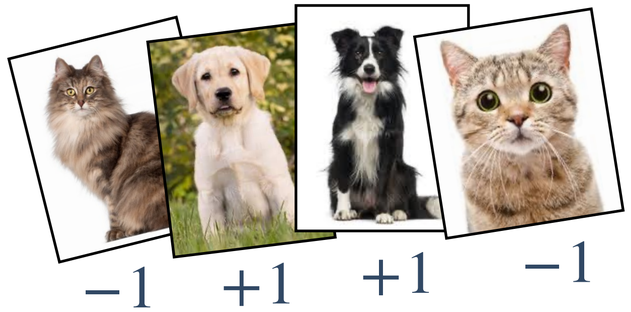
Three main ingredients

1. Data / Task

Three main ingredients

① Data / Task

e.g., supervised learning

dataset: $\mathscr{D} = \left\{ \left(\mathbf{x}_\mu, y_\mu \right) \right\}_{\mu=1}^{P} = \left\{ \right.$  $\left. \right\}$

$-1 \quad +1 \quad +1 \quad -1$

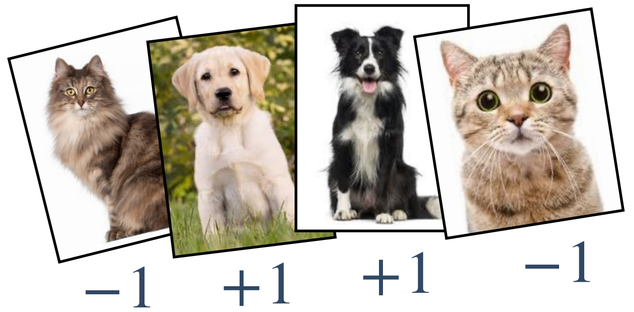
Input data $\mathbf{x}_\mu \in \mathbb{R}^N$ and output labels $y_\mu \in \mathbb{R}$

N = #dimensions

P = #samples

Three main ingredients

① Data / Task

e.g., supervised learning dataset: $\mathscr{D} = \left\{ \left(\mathbf{x}_\mu, y_\mu \right) \right\}_{\mu=1}^{P} =$ 

Input data $\mathbf{x}_\mu \in \mathbb{R}^N$ and output labels $y_\mu \in \mathbb{R}$

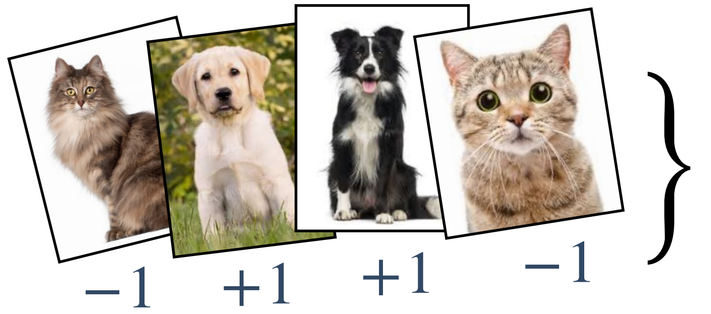
N = #dimensions
P = #samples

Goal: learn the *mapping:* $\mathbf{x} \to y$

$\mathbf{x}_{\text{new}} =$  , $y_{\text{new}} = ?$

Three main ingredients

① Data / Task

e.g., supervised learning dataset: $\mathscr{D} = \left\{ \left(\mathbf{x}_\mu, y_\mu \right) \right\}_{\mu=1}^{P} =$ 

Input data $\mathbf{x}_\mu \in \mathbb{R}^N$ and output labels $y_\mu \in \mathbb{R}$

N = #dimensions
P = #samples

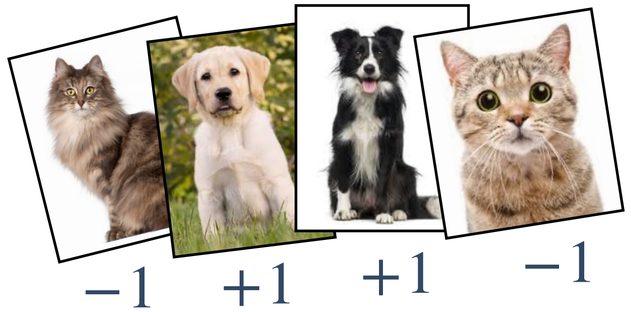
Goal: learn the *mapping:* $\mathbf{x} \to y$

$\mathbf{x}_{\text{new}} =$  , $y_{\text{new}} = ?$

Learn the predictor: $\hat{y}_{\text{new}} = f_{\mathbf{w}} \left(\mathbf{x}_{\text{new}} \right)$

Three main ingredients

① Data / Task

e.g., supervised learning dataset: $\mathscr{D} = \left\{ \left(\mathbf{x}_\mu, y_\mu \right) \right\}_{\mu=1}^{P} =$ 

Input data $\mathbf{x}_\mu \in \mathbb{R}^N$ and output labels $y_\mu \in \mathbb{R}$

N = #dimensions
P = #samples

Goal: learn the *mapping:* $\mathbf{x} \to y$

$\mathbf{x}_{\text{new}} =$  , $y_{\text{new}} = ?$

Learn the predictor: $\hat{y}_{\text{new}} = f_{\mathbf{w}}\left(\mathbf{x}_{\text{new}}\right)$ ⟶ *Learn* the **weights w**

Three main ingredients

② Architecture $\hat{y} = f_{\mathbf{w}}(\mathbf{x})$

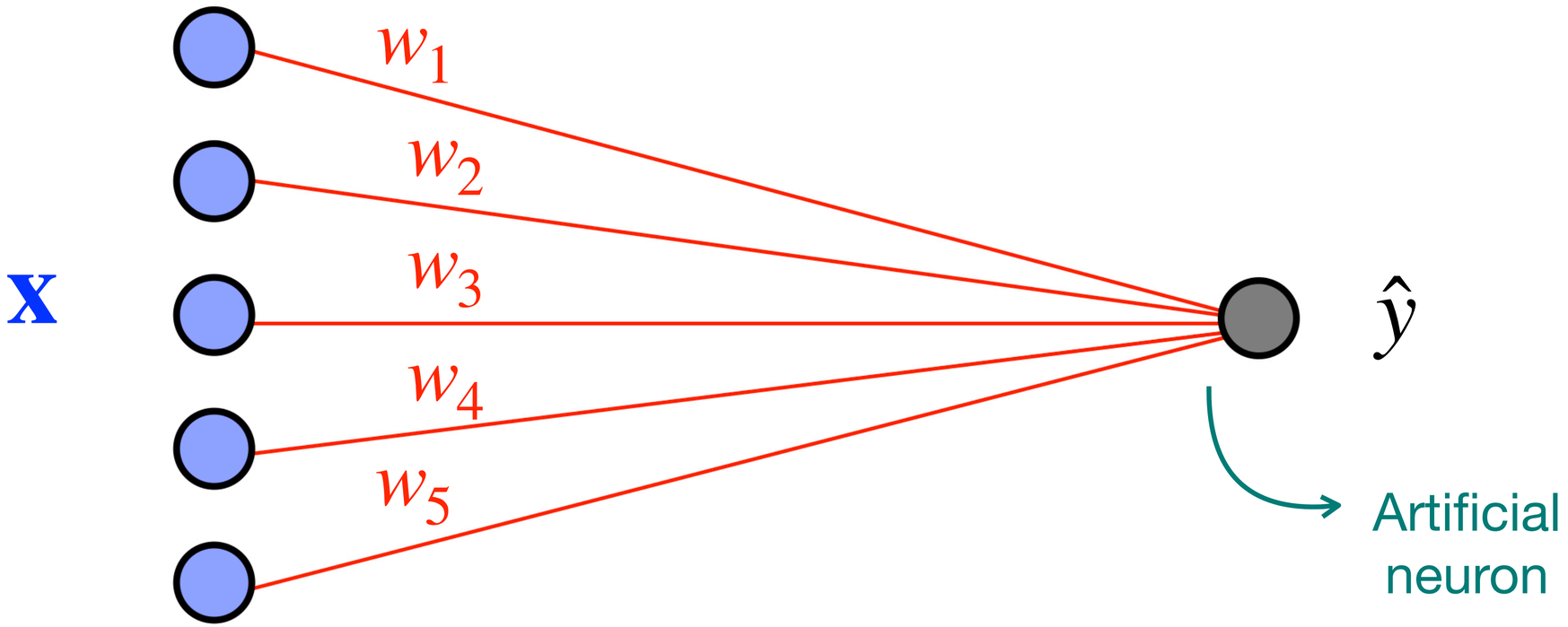
Three main ingredients

② Architecture

$$\hat{y} = f_{\mathbf{w}}(\mathbf{x}) = \phi\left(\mathbf{w}^{\top}\mathbf{x}\right)$$

The perceptron

Single-layer



McCulloch, Pitts (1943); Rosenblatt (1958)

Three main ingredients

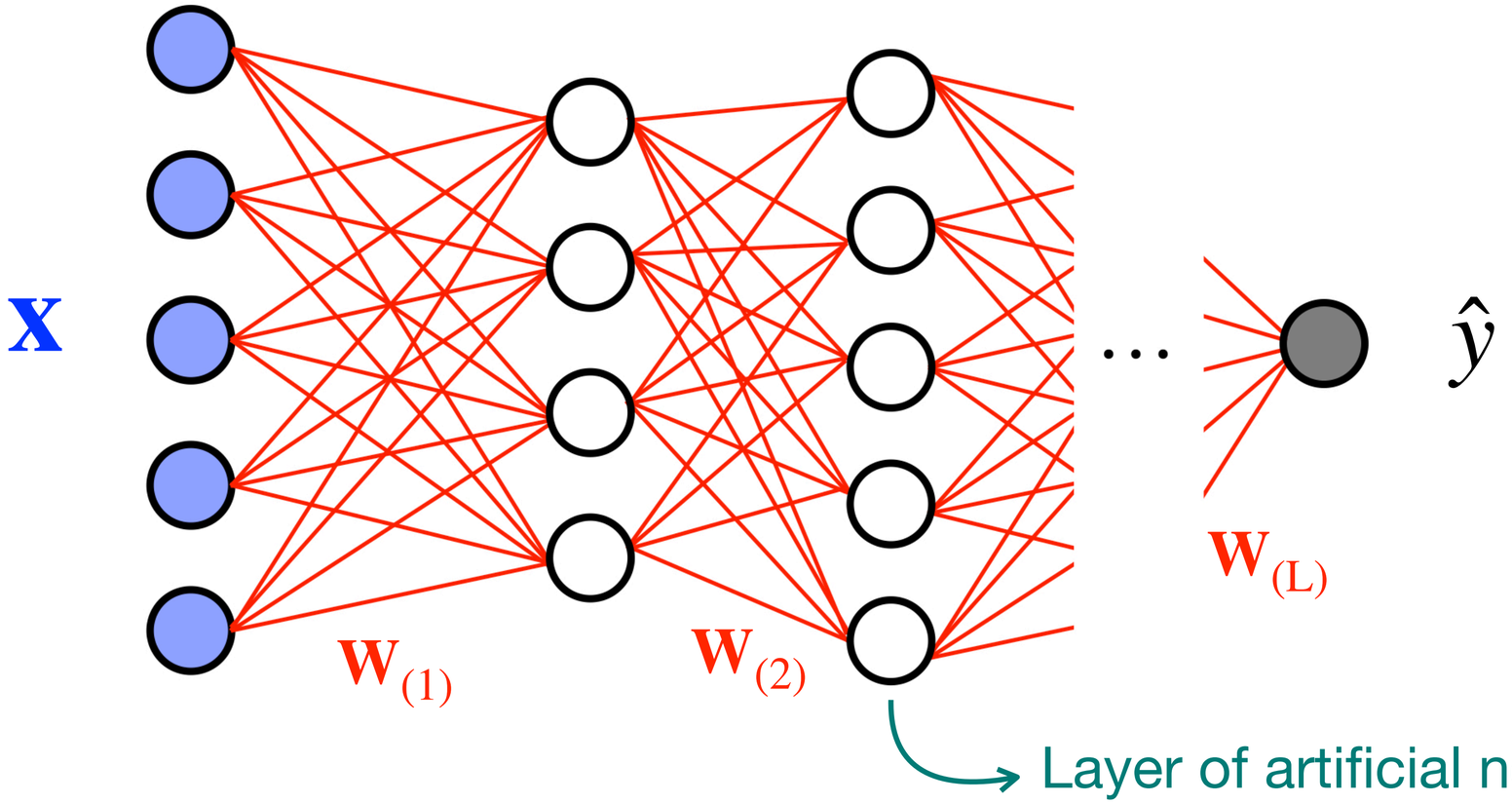
② Architecture

$$\hat{y} = f_{\mathbf{W}}(\mathbf{x}) = \phi^{(\mathrm{L})}\left(\mathbf{W}_{(\mathrm{L})} \ \cdots \ \phi^{(1)}\left(\mathbf{W}_{(1)}\,\mathbf{x}\right)\right)$$

Deep Neural Networks

Multi-layer

(Feedforward, fully-connected)



LeCun, Bengio, Hinton (*Nature*, 2015)

Three main ingredients

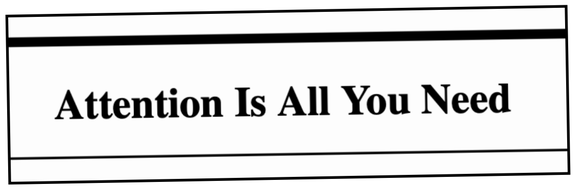
② Architecture e.g., **Transformers**

Sequential data: language, code, protein sequences…

Next-token prediction:

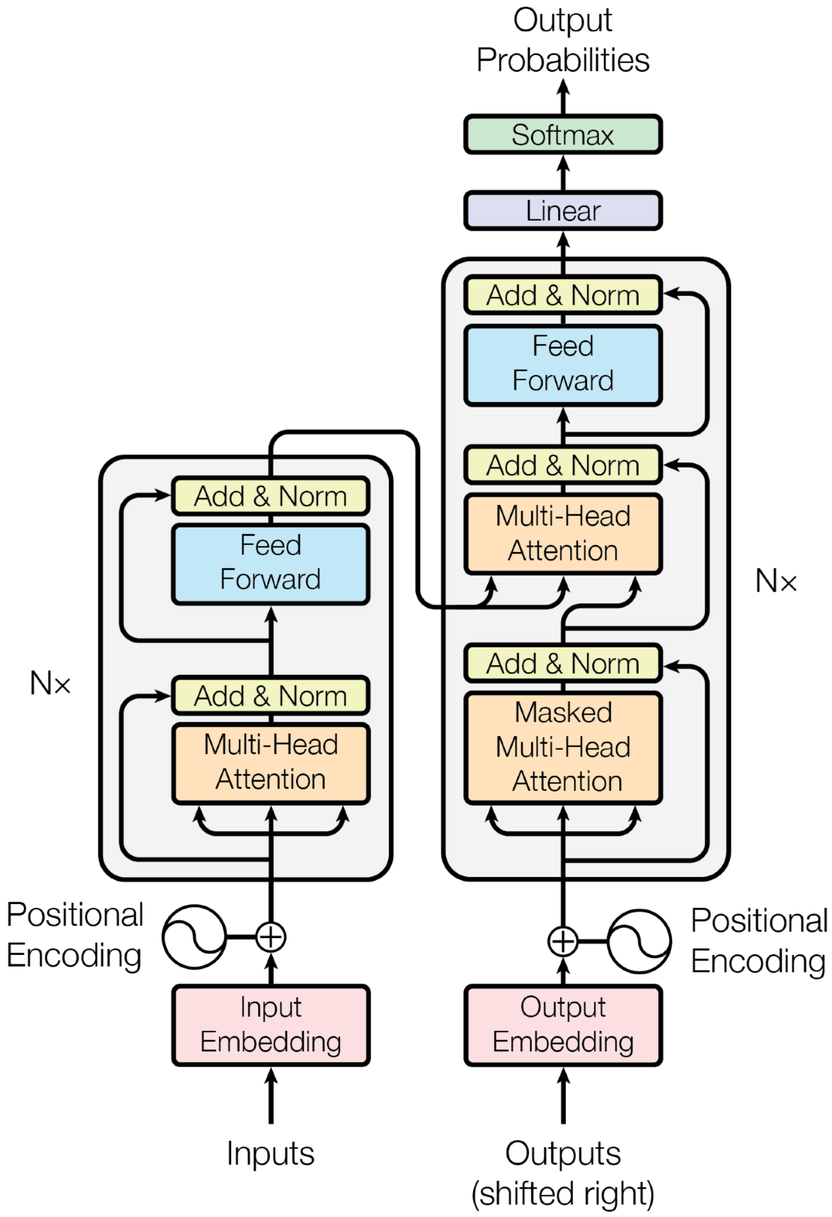
$$x_1, \ldots, x_t \longrightarrow p_W(x_{t+1} \mid x_1, \ldots, x_t)$$

$$\text{The cat sat on the} \longrightarrow \begin{cases} \text{mat} & 0.42 \\ \text{floor} & 0.21 \\ \text{sofa} & 0.08 \\ \cdots \end{cases}$$



Vaswani et al., *NeurIPS* 2017

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\text{T}}}{\sqrt{d_k}}\right)V$$



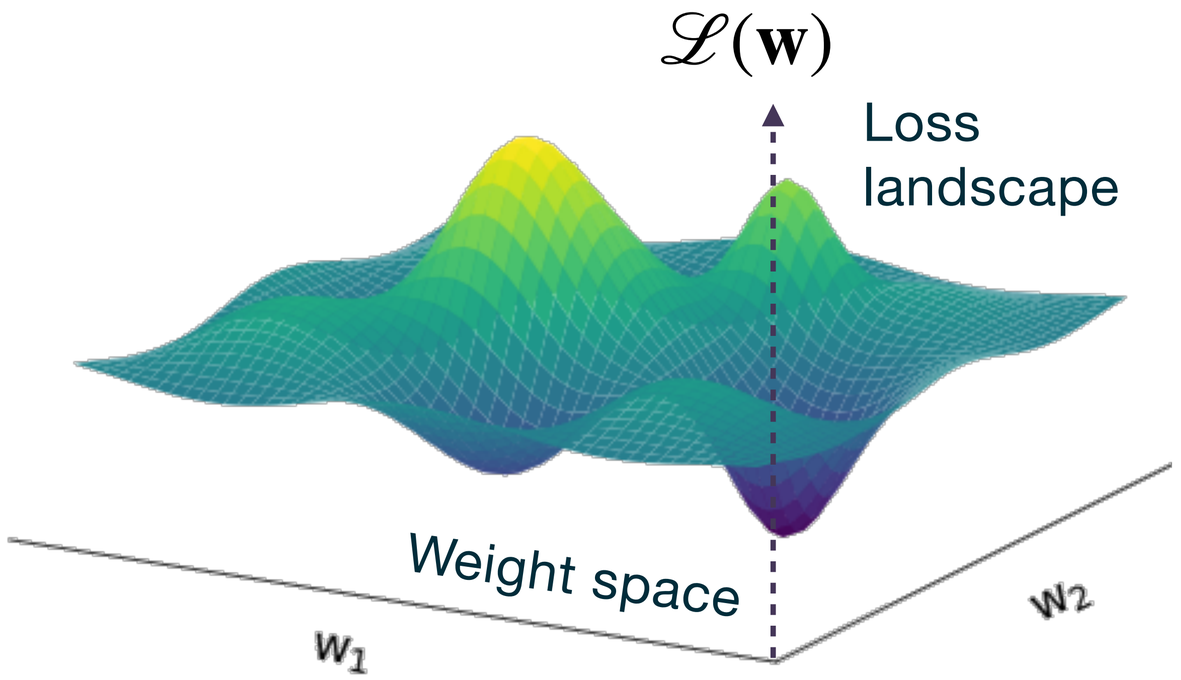
Three main ingredients

③ Algorithm

Three main ingredients

③ Algorithm

$$\mathscr{L}(\mathbf{w}) = \sum_{\mu=1}^{P} \ell\left(y_\mu, \mathbf{x}_\mu; \mathbf{w}\right) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2$$



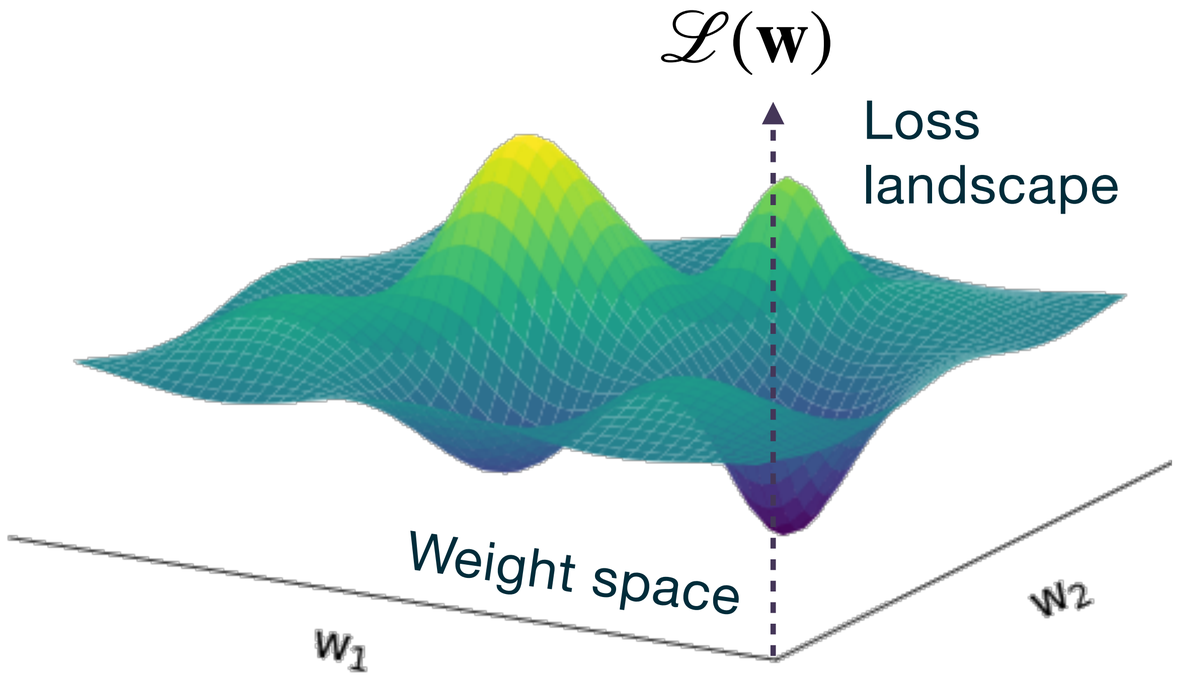
Three main ingredients

③ Algorithm

Regularization

$$\mathscr{L}(\mathbf{w}) = \sum_{\mu=1}^{P} \ell\left(y_\mu, \mathbf{x}_\mu; \mathbf{w}\right) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2$$

Training samples



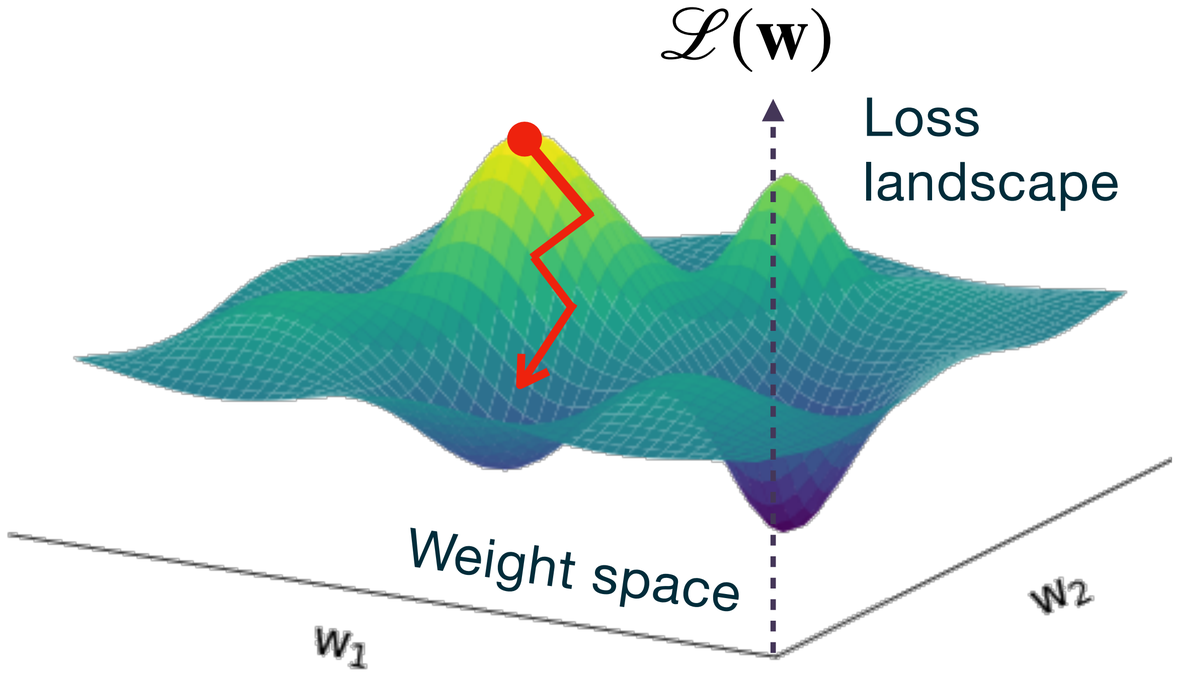
Three main ingredients

③ Algorithm

Regularization

$$\mathscr{L}(\mathbf{w}) = \sum_{\mu=1}^{P} \ell\left(y_\mu, \mathbf{x}_\mu; \mathbf{w}\right) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2$$

Training samples



e.g., ***gradient descent*** :

$$\mathbf{w}^{\tau+1} = \mathbf{w}^{\tau} - \eta \nabla_{\mathbf{w}} \mathscr{L}(\mathbf{w}^{\tau})$$

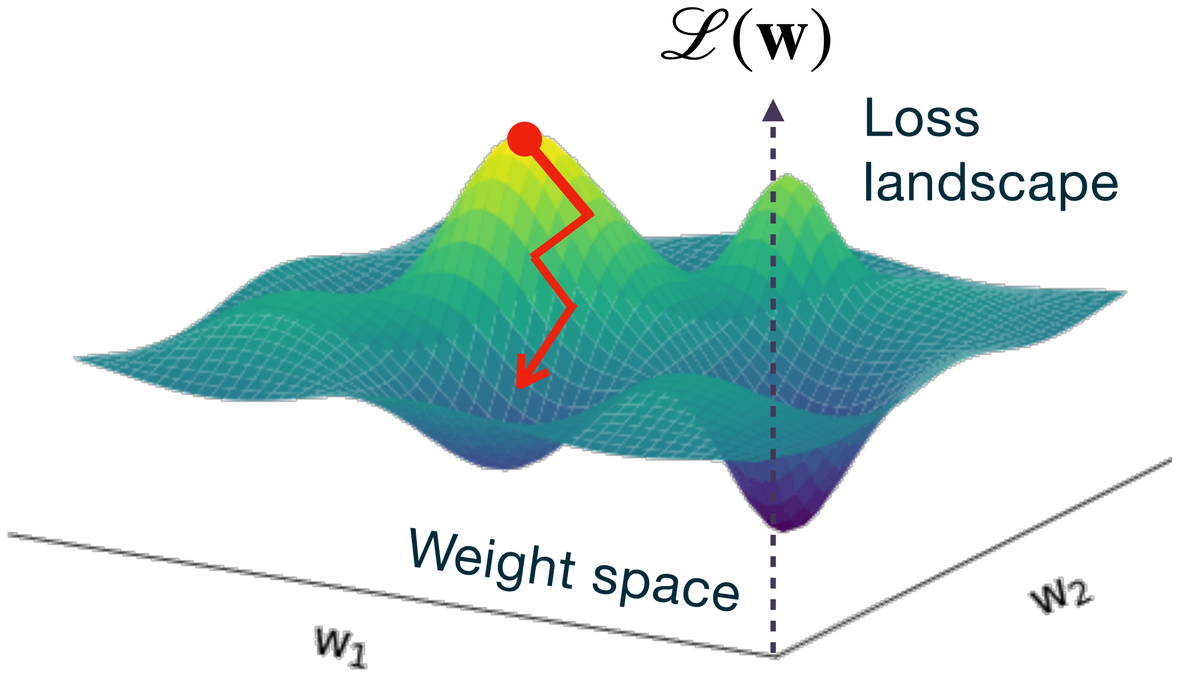
Three main ingredients

③ Algorithm

Regularization

$$\mathscr{L}(\mathbf{w}) = \sum_{\mu=1}^{P} \ell\left(y_\mu, \mathbf{x}_\mu; \mathbf{w}\right) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2$$

Training samples



e.g., ***gradient descent*** :

$$\mathbf{w}^{\tau+1} = \mathbf{w}^{\tau} - \eta \nabla_{\mathbf{w}} \mathscr{L}(\mathbf{w}^{\tau})$$

Learning rate
(hyper-parameter)

Three main ingredients

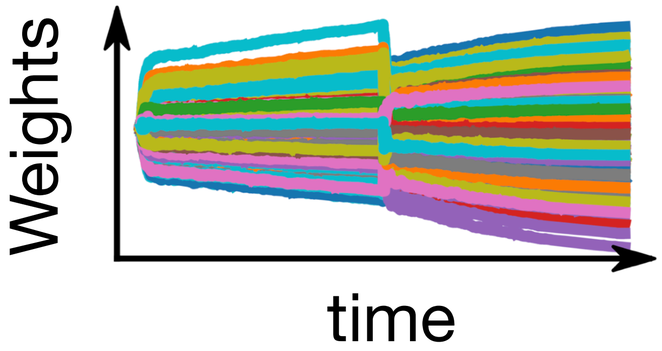
③ Algorithm

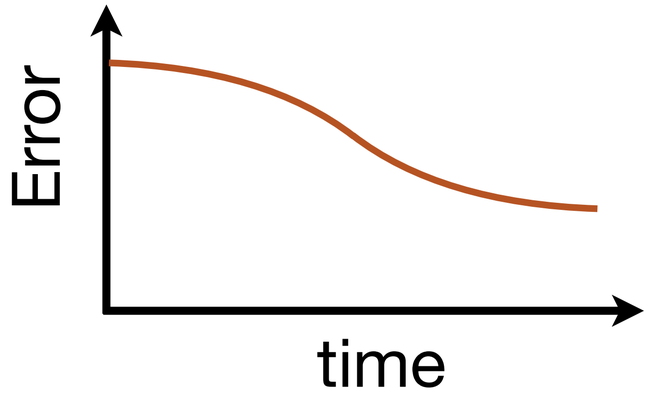
$$\mathscr{L}(\mathbf{w}) = \sum_{\mu=1}^{P} \ell\left(y_\mu, \mathbf{x}_\mu; \mathbf{w}\right) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2$$

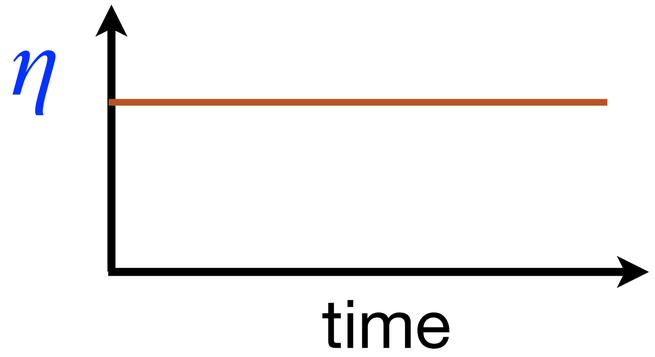
e.g., ***gradient descent*** :

$$\mathbf{w}^{\tau+1} = \mathbf{w}^{\tau} - \eta \nabla_{\mathbf{w}} \mathscr{L}(\mathbf{w}^{\tau})$$

Learning rate
(**hyper-parameter**)







Three main ingredients

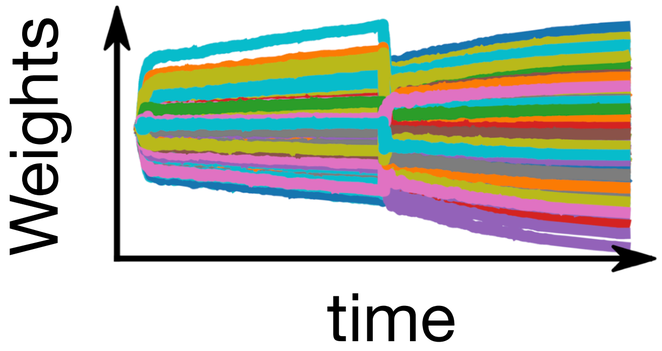
③ Algorithm

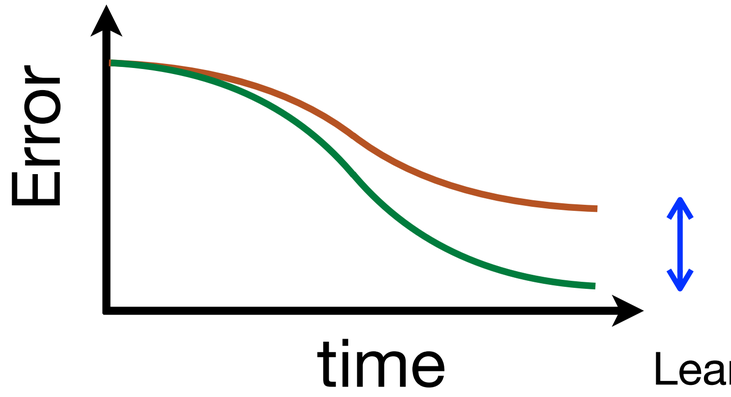
$$\mathscr{L}(\mathbf{w}) = \sum_{\mu=1}^{P} \ell\left(y_\mu, \mathbf{x}_\mu; \mathbf{w}\right) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2$$

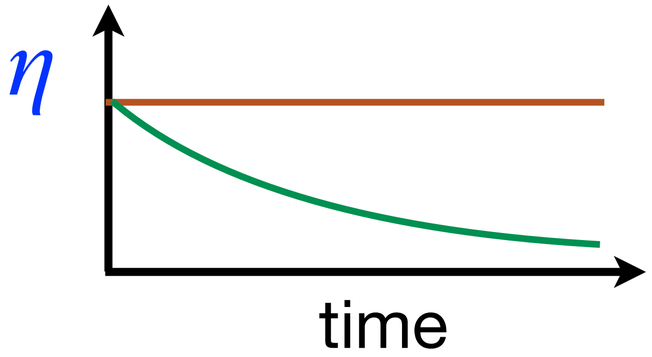
e.g., ***gradient descent*** :

$$\mathbf{w}^{\tau+1} = \mathbf{w}^{\tau} - \eta \nabla_{\mathbf{w}} \mathscr{L}(\mathbf{w}^{\tau})$$

Learning rate
(hyper-parameter)



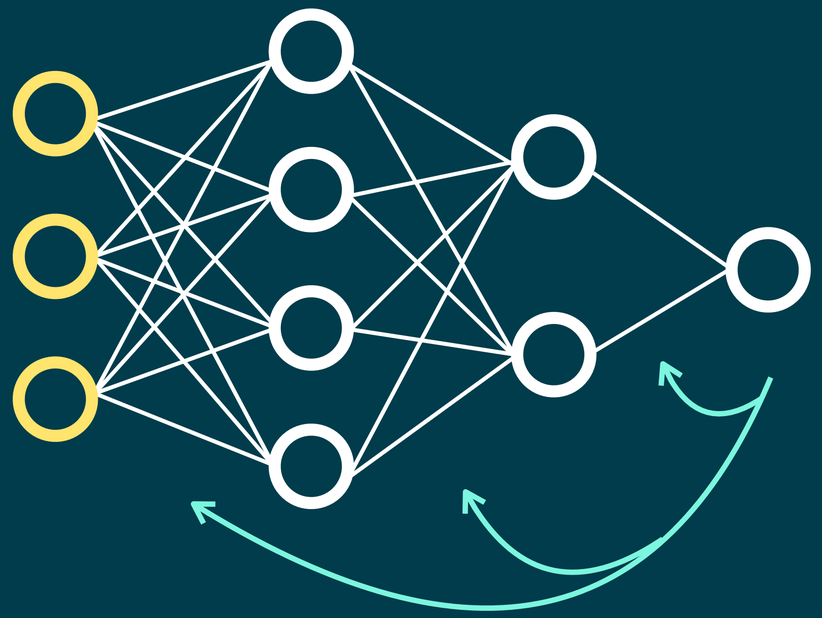




Learning rate **schedules** drive the dynamics towards better solutions

Learning protocols

Architecture



Data / Task

Optimization
Algorithm

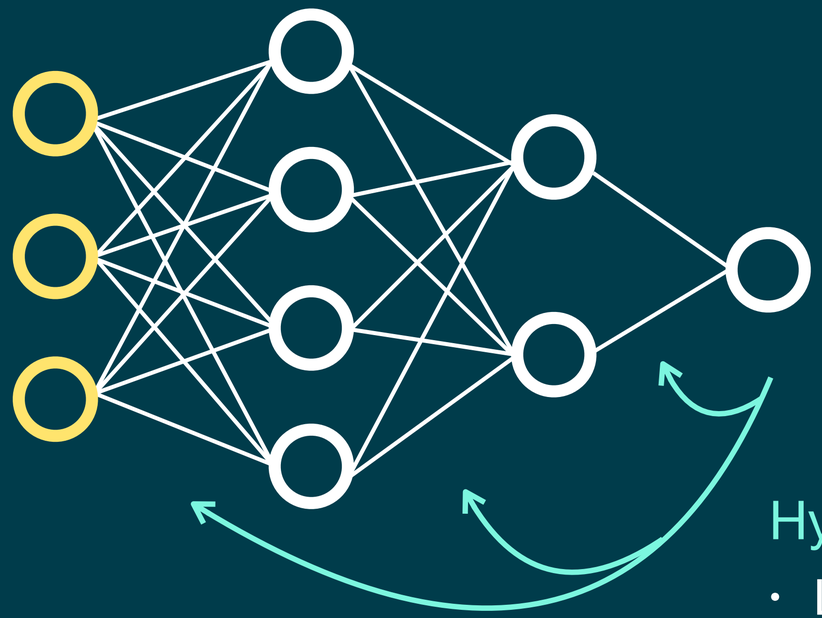
Learning protocols

Architecture

Model optimization:

- Pruning
- Gating
- Dropout
- …

Data / Task



Dynamic data / task selection:

- Active learning
- Curriculum learning
- Task ordering
- Data imbalance
- …

Optimization Algorithm

Hyper-parameter schedules:

- Learning rate
- Momentum
- Batch size
- Regularization
- …

Learning protocols

Goals:

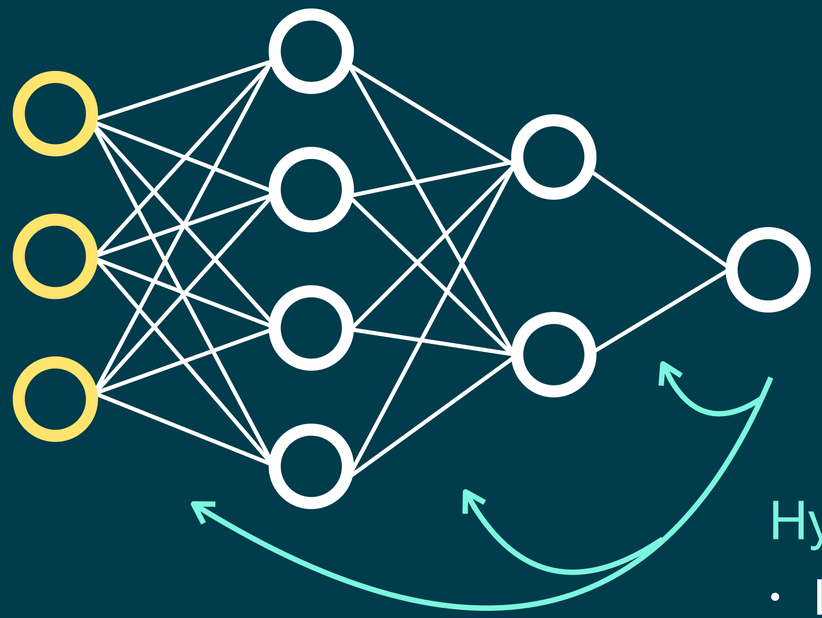
- Speed-up convergence
- Guide the training towards better regions of parameters space

Architecture

Model optimization:

- Pruning
- Gating
- Dropout
- …

Data / Task



Optimization Algorithm

Dynamic data / task selection:

- Active learning
- Curriculum learning
- Task ordering
- Data imbalance
- …

Hyper-parameter schedules:

- Learning rate
- Momentum
- Batch size
- Regularization
- …

*Optimal learning protocols ?

* in the final performance

Goals:

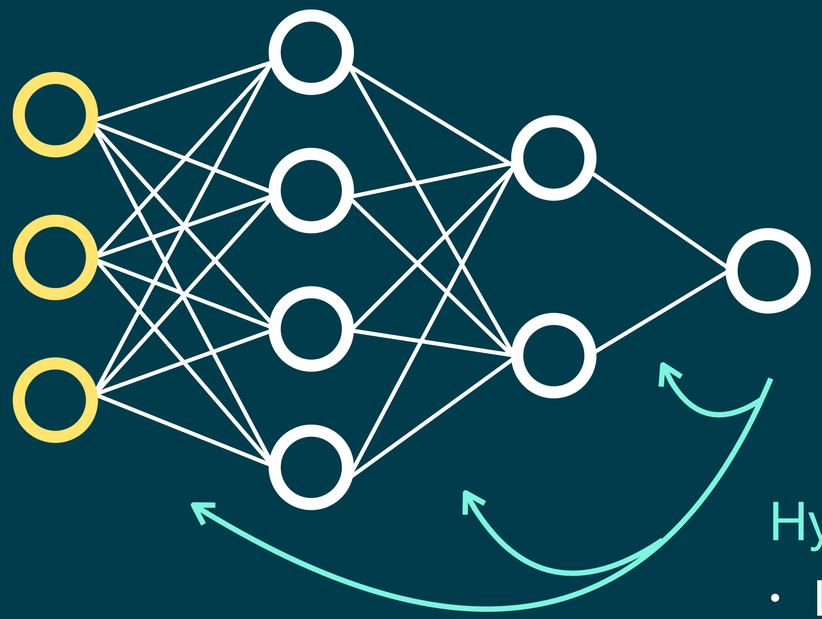
- Speed-up convergence
- Guide the training towards better regions of parameters space

Architecture

Model optimization:

- Pruning
- Gating
- Dropout
- …

Data / Task



Dynamic data / task selection:

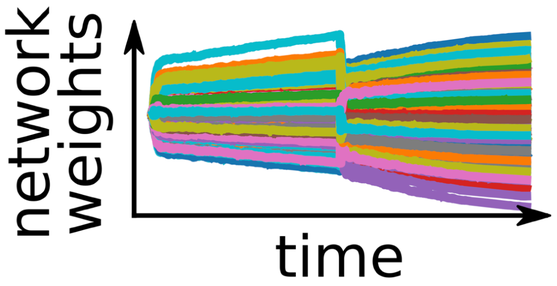
- Active learning
- Curriculum learning
- Task ordering
- Data imbalance
- …

Optimization Algorithm

Hyper-parameter schedules:

- Learning rate
- Momentum
- Batch size
- Regularization
- …

Statistical physics of optimal learning

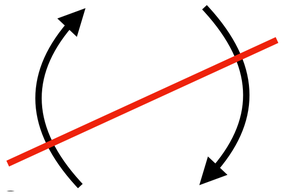


$$\mathbf{w}^{\mu+1} = \mathbf{w}^{\mu} - \nabla \mathscr{L}^{\mu}(\mathbf{u}^{\mu})$$

$$\mathbf{w} \in \mathbb{R}^{N}$$

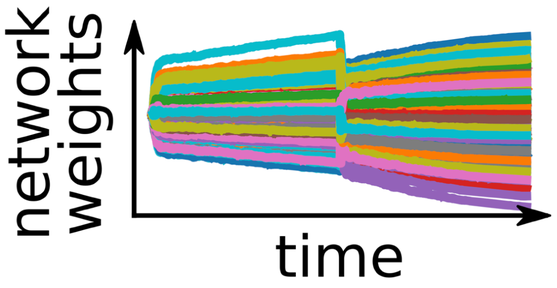
Protocols = control variables

High-dimensional learning dynamics



Control theory is computationally demanding

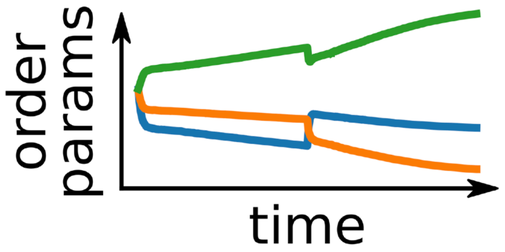
Statistical physics of optimal learning



$$\mathbf{w}^{\mu+1} = \mathbf{w}^{\mu} - \nabla \mathscr{L}^{\mu}(\mathbf{u}^{\mu})$$

$$\mathbf{w} \in \mathbb{R}^N$$

Protocols = control variables



$$\dot{\mathbf{Q}} = f(\mathbf{Q}, \mathbf{u})$$

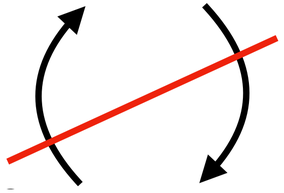
High-dimensional learning dynamics

Statistical Physics

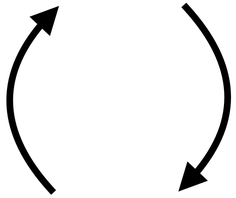
$N \to \infty$

Low-dimensional effective dynamics of order parameters

[Saad & Solla (1995); Biehl & Schwarze (1995); …; Goldt et al (2019); Veiga et al (2022); … and many more]



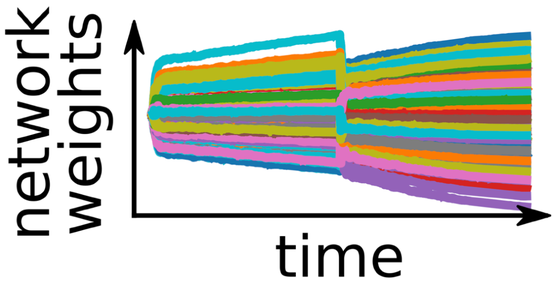
Control theory is computationally demanding



Control theory can be applied

\+ interpretability

Statistical physics of optimal learning

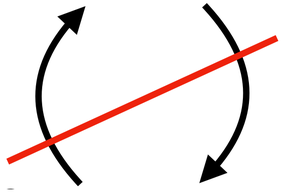


$$\mathbf{w}^{\mu+1} = \mathbf{w}^{\mu} - \nabla \mathscr{L}^{\mu}(\mathbf{u}^{\mu})$$

$$\mathbf{w} \in \mathbb{R}^N$$

Protocols = control variables

High-dimensional learning dynamics

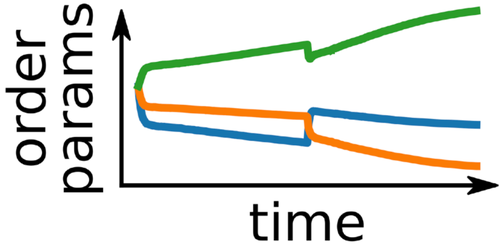


Control theory is computationally demanding

Statistical Physics

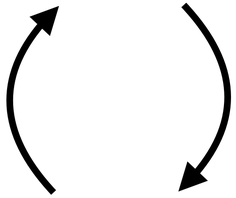
$$N \to \infty$$

[Saad & Solla (1995); Biehl & Schwarze (1995); …; Goldt et al (2019); Veiga et al (2022); … and many more]



$$\dot{\mathbf{Q}} = f(\mathbf{Q}, \mathbf{u})$$

Low-dimensional effective dynamics of order parameters



Control theory can be applied

\+ interpretability

Optimal Control

$\mathbf{u}^*$

[Pontryagin (1962), Bellman (1965); Saad & Rattray (1997)]

FM, F. Mori, PNAS *Nexus* 2026



Francesco Mori
Harvard University



Stefano Sarao Mannelli
Chalmers University
Gothenburg University

Understanding optimal training protocols:

Optimal protocols for continual learning via statistical physics and control theory,
F. Mori, S. Sarao Mannelli, FM, ICLR 2025.

Analytic theory of dropout regularization,
F. Mori, FM, *Phys. Rev. E* 2025.

A statistical physics framework for optimal learning,
FM, F. Mori, PNAS *Nexus* 2026.

Continual learning

Continual learning

Task 1

Task 2

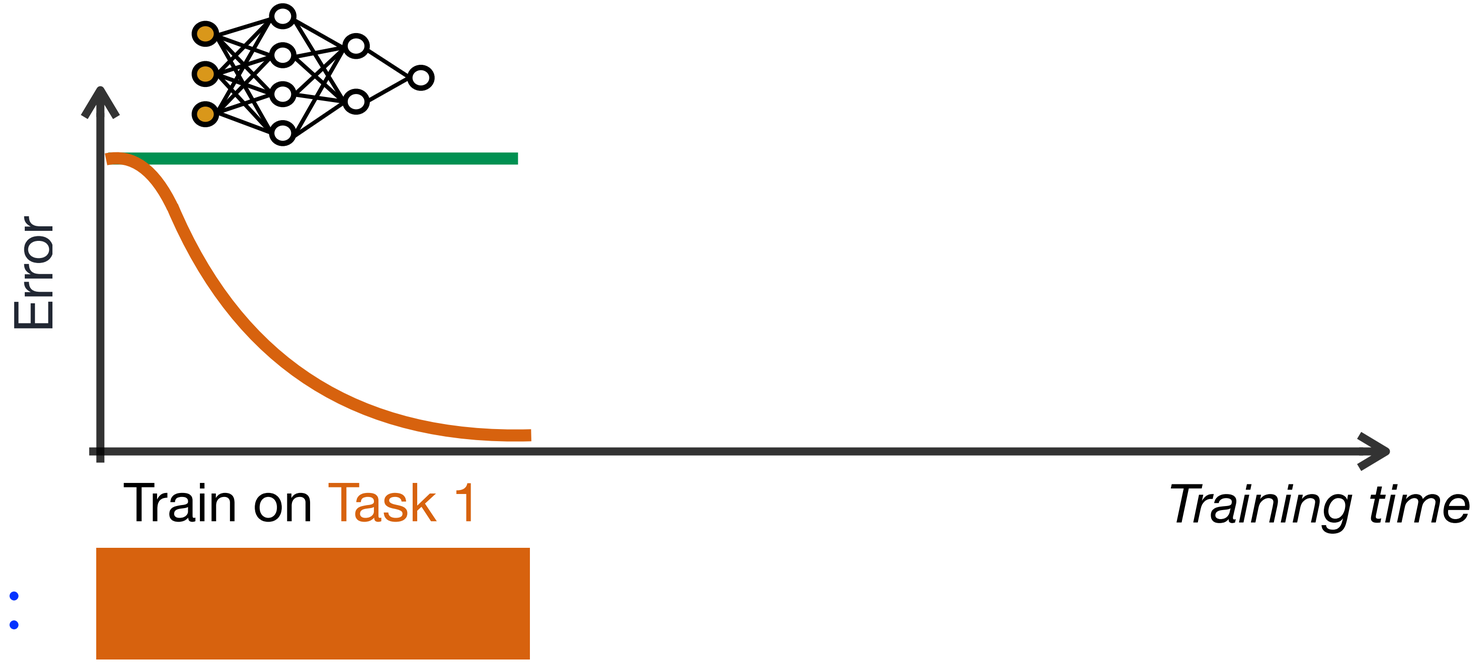


Task protocol u :

Continual learning

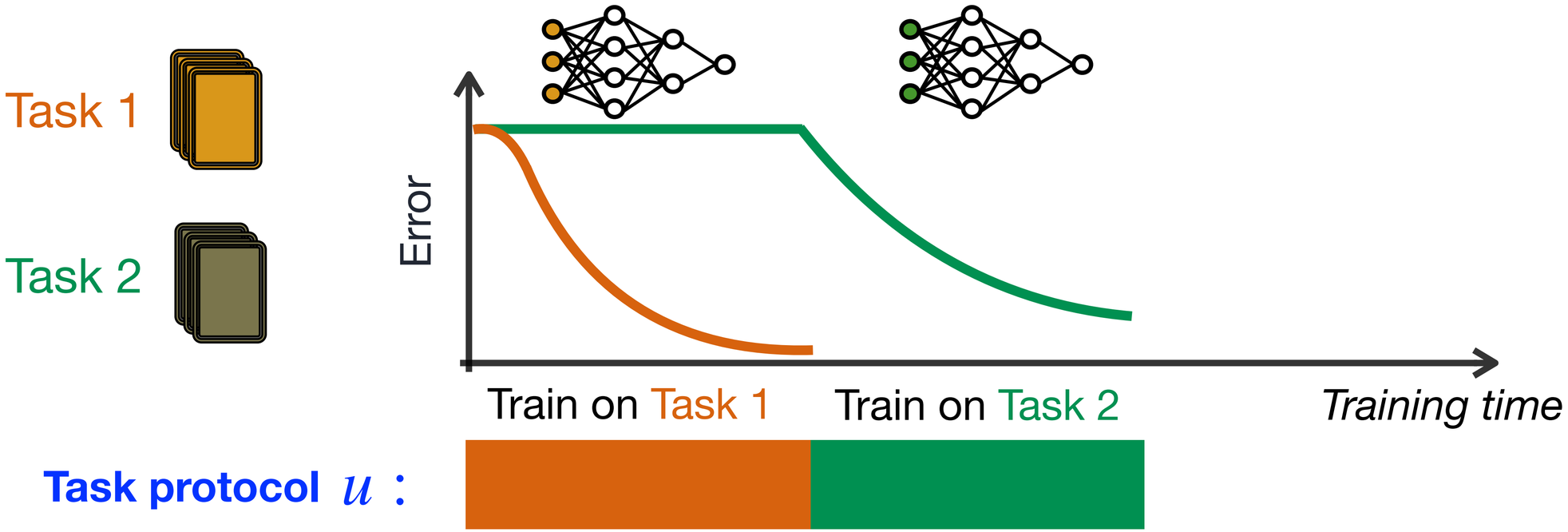
Task 1

Task 2



Task protocol u **:**

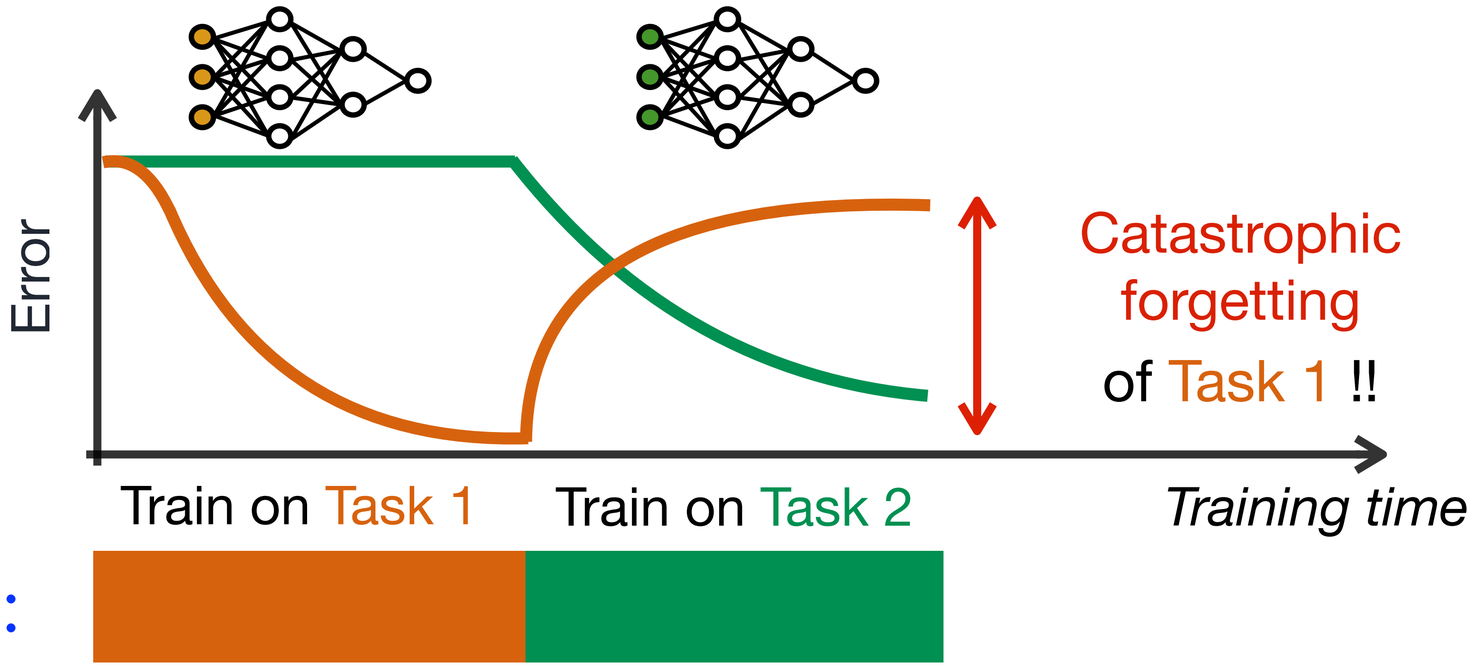
Continual learning



Continual learning

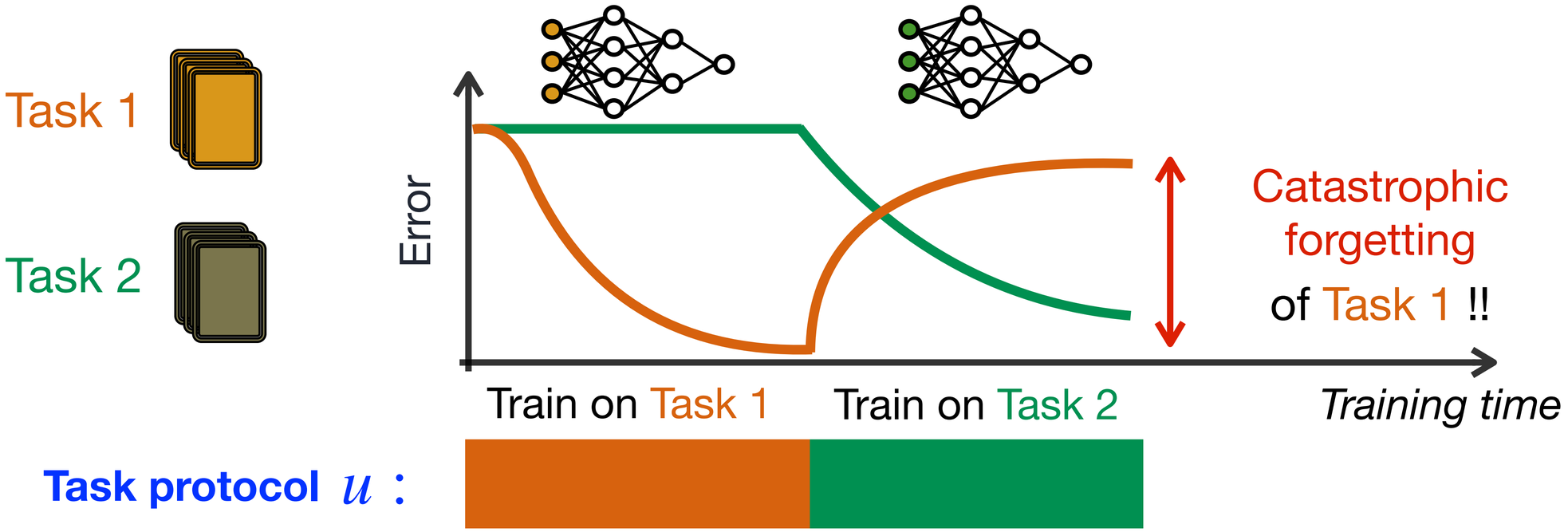
Task 1

Task 2



Task protocol u **:**

Continual learning



Humans & Animals

[McClelland et al (1995); Barnett & Ceci (2002); Calvert et al., (2004); Mareschal et al (2007); Pallier et al (2003); … Flesch et al (2018); Cichon & Gan (2015); Yang et al., 2014) …]

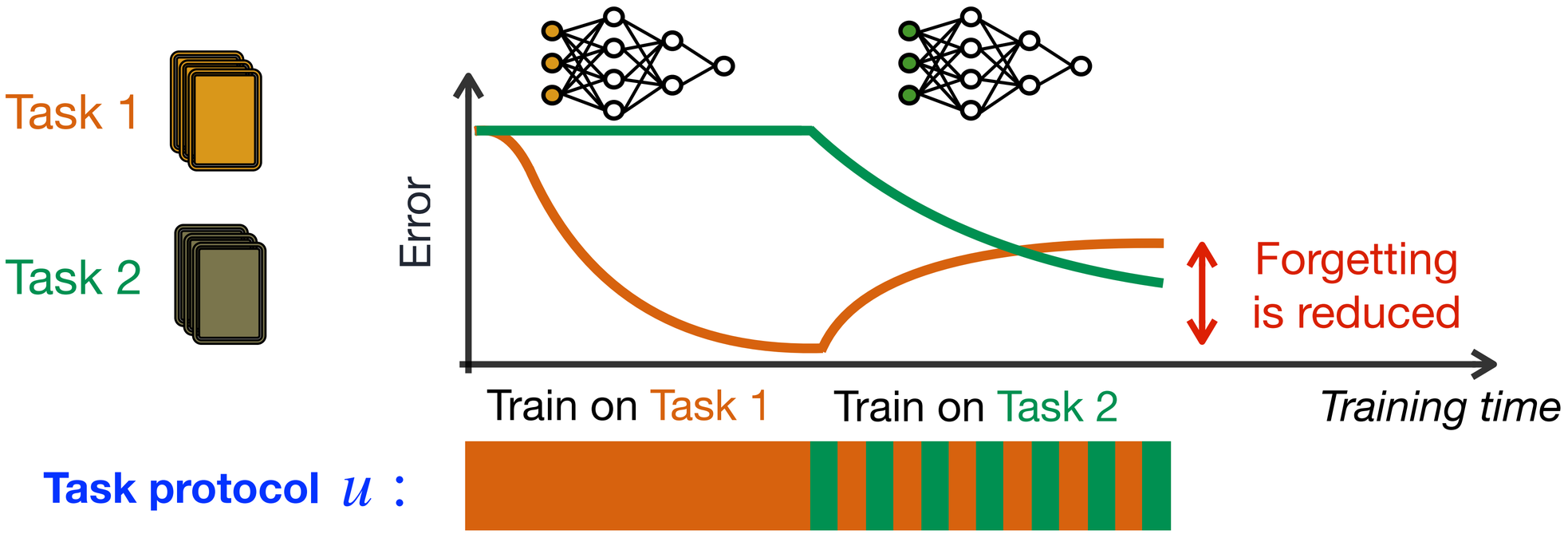
ML (empirical):

[Goodfellow et al (2014); Ruder & Planck (2014); Nguyen et al (2019); Parisi et al (2019); Mirzadeh et al (2020); Neyshabur et al (2020)]

ML (theory):

[Mirzadeh et al (2021); Lee et al. (2021, 2022); Asanuma et al (2021); Doan et al (2021); Evron et al (2022, 2023); Shan et al (2024)]

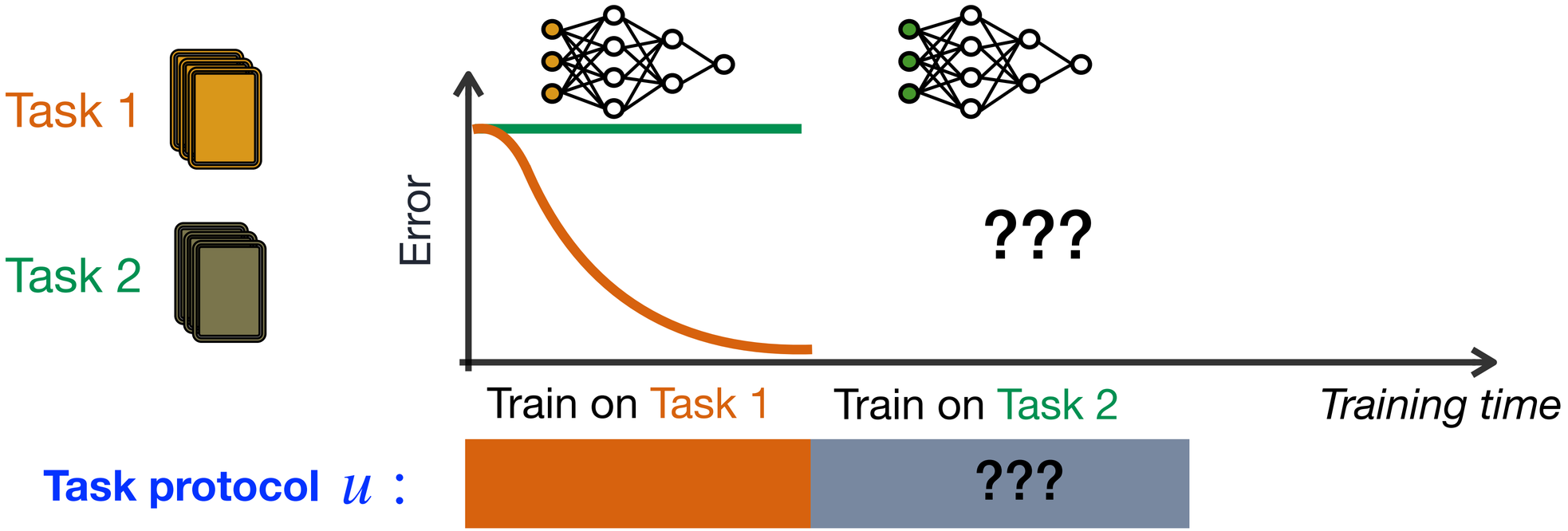
Continual learning



(Heuristic) interleaved *replay* strategy

[Shin et al., (2017); Draelos et al., (2017); Rolnick et al., (2019)]

Continual learning



What is the ***optimal*** replay protocol?

A teacher-student model for continual learning

Introduced in: *Lee, Goldt, & Saxe (ICML 2021)*

Task 1

$$\mathscr{D}_1 = \{\mathbf{x}_\mu^{(1)}, y_\mu^{(1)}\}_{\mu=1}^{P}$$

$$\mathbf{x}_\mu^{(1)} \sim \mathcal{N}(0\,, I_N) \in \mathbb{R}^N$$

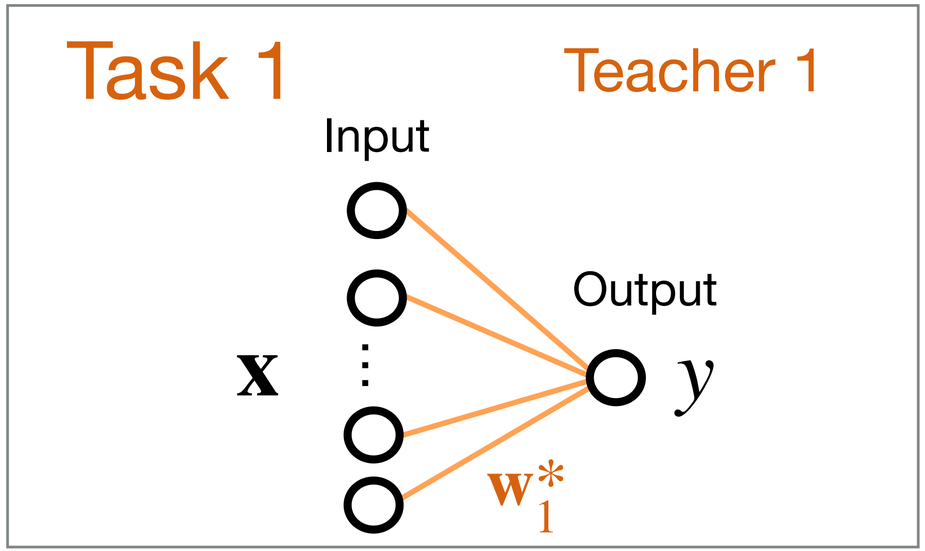
Task 2

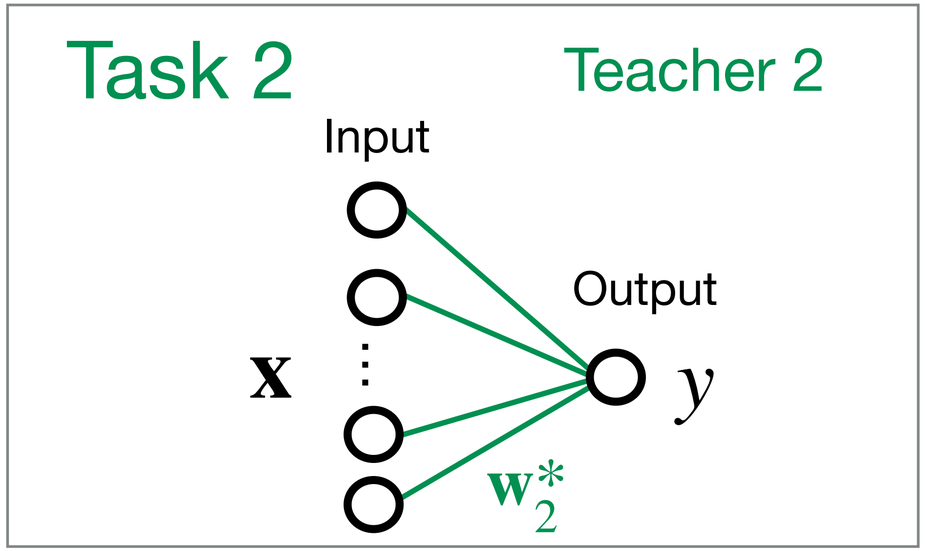
$$\mathscr{D}_2 = \{\mathbf{x}_\mu^{(2)}, y_\mu^{(2)}\}_{\mu=1}^{P}$$

$$\mathbf{x}_\mu^{(2)} \sim \mathcal{N}(0\,, I_N) \in \mathbb{R}^N$$

A teacher-student model for continual learning

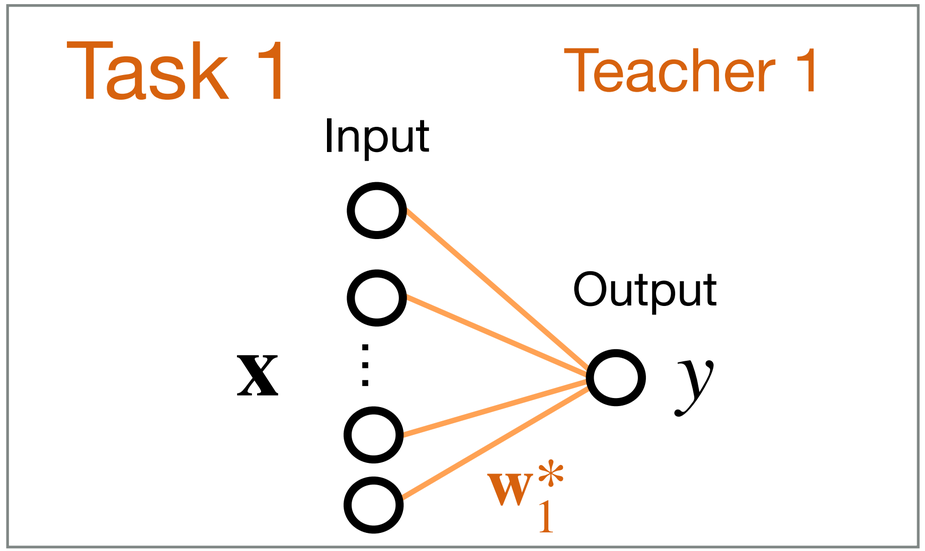
Introduced in: *Lee, Goldt, & Saxe (ICML 2021)*

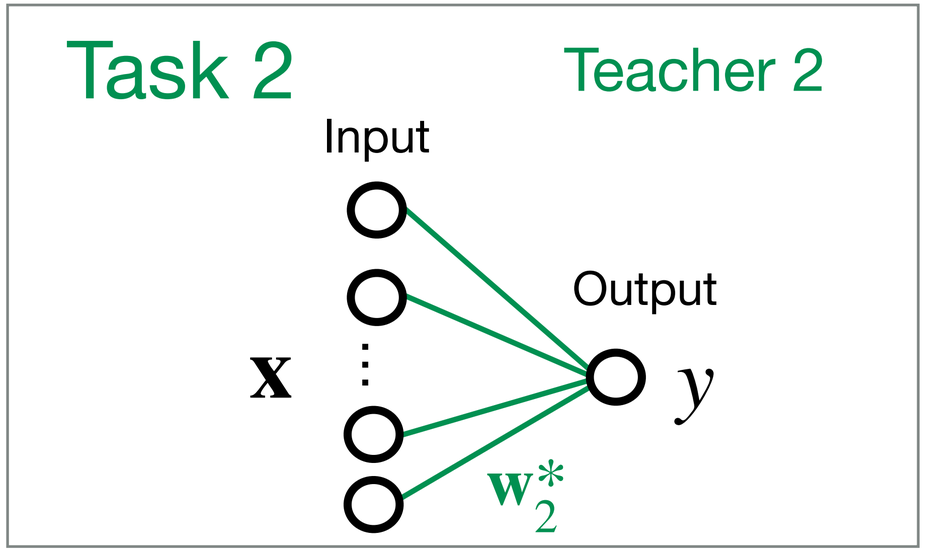


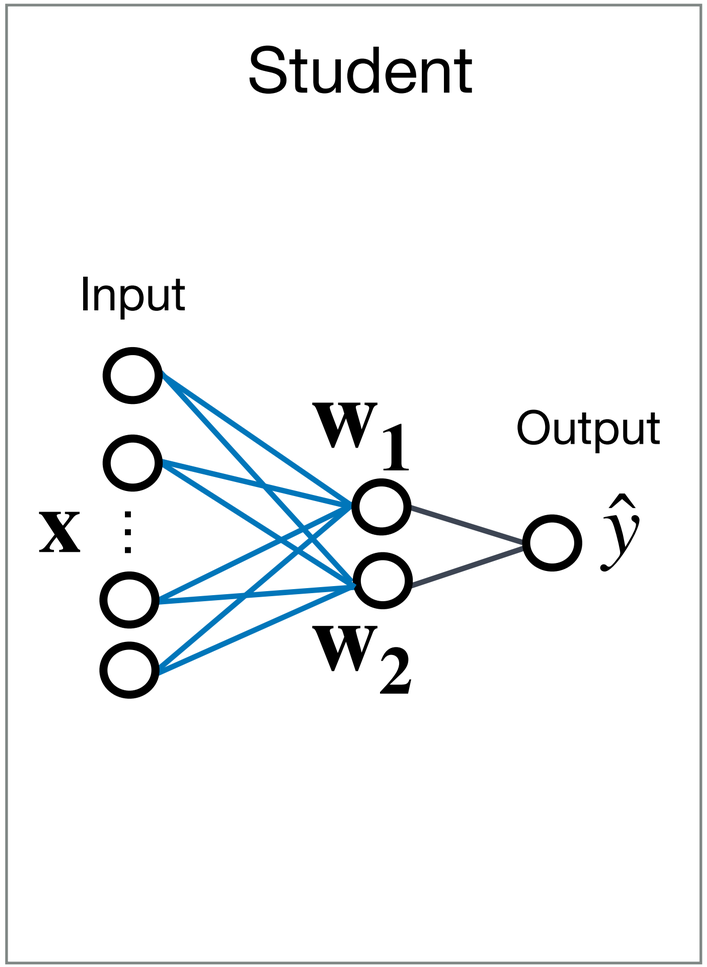


A teacher-student model for continual learning

Introduced in: *Lee, Goldt, & Saxe (ICML 2021)*

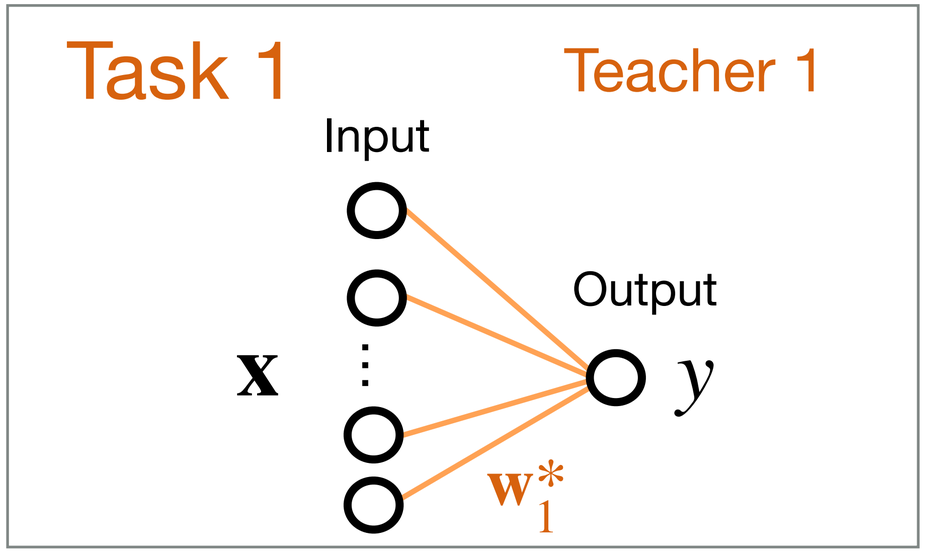


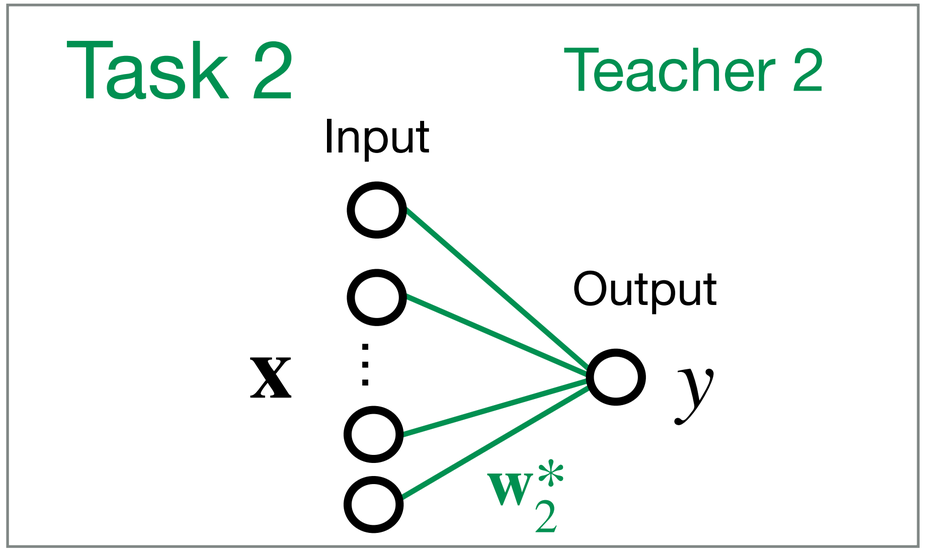


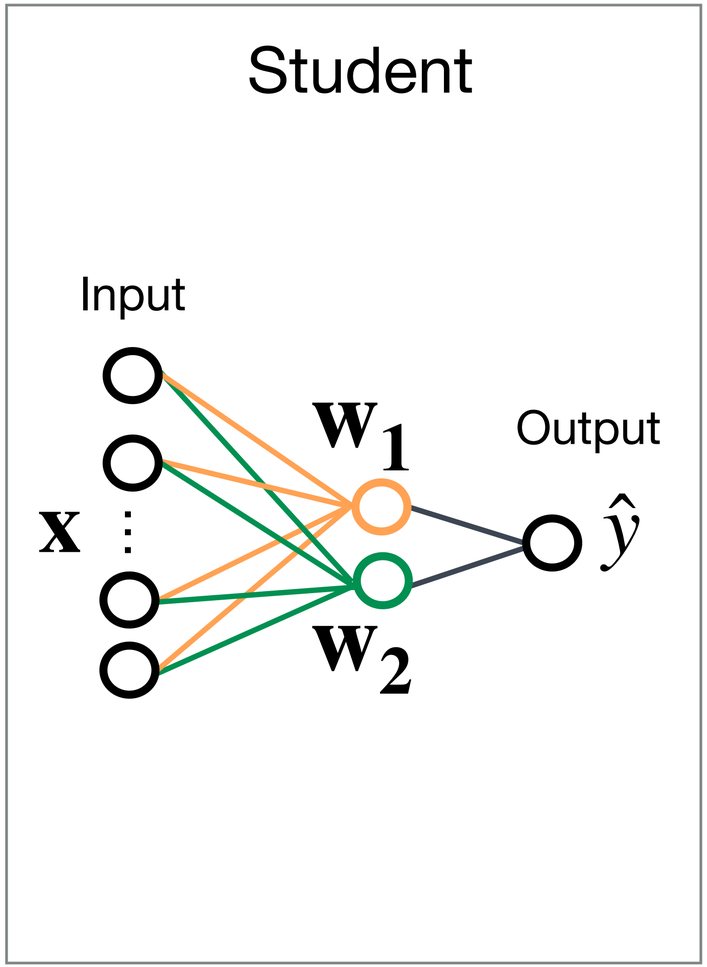


A teacher-student model for continual learning

Introduced in: *Lee, Goldt, & Saxe (ICML 2021)*



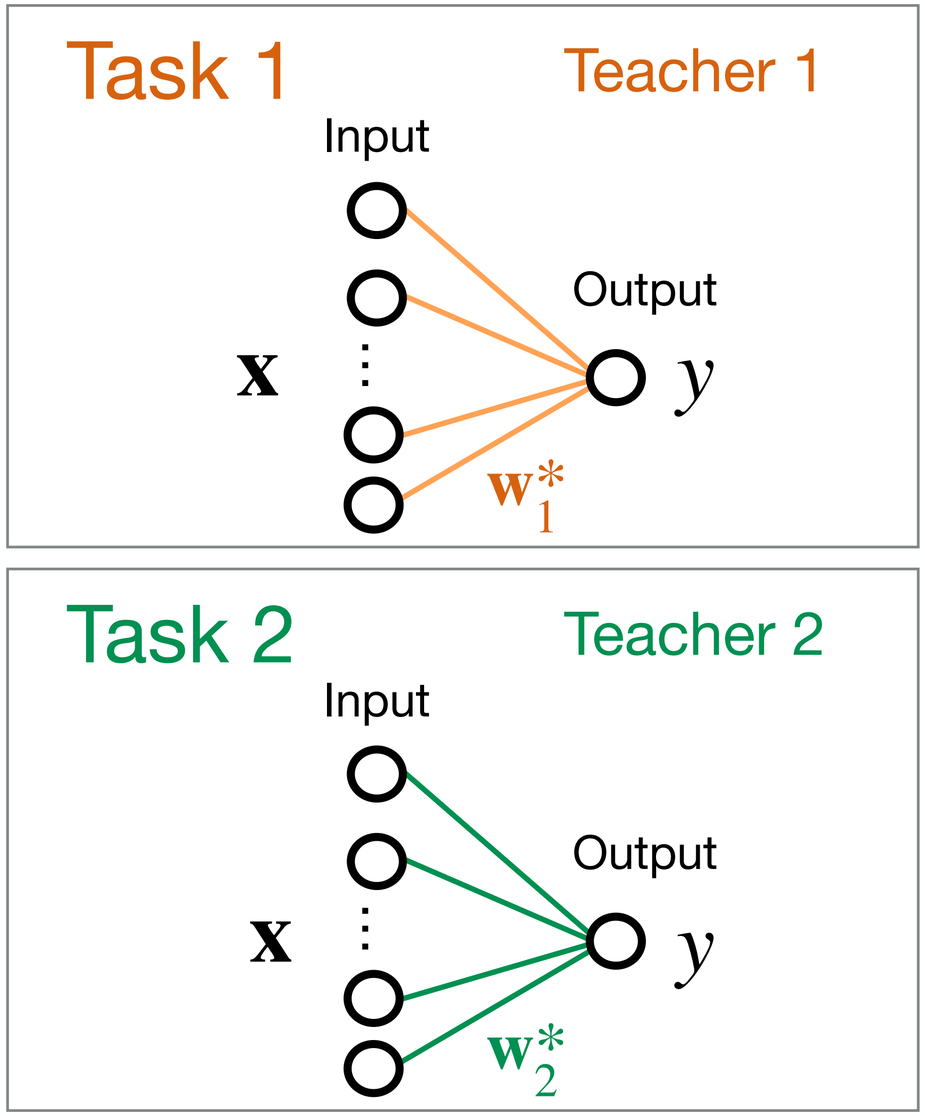


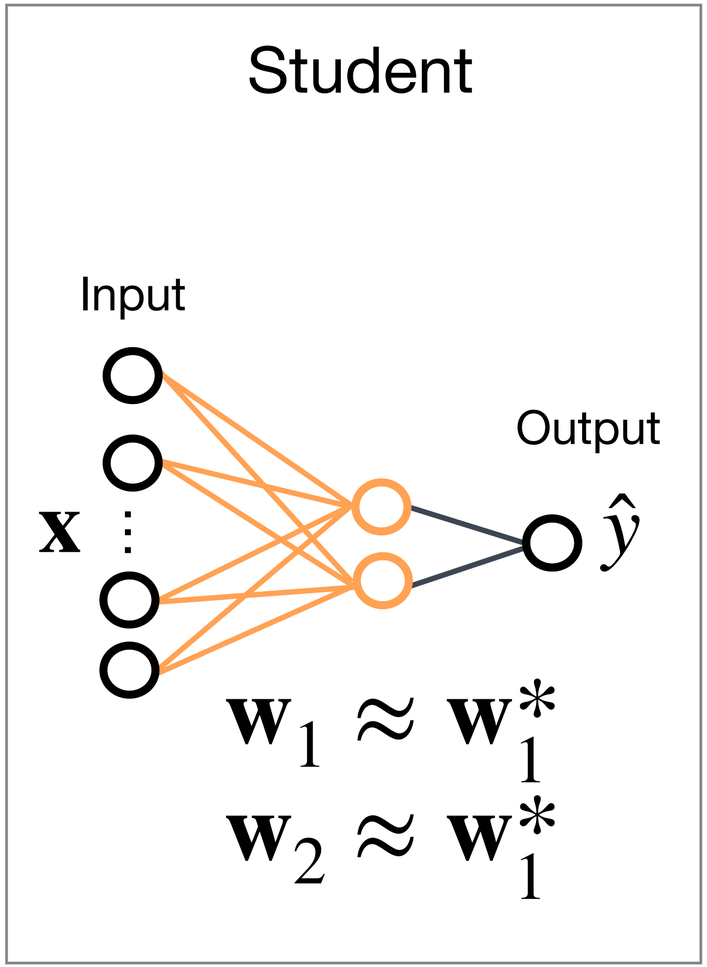


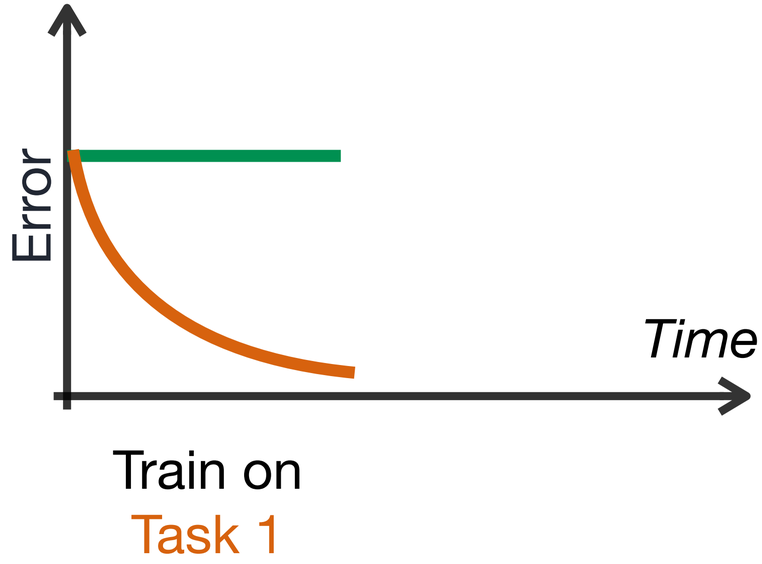
The student has the capacity to learn both tasks.

A teacher-student model for continual learning

Introduced in: *Lee, Goldt, & Saxe (ICML 2021)*

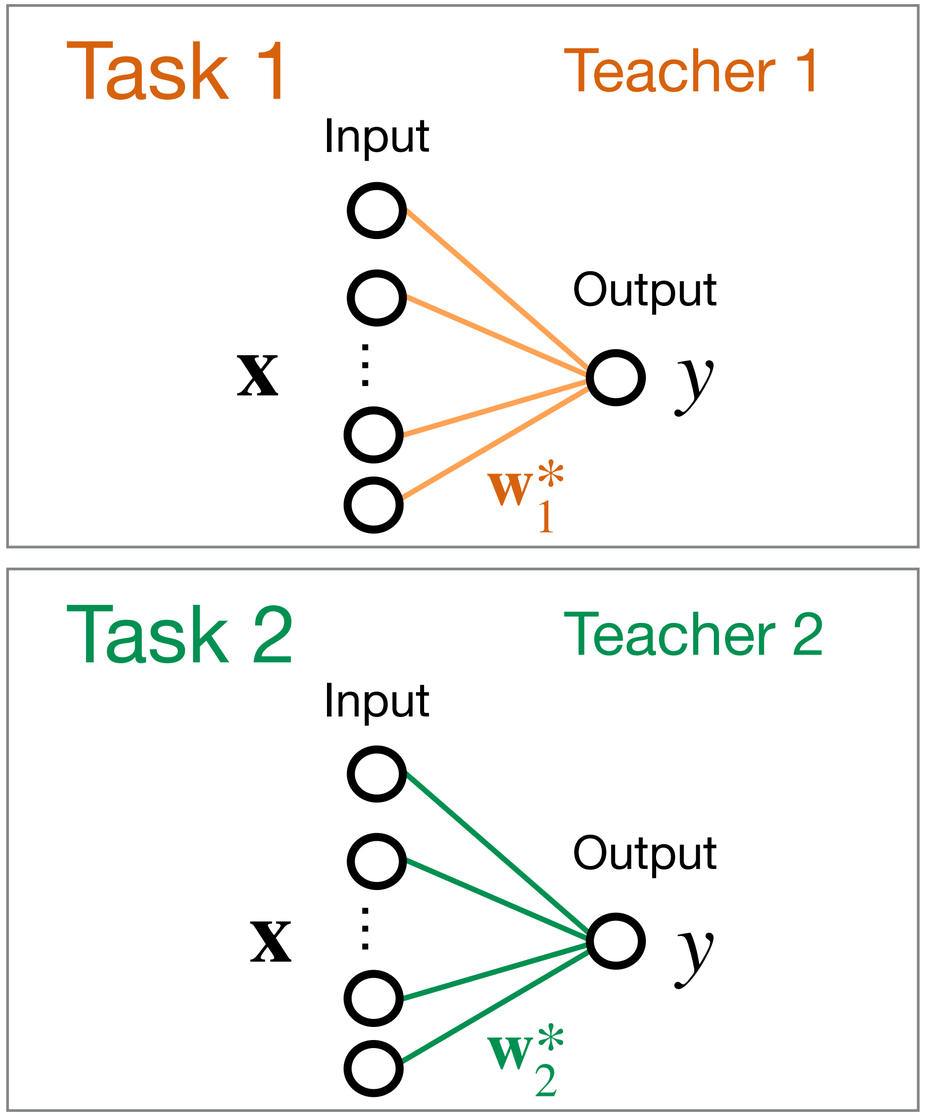


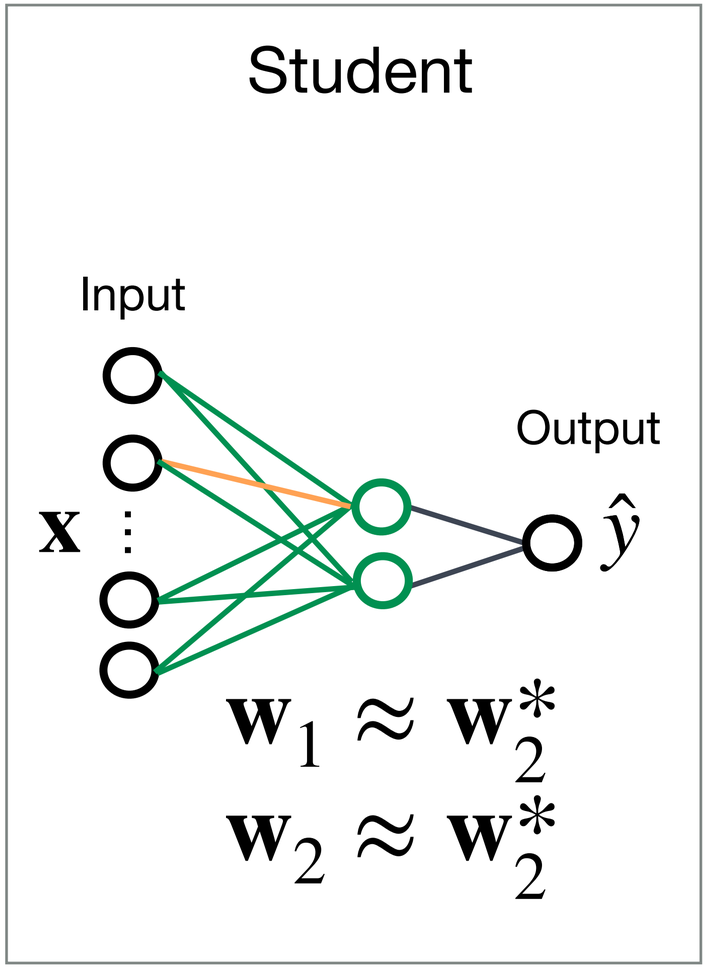


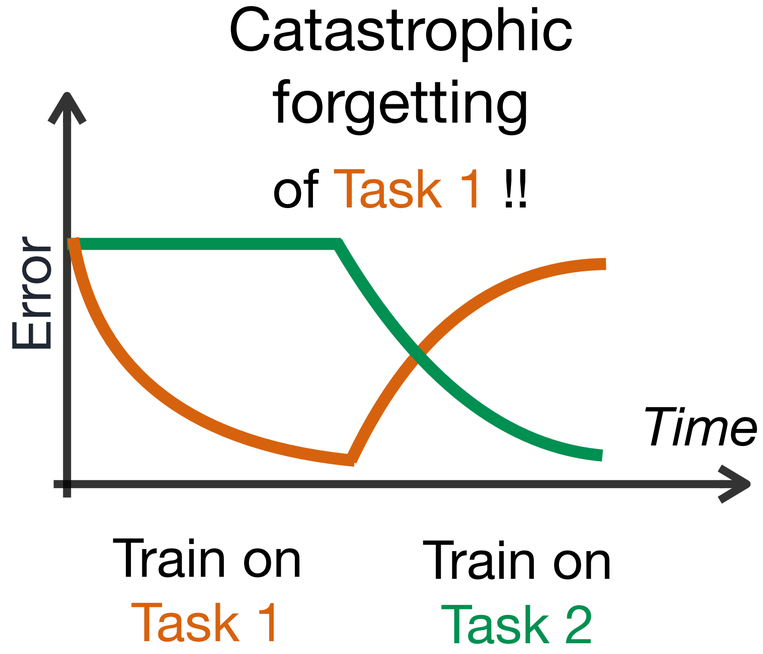


A teacher-student model for continual learning

Introduced in: *Lee, Goldt, & Saxe (ICML 2021)*





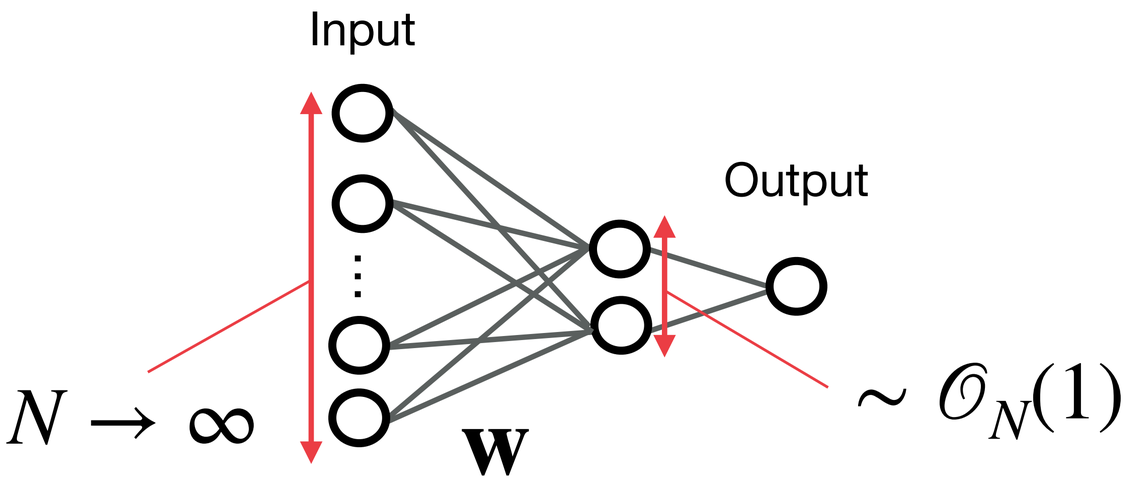


Online SGD dynamics of two-layer networks

High-dimensional stochastic equation

Low-dimensional ordinary differential equations

$$\mathbf{w}^{\mu+1} = \mathbf{w}^{\mu} - \eta \nabla_{\mathbf{w}} \mathscr{L}(\mathbf{u}_{\mu}) \xrightarrow[\alpha = \mu/N]{N, \mu \to \infty} \frac{\mathrm{d}\mathbf{Q}}{\mathrm{d}\alpha} = f(\mathbf{Q}, \mathbf{u})$$



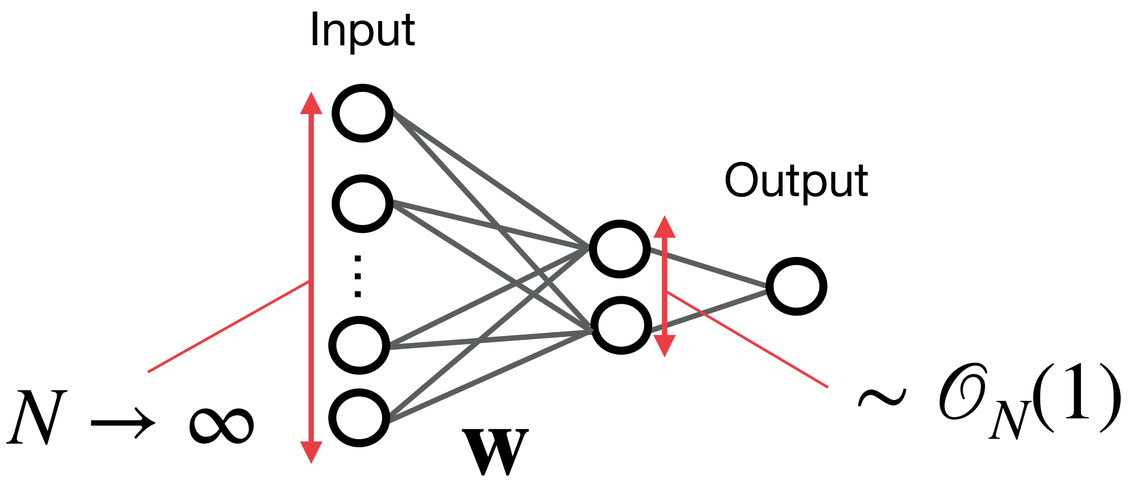
D. Saad, S. A. Solla, PRL (1995).
M Biehl and H Schwarze, J. Phys. A (1995).

Online SGD dynamics of two-layer networks

High-dimensional stochastic equation

Low-dimensional ordinary differential equations

$$\mathbf{w}^{\mu+1} = \mathbf{w}^{\mu} - \eta \nabla_{\mathbf{w}} \mathscr{L}(\mathbf{u}_{\mu}) \xrightarrow[\alpha = \mu/N]{N, \mu \to \infty} \frac{\mathrm{d}\mathbf{Q}}{\mathrm{d}\alpha} = f(\mathbf{Q}, \mathbf{u})$$



The prediction error depends on the weights only through the order parameters:

$$\boldsymbol{\mathcal{E}}_g^{(t)}(\mathbf{w}) = \mathbb{E}_{\mathbf{x},y\sim\mathscr{D}_t}\left[\left(y - \hat{y}_{\mathbf{w}}(\mathbf{x})\right)^2\right]$$

$$= \epsilon_g^{(t)}(\mathbf{Q}) \qquad \text{(Task: } t = 1,2\text{)}$$

D. Saad, S. A. Solla, PRL (1995).
M Biehl and H Schwarze, J. Phys. A (1995).

Online SGD dynamics of two-layer networks

High-dimensional stochastic equation

$$\mathbf{w}^{\mu+1} = \mathbf{w}^{\mu} - \eta \nabla_{\mathbf{w}} \mathscr{L}(\mathbf{u}_{\mu})$$

$$\xrightarrow[\alpha = \mu/N]{N, \mu \to \infty}$$

Low-dimensional ordinary differential equations

$$\frac{\mathrm{d}\mathbf{Q}}{\mathrm{d}\alpha} = f(\mathbf{Q}, \mathbf{u})$$

$\mathbf{w}_t^* =$ Teacher vectors

(Task: $t = 1,2$)

$\mathbf{w}_i =$ Student vectors

(Node: $i = 1,2$)

Order parameters: $\mathbf{Q} = (\mathbf{M}, \mathbf{C})$

"Magnetization": $\mathbf{M}_{it} = \mathbf{w}_i^{\top} \mathbf{w}_t^*$

Self-overlap: $\mathbf{C}_{jk} = \mathbf{w}_i^{\top} \mathbf{w}_k$

D. Saad, S. A. Solla, PRL (1995).
M Biehl and H Schwarze, J. Phys. A (1995).

Online learning in narrow networks

… several architectures are described by variants of this model:

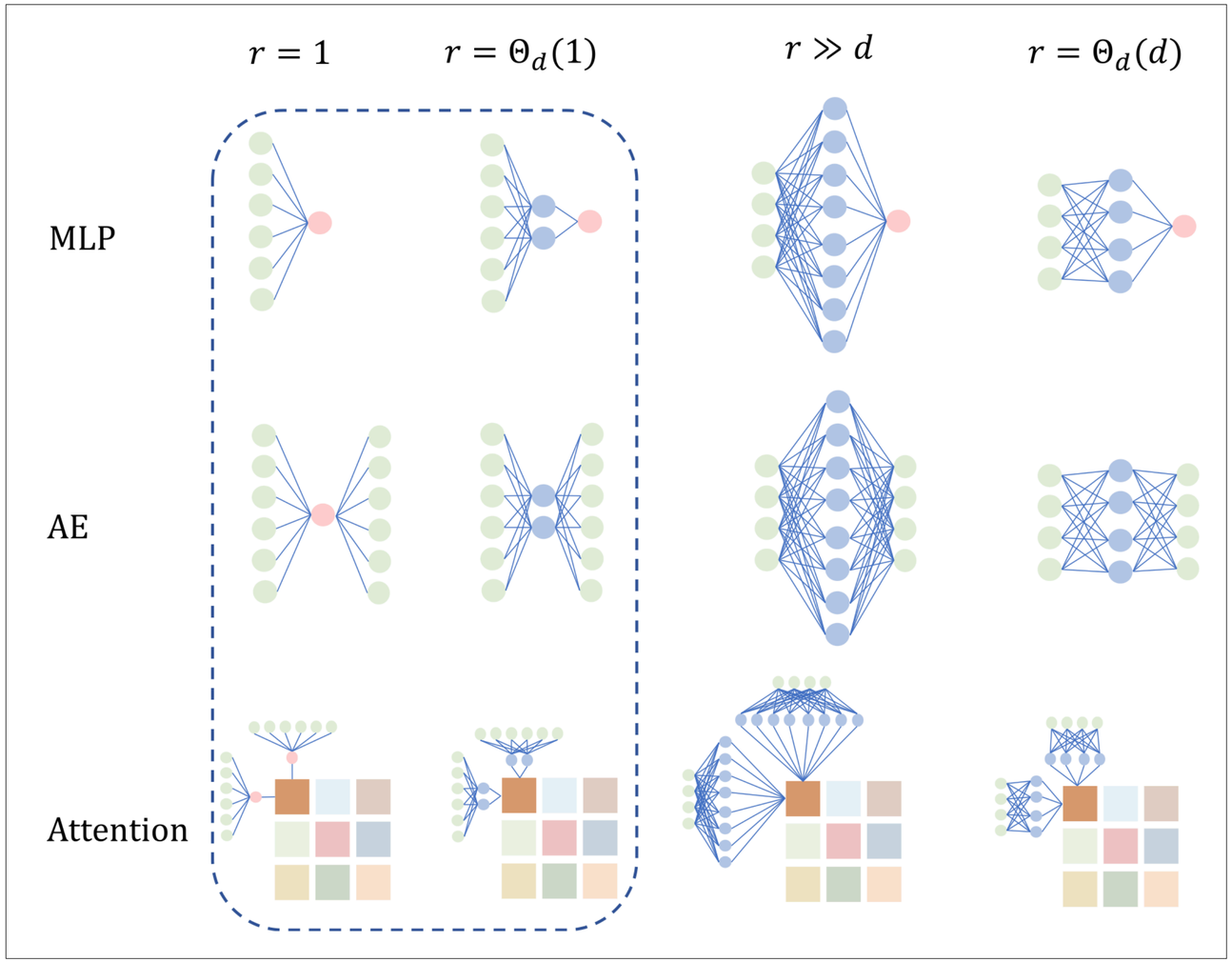


Figure from:

Hugo Cui, *J. Stat. Mech.* (2025) 023402

See also: **FM, F. Mori, *PNAS Nexus* (2026)** for the derivation of online SGD equations.

Pontryagin’s maximum principle

Forward dynamics

$$\frac{\mathrm{d}\mathbf{Q}}{\mathrm{d}\alpha} = f(\mathbf{Q}, \mathbf{u})$$

Order parameters

Control variable

Pontryagin's maximum principle

Cost functional:

$$\mathcal{F}[\mathbf{Q}, \mathbf{u}] = \epsilon_g(\mathbf{Q}(\alpha_F)) + \int_0^{\alpha_F} \mathrm{d}\alpha\, \phi\left(\mathbf{Q}(\alpha), \mathbf{u}(\alpha)\right)$$

Running cost

Forward dynamics

$$\frac{\mathrm{d}\mathbf{Q}}{\mathrm{d}\alpha} = f(\mathbf{Q}, \mathbf{u})$$

Order parameters

Control variable

Pontryagin's maximum principle

Cost functional:

$$\mathcal{F}[\mathbf{Q}, \mathbf{\Lambda}, \mathbf{u}, \nu] = \epsilon_g(\mathbf{Q}(\alpha_F)) + \int_0^{\alpha_F} \mathrm{d}\alpha\, \phi\left(\mathbf{Q}(\alpha), \mathbf{u}(\alpha)\right) + \int_0^{\alpha_F} \mathrm{d}\alpha\, \mathbf{\Lambda}(\alpha) \cdot \left[-\frac{\mathrm{d}\mathbf{\Omega}}{\mathrm{d}\alpha} + f(\mathbf{Q}, \mathbf{u})\right] + \int_0^{\alpha_F} \mathrm{d}\alpha\, \nu(\alpha) \cdot \mathbf{C}(\mathbf{u}(\alpha))$$

Running cost

Costate variables

Constraints

Forward dynamics

$$\frac{\mathrm{d}\mathbf{Q}}{\mathrm{d}\alpha} = f(\mathbf{Q}, \mathbf{u})$$

Order parameters

Control variable

Pontryagin's maximum principle

Cost functional:

$$\mathcal{F}[\mathbf{Q}, \mathbf{\Lambda}, \mathbf{u}, \nu] = \epsilon_g(\mathbf{Q}(\alpha_F)) + \int_0^{\alpha_F} \mathrm{d}\alpha\, \phi\left(\mathbf{Q}(\alpha), \mathbf{u}(\alpha)\right) + \int_0^{\alpha_F} \mathrm{d}\alpha\, \mathbf{\Lambda}(\alpha) \cdot \left[-\frac{\mathrm{d}\mathbf{\Omega}}{\mathrm{d}\alpha} + f(\mathbf{Q}, \mathbf{u})\right] + \int_0^{\alpha_F} \mathrm{d}\alpha\, \nu(\alpha) \cdot \mathbf{C}(\mathbf{u}(\alpha))$$

Running cost

Costate variables

Constraints

Forward dynamics

$$\frac{\mathrm{d}\mathbf{Q}}{\mathrm{d}\alpha} = f(\mathbf{Q}, \mathbf{u})$$

Order parameters

Control variable

Backward dynamics

$$-\frac{\mathrm{d}\mathbf{\Lambda}}{\mathrm{d}\alpha} = \nabla_{\mathbf{Q}} f(\mathbf{Q}, \mathbf{u})^{\mathsf{T}} \mathbf{\Lambda} + \nabla_{\mathbf{Q}} \phi(\mathbf{Q}, \mathbf{u})$$

$$\mathbf{\Lambda}(\alpha_F) = \nabla_{\mathbf{Q}} \varepsilon_g\left(\mathbf{Q}(\alpha_F)\right)$$

Pontryagin's maximum principle

Cost functional:

$$\mathcal{F}[\mathbf{Q}, \mathbf{\Lambda}, \mathbf{u}, \nu] = \epsilon_g(\mathbf{Q}(\alpha_F)) + \int_0^{\alpha_F} \mathrm{d}\alpha\, \phi\left(\mathbf{Q}(\alpha), \mathbf{u}(\alpha)\right) + \int_0^{\alpha_F} \mathrm{d}\alpha\, \mathbf{\Lambda}(\alpha) \cdot \left[-\frac{\mathrm{d}\mathbf{\Omega}}{\mathrm{d}\alpha} + f(\mathbf{Q}, \mathbf{u})\right] + \int_0^{\alpha_F} \mathrm{d}\alpha\, \nu(\alpha) \cdot \mathbf{C}(\mathbf{u}(\alpha))$$

Running cost

Costate variables

Constraints

Forward dynamics

$$\frac{\mathrm{d}\mathbf{Q}}{\mathrm{d}\alpha} = f(\mathbf{Q}, \mathbf{u})$$

Order parameters

Control variable

Backward dynamics

$$-\frac{\mathrm{d}\mathbf{\Lambda}}{\mathrm{d}\alpha} = \nabla_{\mathbf{Q}} f(\mathbf{Q}, \mathbf{u})^{\top} \mathbf{\Lambda} + \nabla_{\mathbf{Q}} \phi(\mathbf{Q}, \mathbf{u})$$

$$\mathbf{\Lambda}(\alpha_F) = \nabla_{\mathbf{Q}} \varepsilon_g\left(\mathbf{Q}(\alpha_F)\right)$$

Optimality condition: $\mathbf{u}^*(\alpha) = \arg \min_{\mathbf{u}:\mathbf{C}(\mathbf{u})=0} \left[\phi\left(\mathbf{Q}(\alpha), \mathbf{u}\right) + \mathbf{\Lambda}(\alpha)^{\top} f\left(\mathbf{Q}(\alpha), \mathbf{u}\right)\right]$

Pontryagin's maximum principle

Cost functional:

$$\mathcal{F}[\mathbf{Q}, \mathbf{\Lambda}, \mathbf{u}, \nu] = \epsilon_g(\mathbf{Q}(\alpha_F)) + \int_0^{\alpha_F} \mathrm{d}\alpha\, \phi\left(\mathbf{Q}(\alpha), \mathbf{u}(\alpha)\right) + \int_0^{\alpha_F} \mathrm{d}\alpha\, \mathbf{\Lambda}(\alpha) \cdot \left[-\frac{\mathrm{d}\mathbf{\Omega}}{\mathrm{d}\alpha} + f(\mathbf{Q}, \mathbf{u})\right] + \int_0^{\alpha_F} \mathrm{d}\alpha\, \nu(\alpha) \cdot \mathbf{C}(\mathbf{u}(\alpha))$$

Running cost

Costate variables

Constraints

Forward dynamics

$$\frac{\mathrm{d}\mathbf{Q}}{\mathrm{d}\alpha} = f(\mathbf{Q}, \mathbf{u})$$

Order parameters

Control variable

Solve self-consistently

Backward dynamics

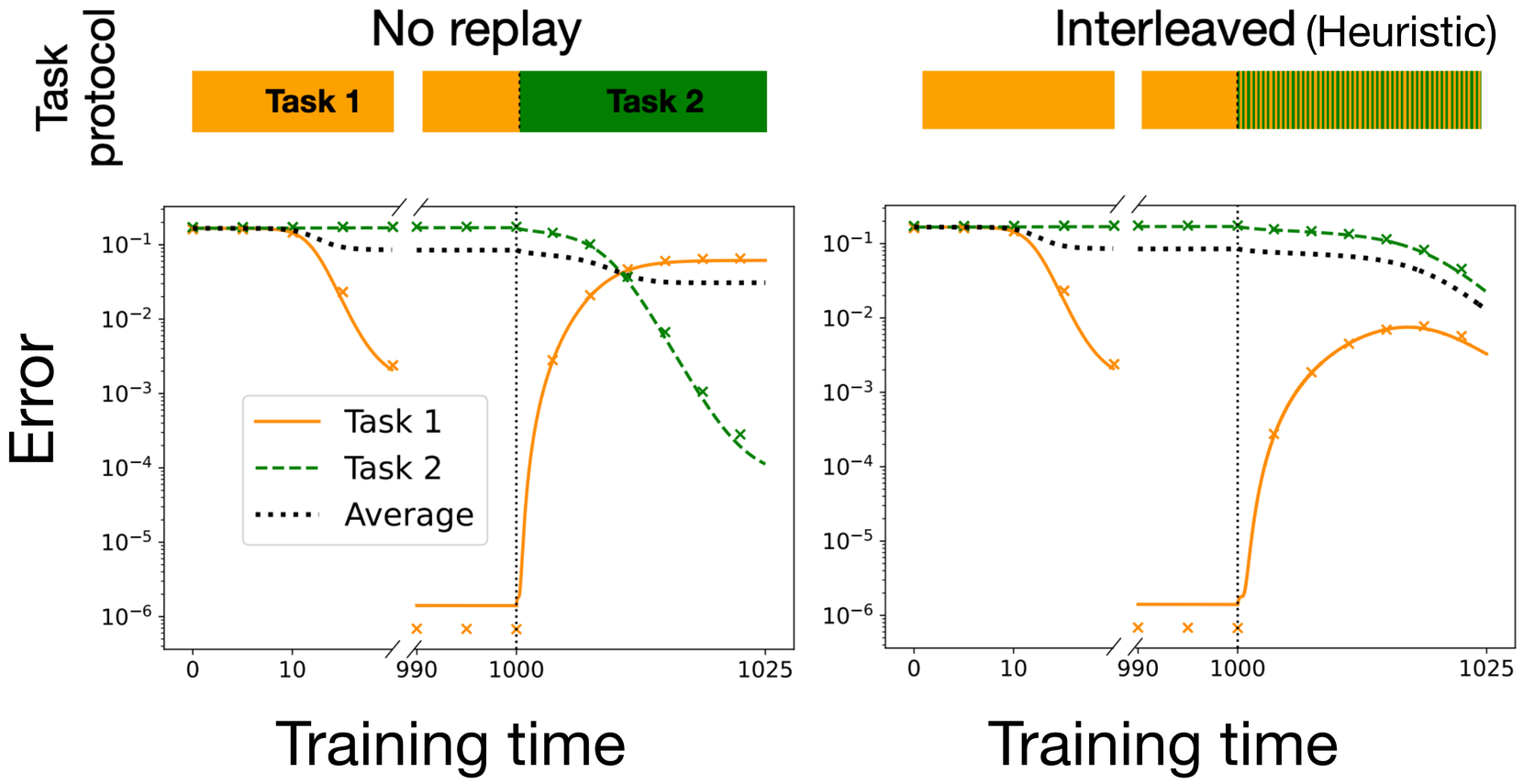
$$-\frac{\mathrm{d}\mathbf{\Lambda}}{\mathrm{d}\alpha} = \nabla_{\mathbf{Q}} f(\mathbf{Q}, \mathbf{u})^{\mathsf{T}} \mathbf{\Lambda} + \nabla_{\mathbf{Q}} \phi(\mathbf{Q}, \mathbf{u})$$

$$\mathbf{\Lambda}(\alpha_F) = \nabla_{\mathbf{Q}} \varepsilon_g\left(\mathbf{Q}(\alpha_F)\right)$$

Optimality condition: $\mathbf{u}^*(\alpha) = \arg \min_{\mathbf{u}:\mathbf{C}(\mathbf{u})=0} \left[\phi\left(\mathbf{Q}(\alpha), \mathbf{u}\right) + \mathbf{\Lambda}(\alpha)^{\mathsf{T}} f\left(\mathbf{Q}(\alpha), \mathbf{u}\right)\right]$

Optimal strategy vs benchmarks

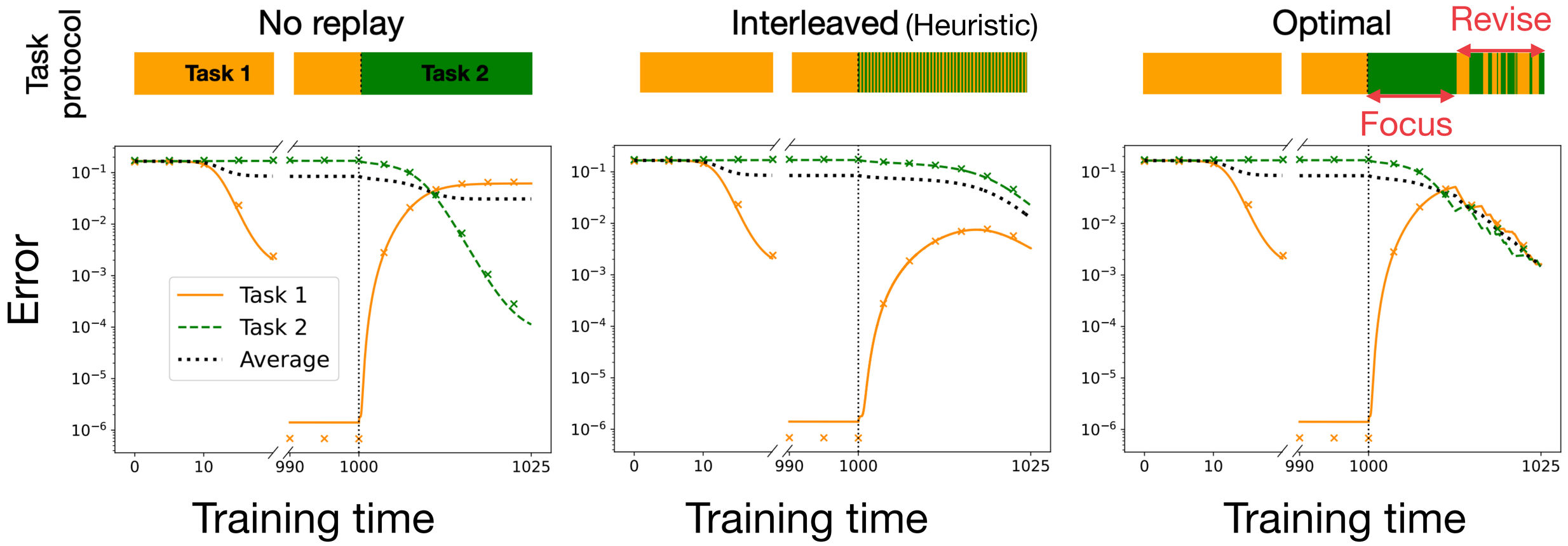
Control: $\mathbf{u} = \text{task}$



F. Mori, S. Sarao Mannelli, FM, *ICLR* 2025

Optimal strategy vs benchmarks

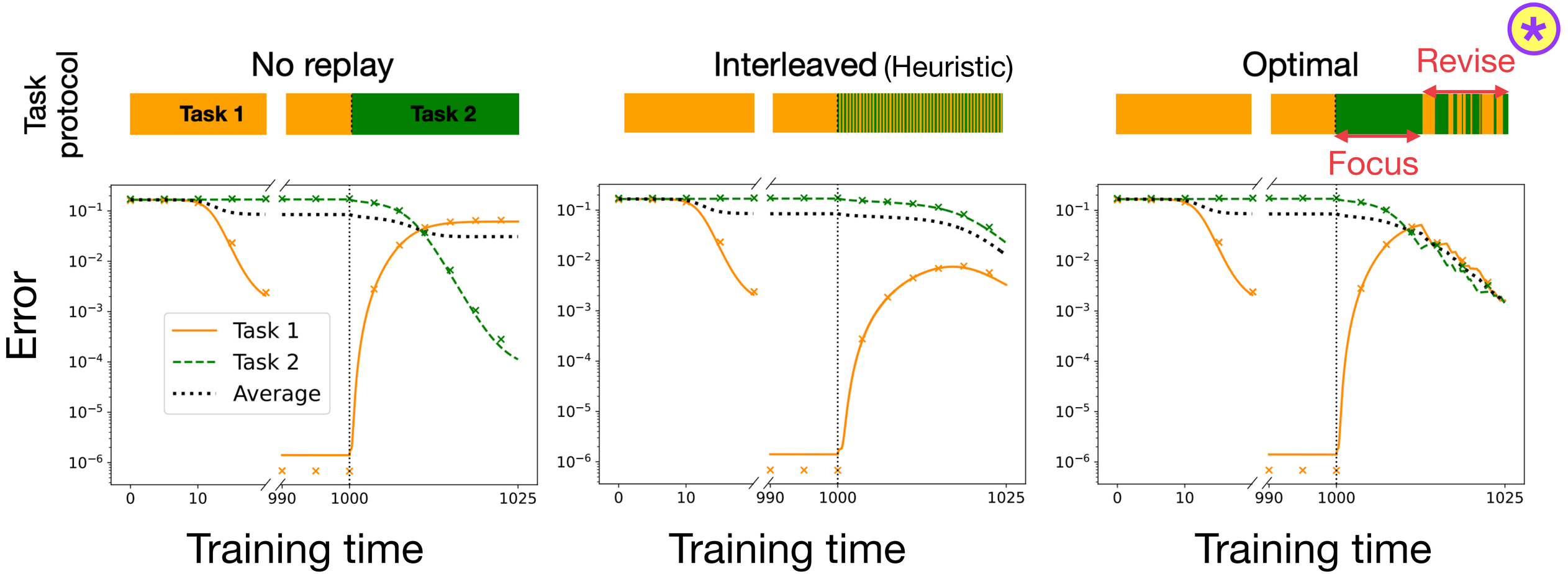
Control: $\mathbf{u} = \text{task}$



F. Mori, S. Sarao Mannelli, FM, *ICLR* 2025

Optimal strategy vs benchmarks

Control: $\mathbf{u} = \text{task}$



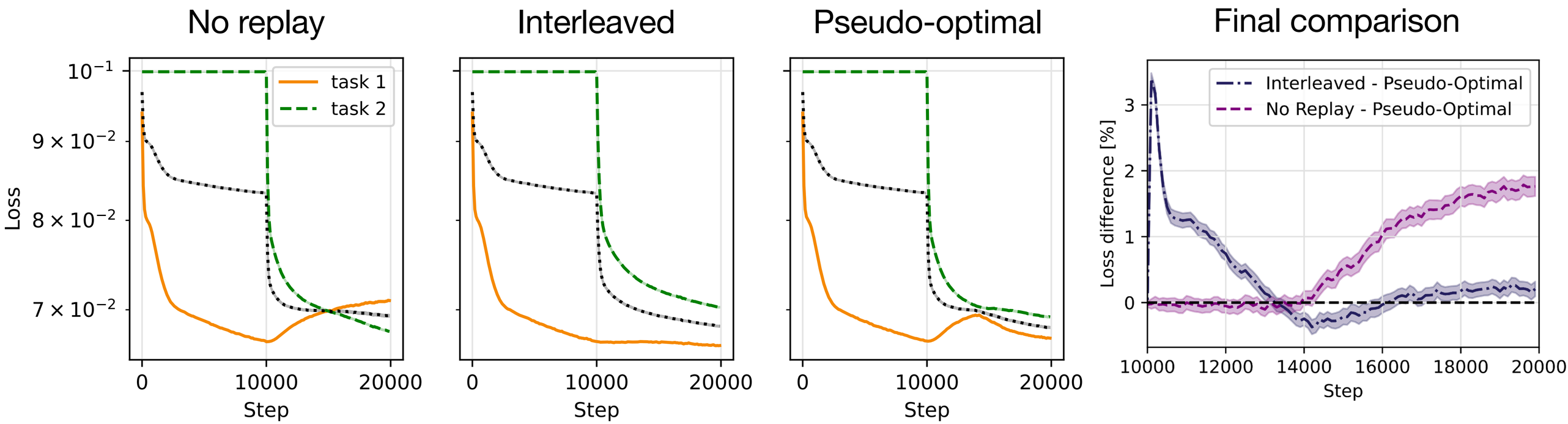
F. Mori, S. Sarao Mannelli, FM, *ICLR* 2025

Task-order does not matter in the revision phase!

Pseudo-optimal strategy on real datasets

Experiments on CIFAR10:

(Similar results for FashionMNIST)



F. Mori, S. Sarao Mannelli, FM, *ICLR* 2025

The impact of noise injection on the learning dynamics

Dropout regularization

Journal of Machine Learning Research 15 (2014) 1929-1958 Submitted 11/13; Published 6/14

Dropout: A Simple Way to Prevent Neural Networks from Overfitting

Nitish Srivastava NITISH@CS.TORONTO.EDU
Geoffrey Hinton HINTON@CS.TORONTO.EDU
Alex Krizhevsky KRIZ@CS.TORONTO.EDU
Ilya Sutskever ILYA@CS.TORONTO.EDU
Ruslan Salakhutdinov RSALAKHU@CS.TORONTO.EDU

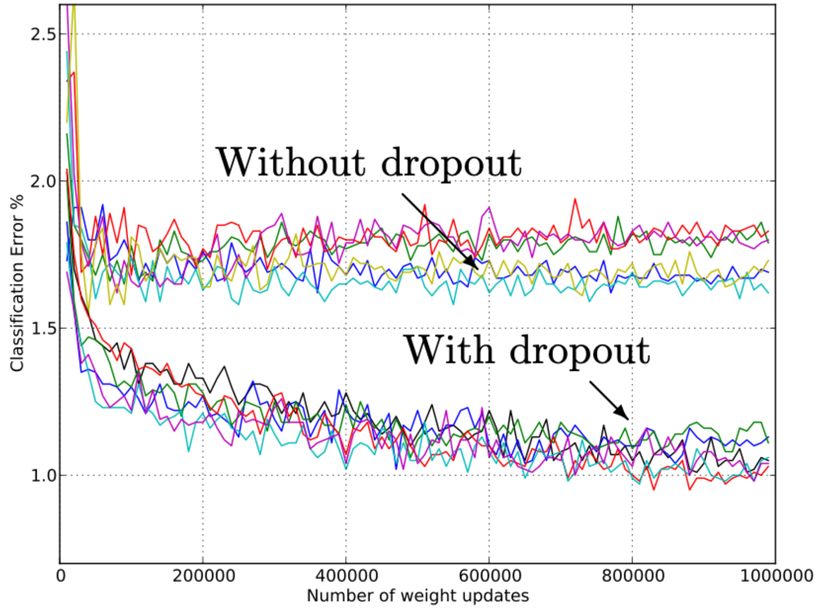
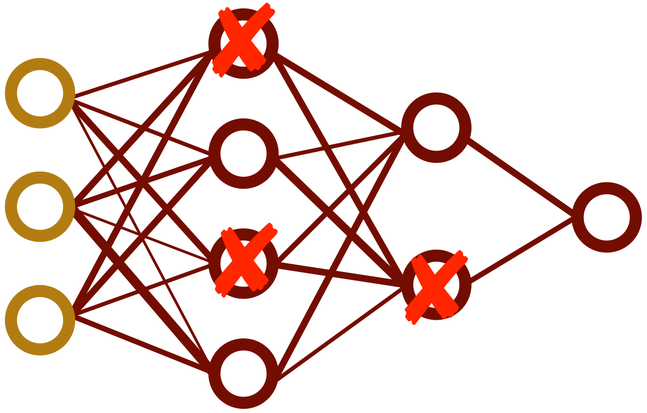


Figure 4: Test error for different architectures with and without dropout. The networks have 2 to 4 hidden layers each with 1024 to 2048 units.

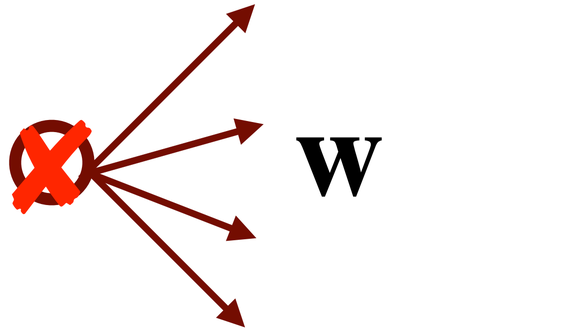


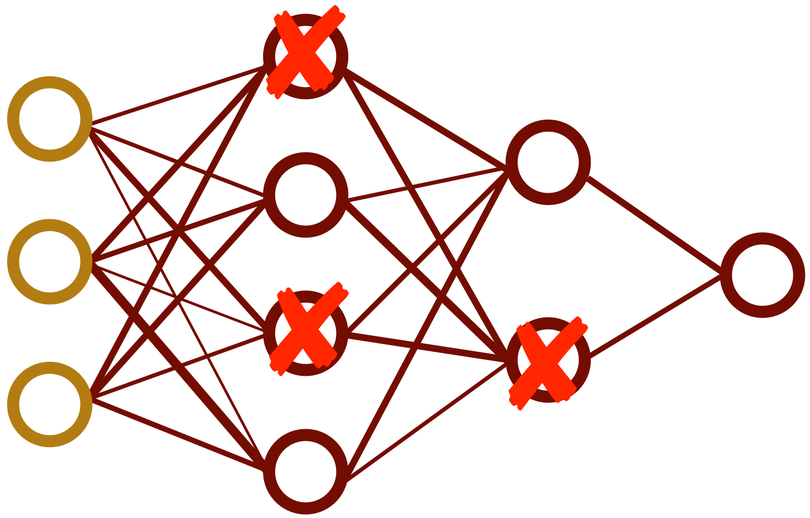
✗ = remove node with probability $1 - p$

Dropout regularization

Train

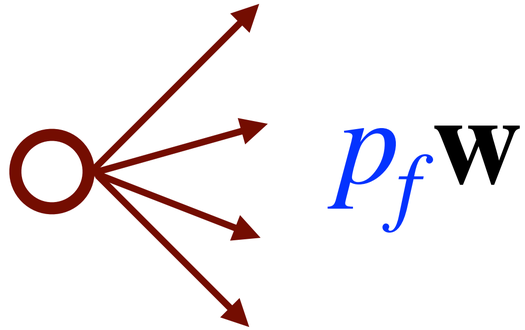
= remove node with probability $1-p$

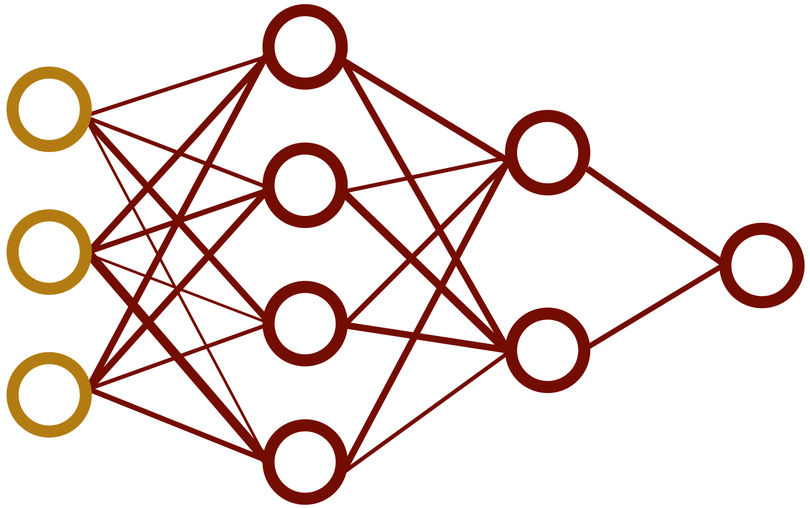




Test

Always present but *rescaled* weights

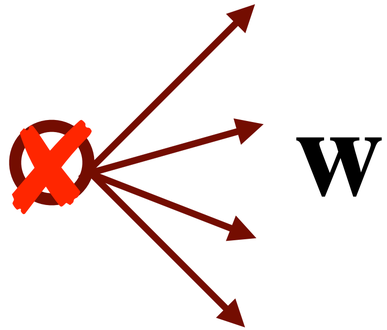




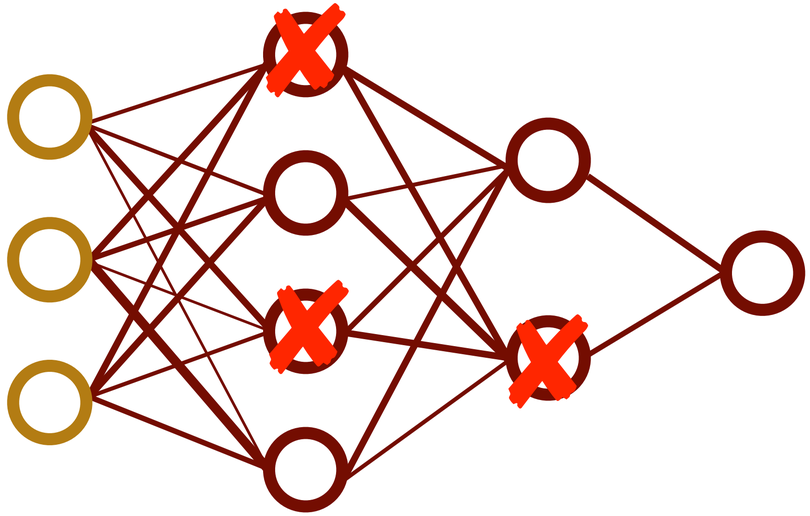
What is the *optimal* dropout probability?

Train

= remove node with probability $1-p$



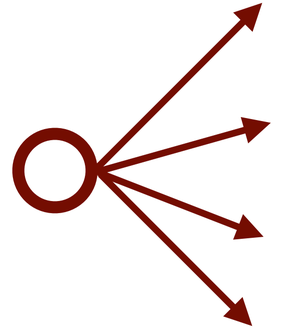
$\mathbf{w}$



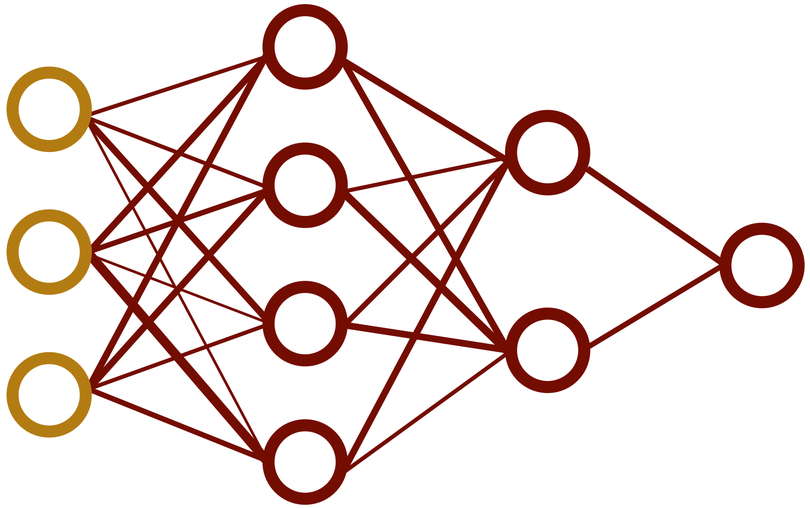
In practice: set heuristically

Test

Always present but *rescaled* weights



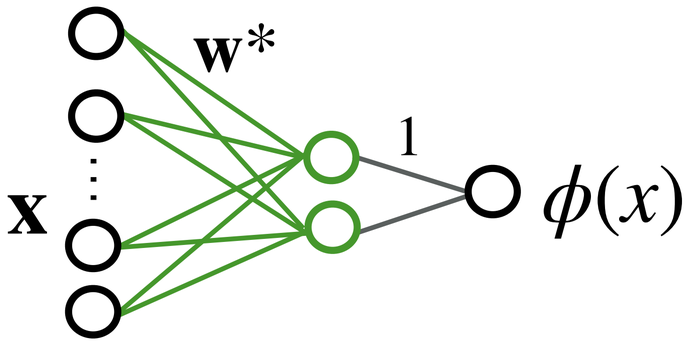
$p_f \mathbf{w}$



A teacher-student model of dropout

Assume i.i.d. standard Gaussian input data: $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_N)$

Teacher



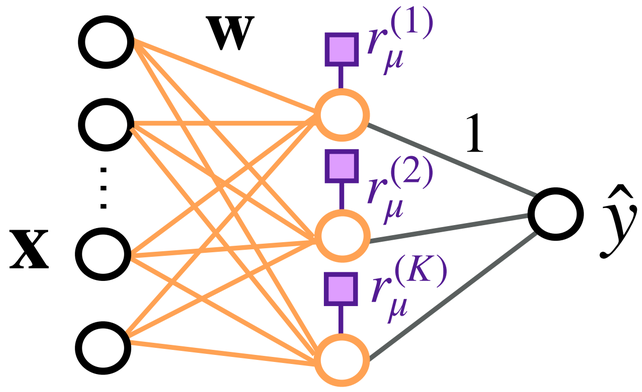
M hidden nodes

$$y = \phi(x) + \sigma_n z$$

Label noise: $z \sim \mathcal{N}(0,1)$

Student

(at training step μ)



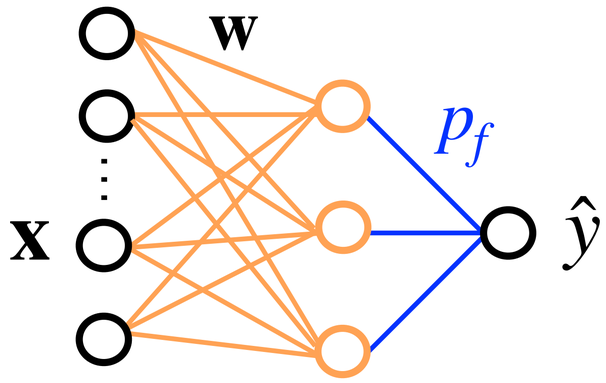
K hidden nodes

Node-activation variables

$r_\mu^{(1)}, r_\mu^{(2)}, \ldots, r_\mu^{(K)} \sim$ Bernoulli(p_μ)

Student

(at testing time)



K hidden nodes

Rescaling factor: p_f

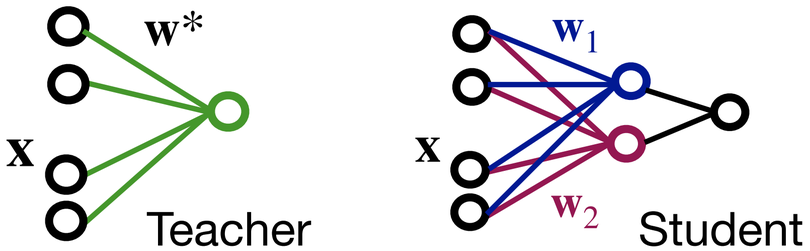
Online learning with **constant** activation prob.

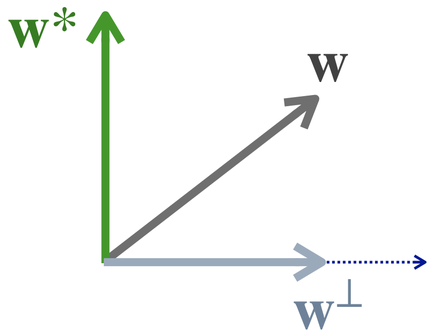
1. **Dropout effectively decorrelates the hidden units of the student network.**

Online learning with **constant** activation prob.

1. **Dropout effectively decorrelates the hidden units of the student network.**

e.g., take $M = 1$ *and* $K = 2$





$$\mathbf{w}_1 = R_1\mathbf{w}^* + \mathbf{w}_1^\perp$$

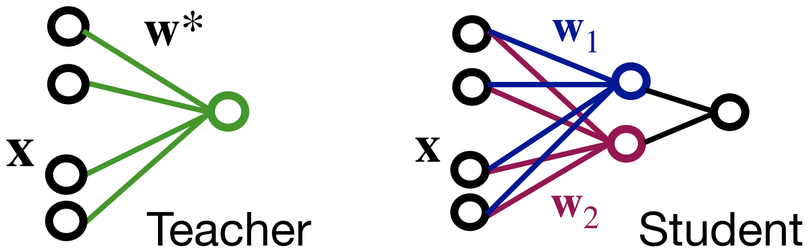
$$\mathbf{w}_2 = R_2\mathbf{w}^* + \mathbf{w}_2^\perp$$

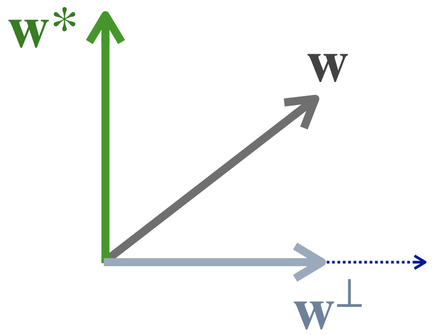
Noise overfitting

Online learning with **constant** activation prob.

1. Dropout effectively decorrelates the hidden units of the student network.

e.g., take $M = 1$ *and* $K = 2$



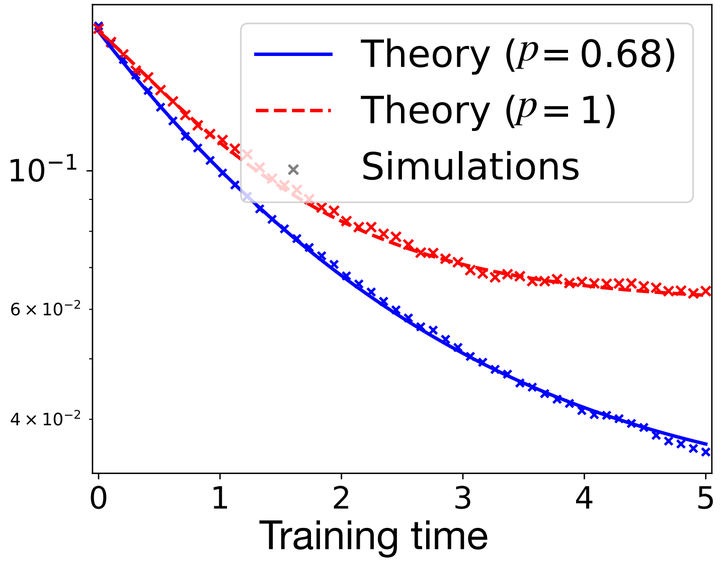


$$\mathbf{w}_1 = R_1\mathbf{w}^* + \mathbf{w}_1^\perp$$

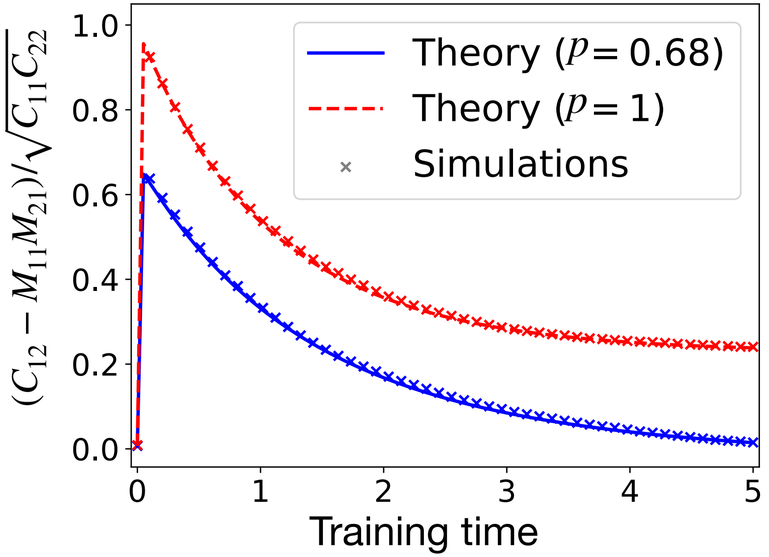
$$\mathbf{w}_2 = R_2\mathbf{w}^* + \mathbf{w}_2^\perp$$

Noise overfitting

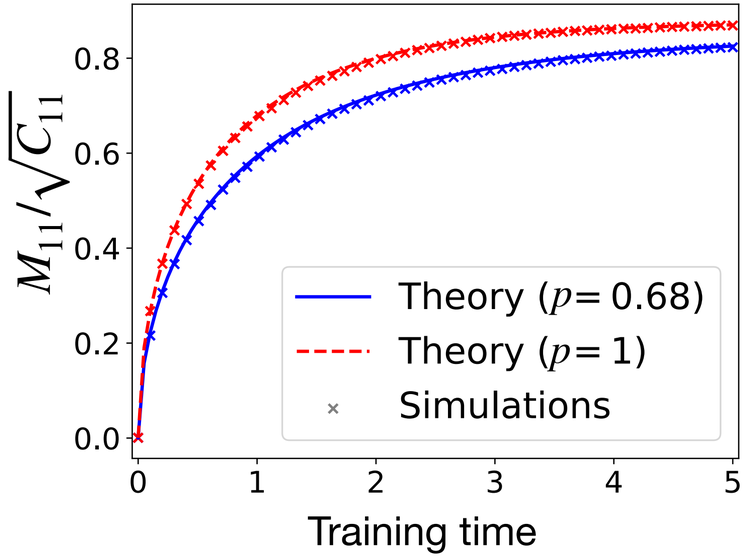
Error



Noise fitting



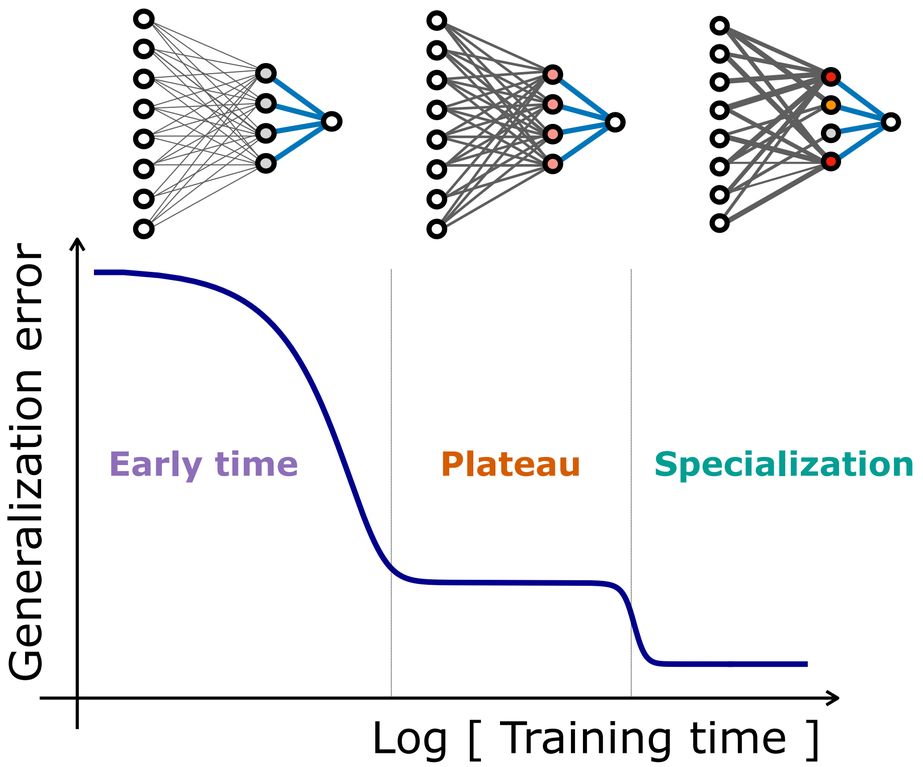
Signal alignment



Francesco Mori, FM, PRE (2025)

Online learning with **constant** activation prob.

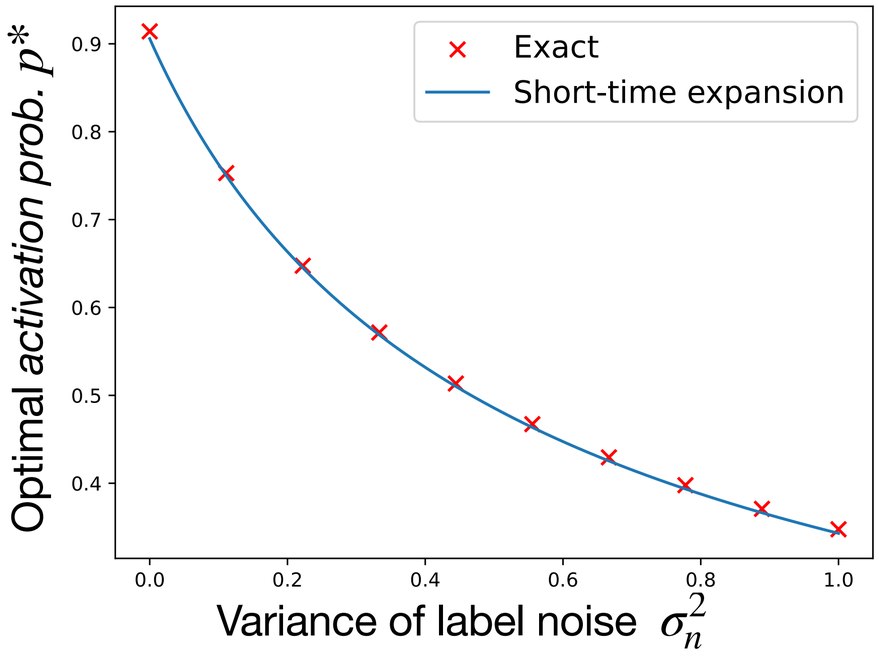
1. **Dropout effectively decorrelates the hidden units of the student network.**
2. **Exact analysis of three learning phases.** $(M > 1)$

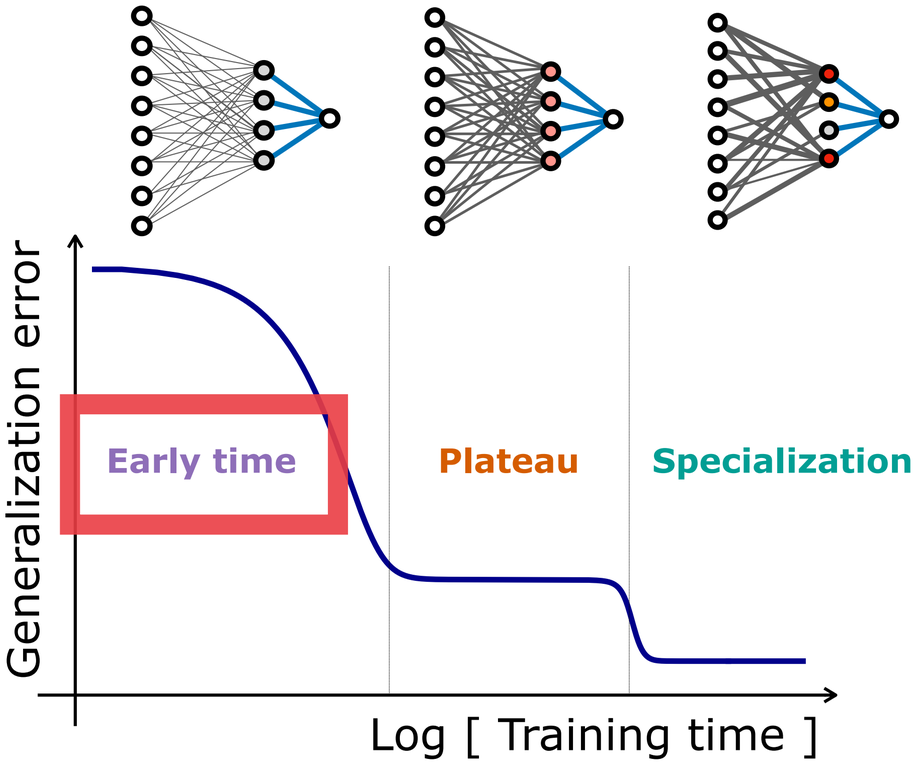


Online learning with **constant** activation prob.

1. **Dropout effectively decorrelates the hidden units of the student network.**
2. **Exact analysis of three learning phases.** $(M > 1)$

- The **optimal constant activation probability** depends on label noise, width, and learning rate.





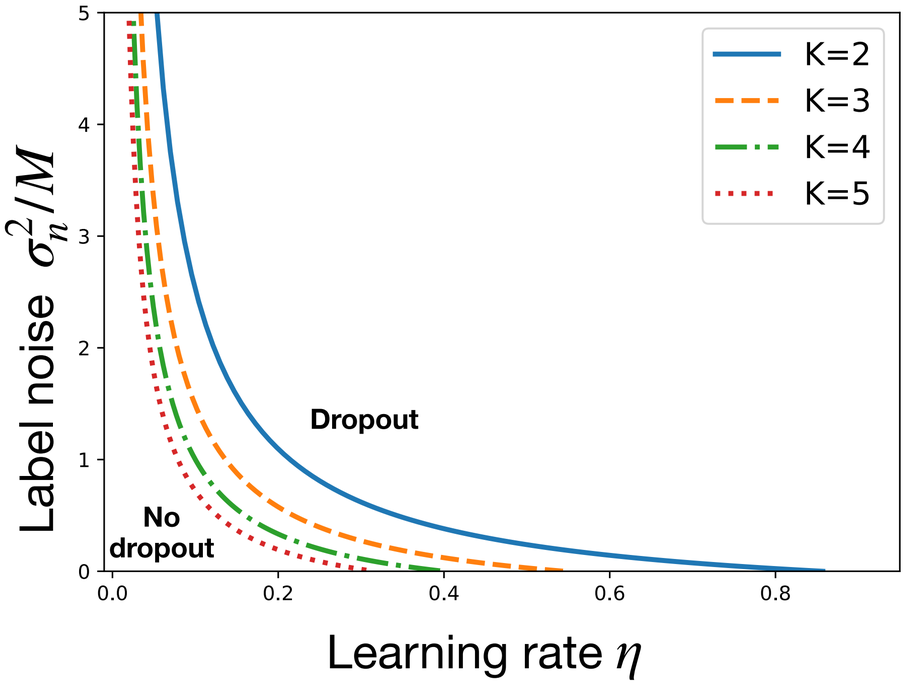
$$p^* = \min\{1; \frac{12M}{3M\eta + 9\eta\sigma_n^2 + \sqrt{3\eta(M+3\sigma_n^2)\left[M(32(K-1)+3\eta)+9\eta\sigma_n^2\right]}}\}$$

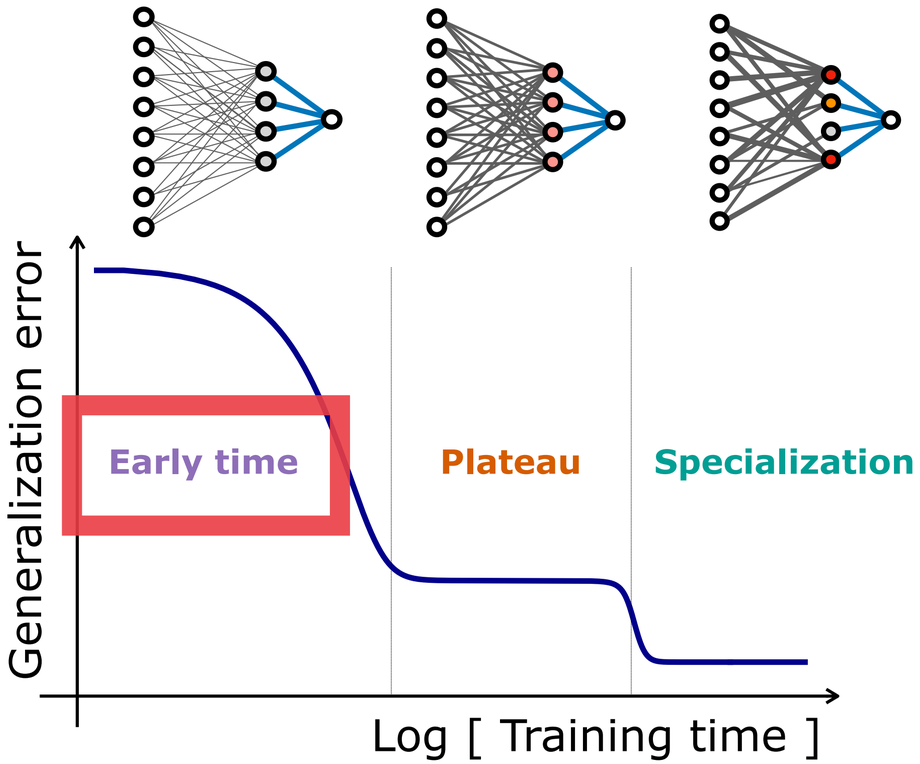
Francesco Mori, FM, PRE (2025)

Online learning with **constant** activation prob.

1. **Dropout effectively decorrelates the hidden units of the student network.**
2. **Exact analysis of three learning phases.** $(M > 1)$

- The **optimal constant activation probability** depends on label noise, width, and learning rate.





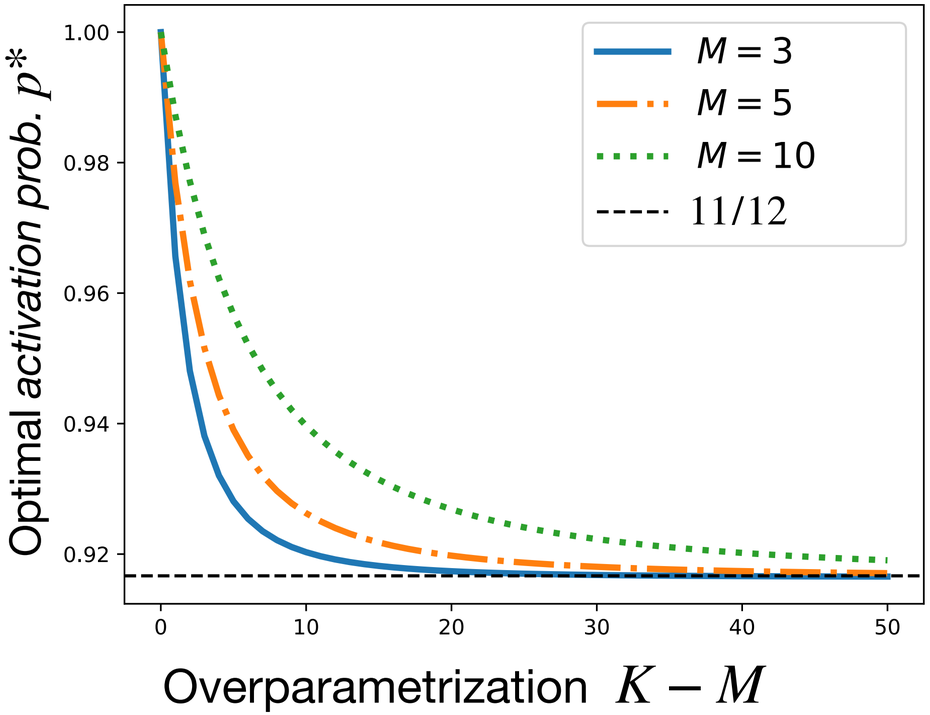
Dropout is optimal if: $\sigma_n^2 \geq M \left[\frac{2}{(4K-1)\eta} - \frac{1}{3} \right]$

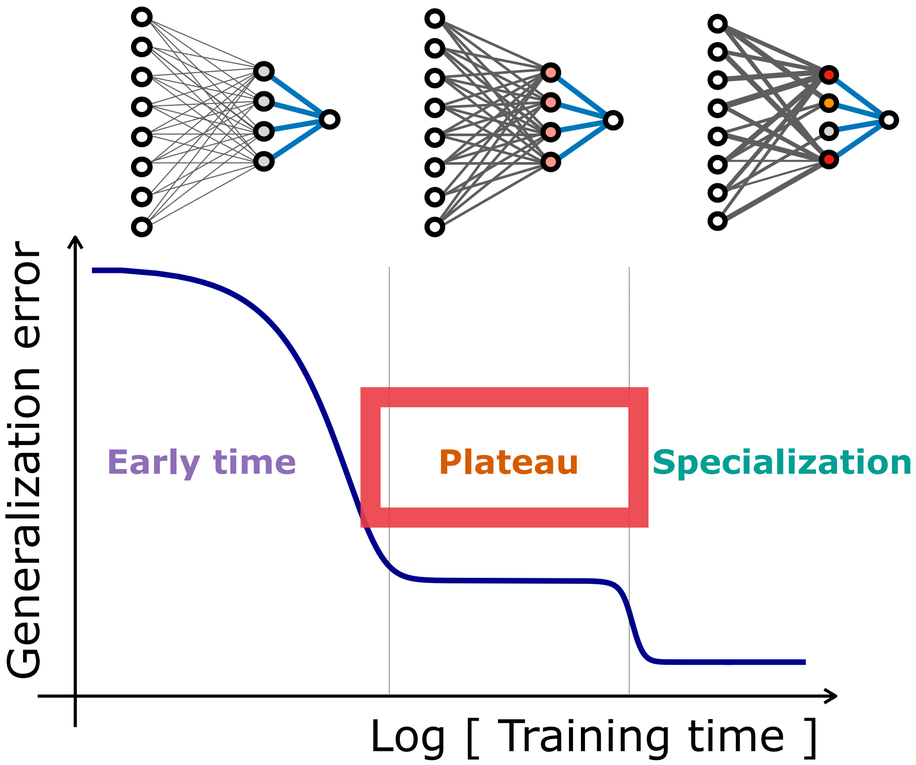
Francesco Mori, FM, PRE (2025)

Online learning with **constant** activation prob.

1. **Dropout effectively decorrelates the hidden units of the student network.**
2. **Exact analysis of three learning phases.** $(M > 1)$

- The **optimal constant activation probability** depends on label noise, width, and learning rate.



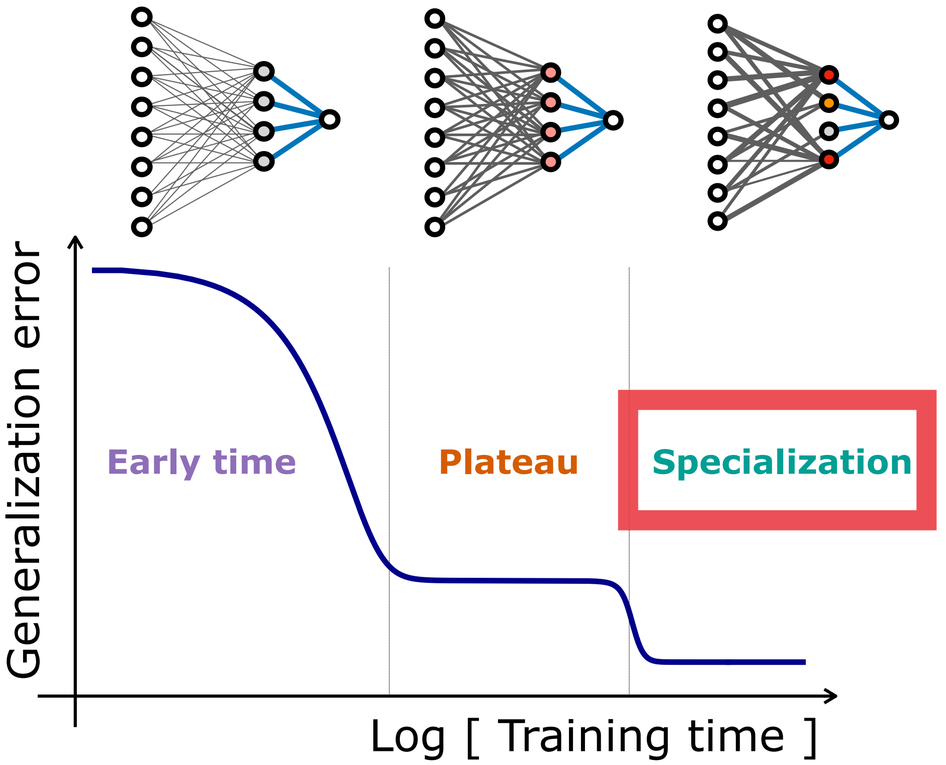


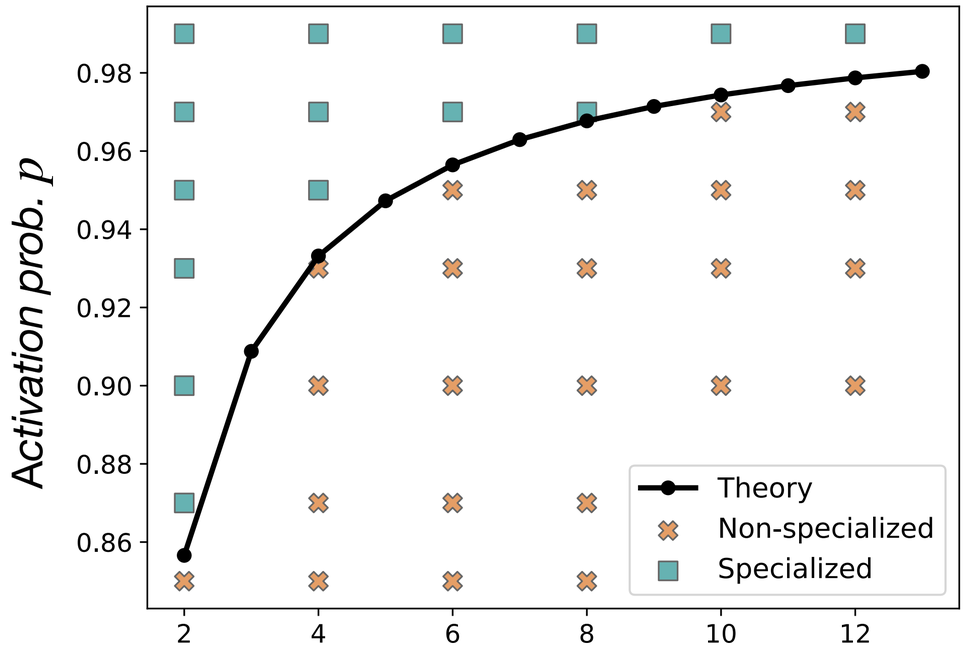
Francesco Mori, FM, PRE (2025)

Online learning with **constant** activation prob.

1. **Dropout effectively decorrelates the hidden units of the student network.**
2. **Exact analysis of three learning phases.** $(M > 1)$

- The **optimal constant activation probability** depends on label noise, width, and learning rate.
- High dropout probability prevents specialization.





$\eta \to 0$

Francesco Mori, FM, PRE (2025)

Adaptive dropout probability

ANNEALED DROPOUT TRAINING OF DEEP NETWORKS

Steven J. Rennie, Vaibhava Goel, and Samuel Thomas

IBM Thomas J. Watson Research Center
NY, USA
{sjrennie, vgoel, sthomas}@us.ibm.com

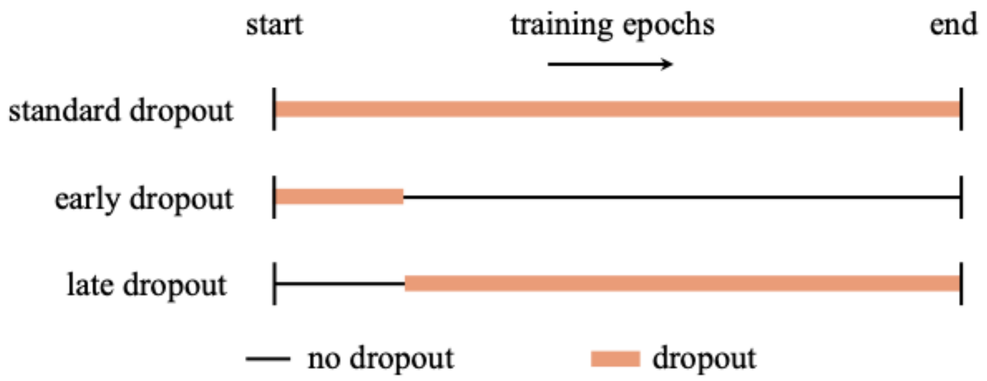
Curriculum Dropout

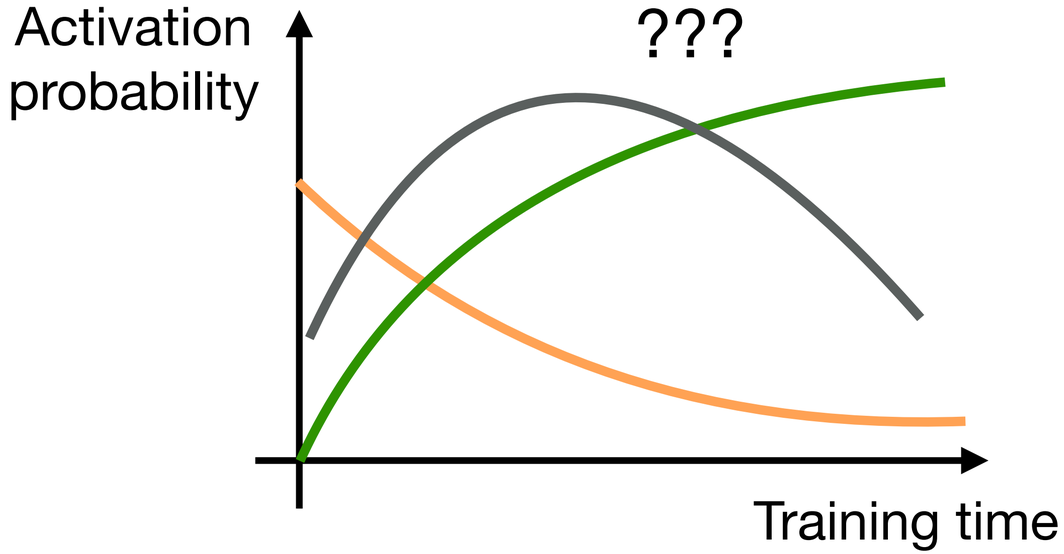
Pietro Morerio[1], Jacopo Cavazza[1,2], Riccardo Volpi[1,2], René Vidal[3] and Vittorio Murino[1,4]

[1]Pattern Analysis & Computer Vision (PAVIS) – Istituto Italiano di Tecnologia – *Genova, 16163, Italy*
[2]Electrical, Electronics and Telecommunication Engineering and Naval Architecture Department (DITEN) – Università degli Studi di Genova – *Genova, 16145, Italy*
[3]Department of Biomedial Engineering – Johns Hopkins University – *Baltimore, MD 21218, USA*
[4]Computer Science Department – Università di Verona – *Verona, 37134, Italy*
{pietro.morerio,jacopo.cavazza,riccardo.volpi,vittorio.murino}@iit.it, rvidal@cis.jhu.edu

Dropout Reduces Underfitting

Zhuang Liu [*1] Zhiqiu Xu [*2] Joseph Jin [2] Zhiqiang Shen [3] Trevor Darrell [2]





Adaptive dropout probability

GOAL

Minimize the final error

Low-dimensional ODEs

$$\frac{\mathrm{d}\mathbf{Q}}{\mathrm{d}\alpha} = f(\mathbf{Q}, p)$$

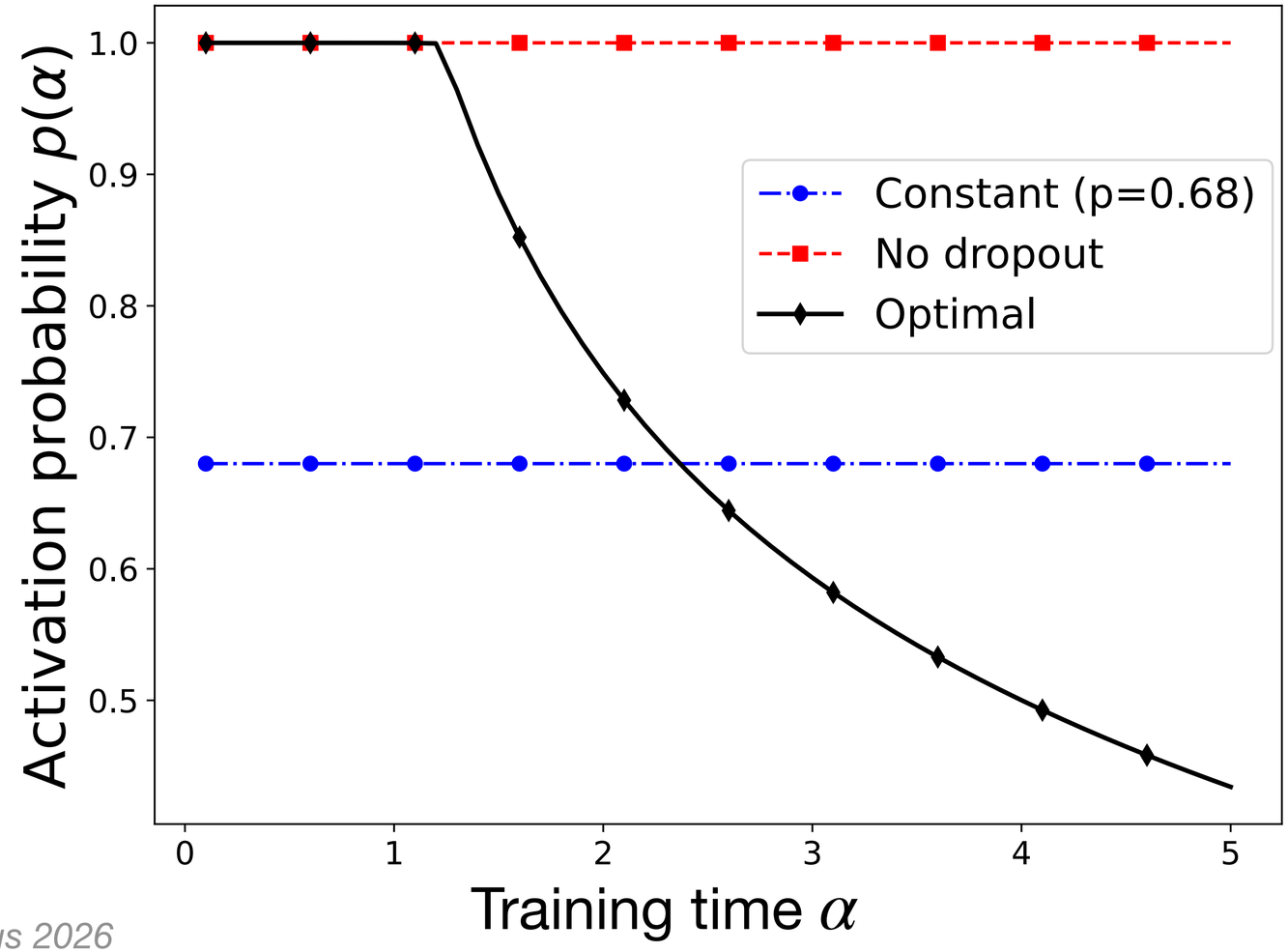
Optimal control

$$\frac{d\mathbf{Q}}{d\alpha} = f(\mathbf{Q}, p^*)$$

Optimal dropout **schedules**

Teacher-student setting $(K = 2\,, M = 1)$

1. The optimal schedule decreases monotonically the node-activation probability.



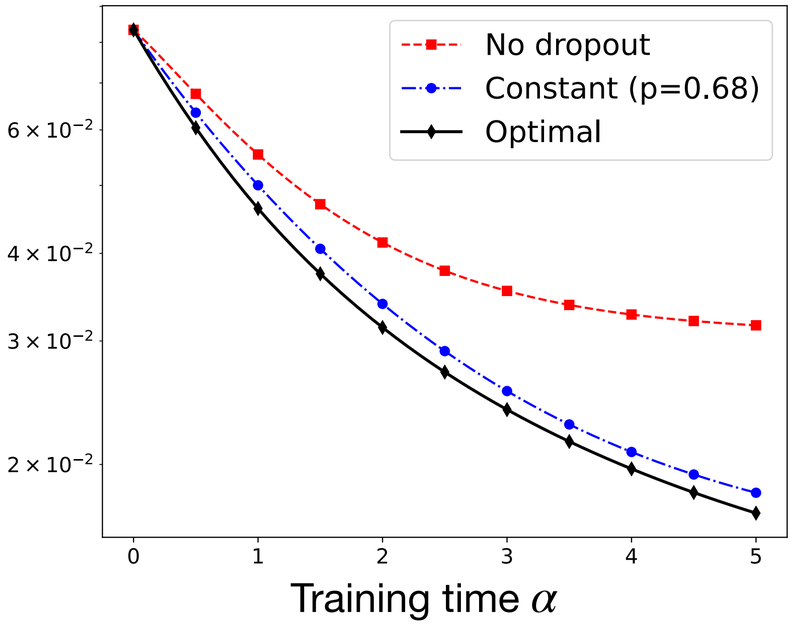
$\eta = 1\,, \alpha_F = 5\,, \sigma_n = 0.3\,, Q_0 = 0$

Optimal dropout **schedules**

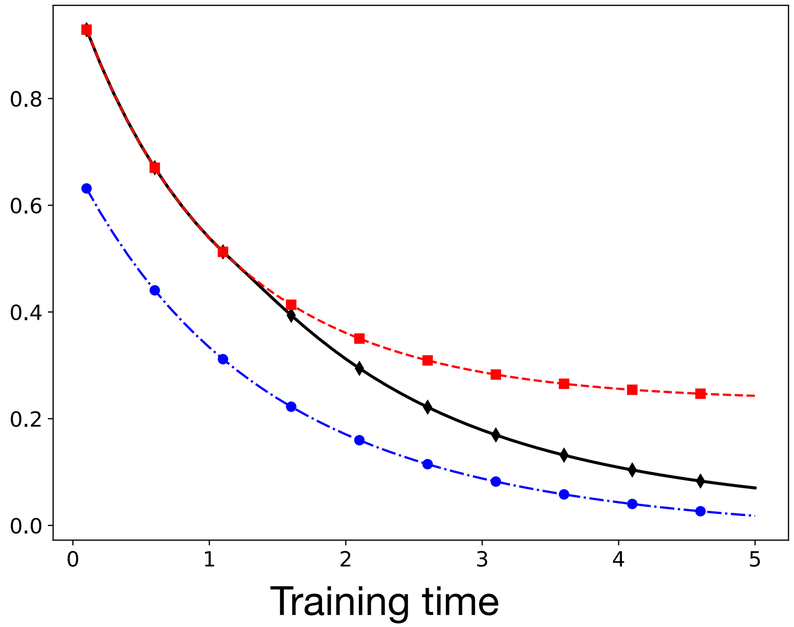
Teacher-student setting $(K = 2\,, M = 1)$

1. The optimal schedule decreases monotonically the node-activation probability.
2. Adaptive dropout achieves the optimal balance between node decorrelation and signal recovery.

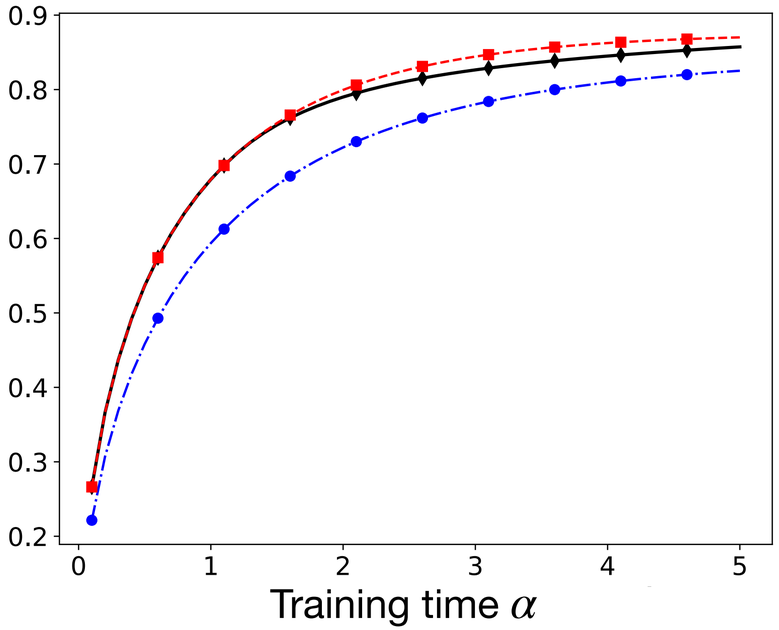
Error



Noise fitting



Signal alignment

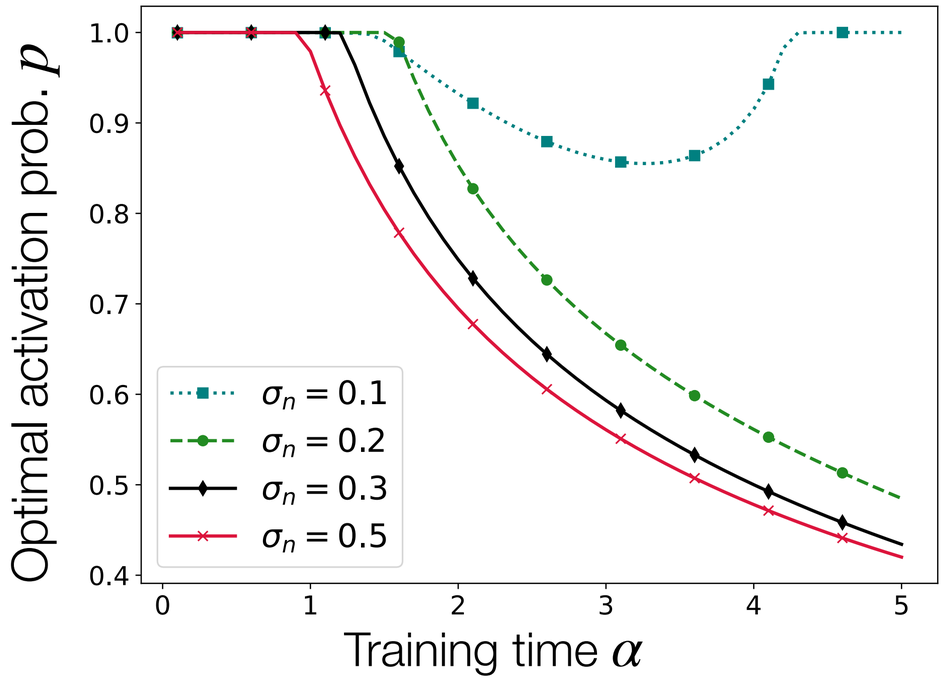


FM, Francesco Mori, *PNAS Nexus 2026*

Optimal dropout **schedules**

Teacher-student setting $(K = 2\,, M = 1)$

1. The optimal schedule decreases monotonically the node-activation probability.
2. Adaptive dropout achieves the optimal balance between node decorrelation and signal recovery.
3. The dropout schedule adapts to the noise level in the task.



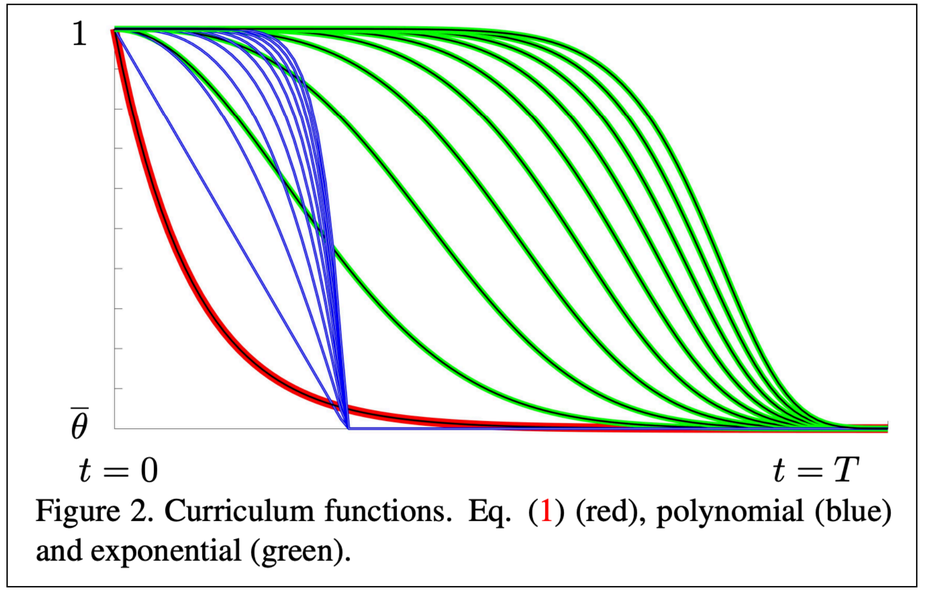


Figure 2. Curriculum functions. Eq. (1) (red), polynomial (blue) and exponential (green).

Morerio, P., Cavazza, J., Volpi, R., Vidal, R., & Murino, V. (2017). ***Curriculum dropout***. In Proceedings of the IEEE international conference on computer vision (pp. 3544-3552)

(Optional)

Curriculum learning

Curriculum learning

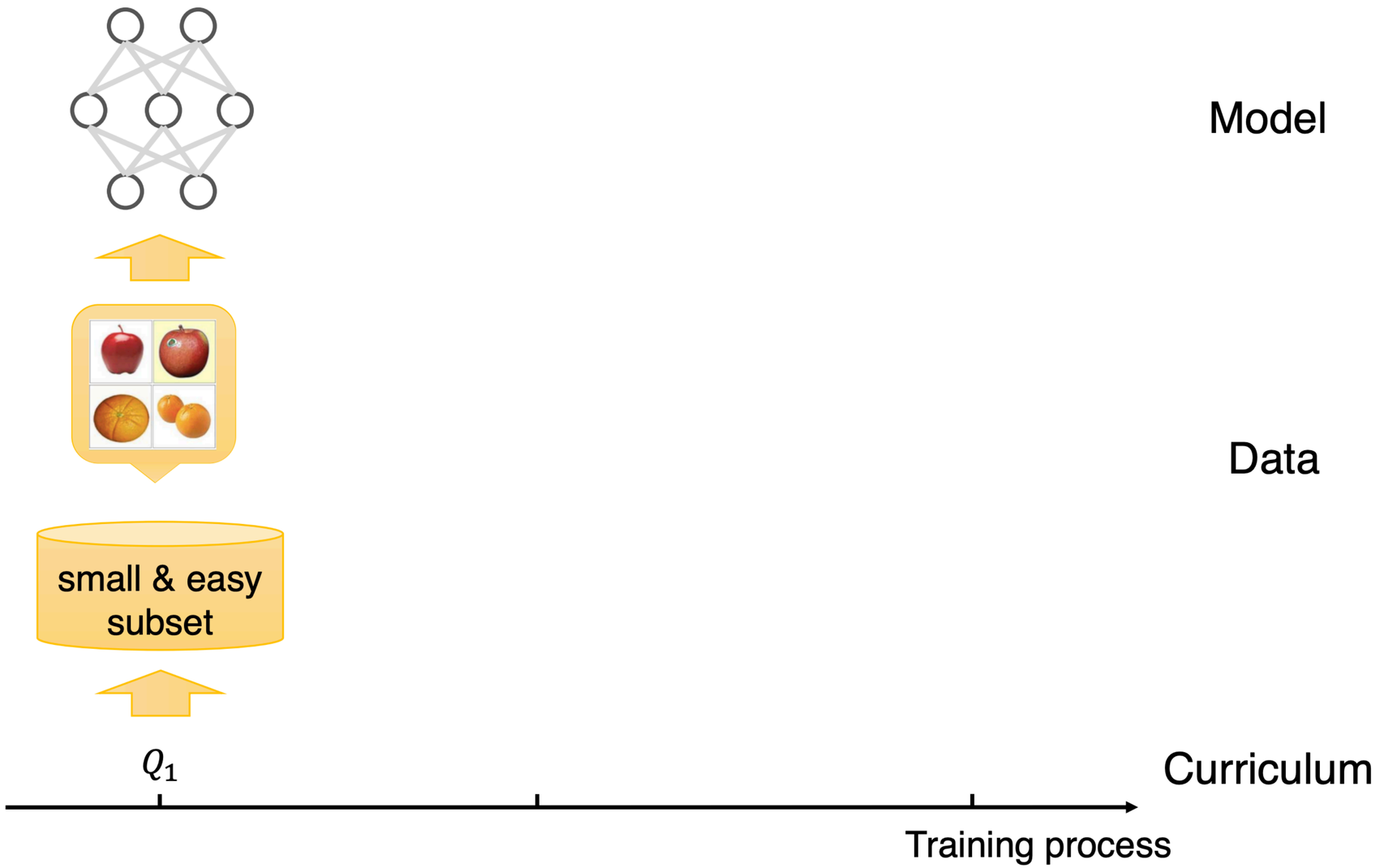


Image from: *Wang, Xin, Yudong Chen, and Wenwu Zhu. IEEE transactions on pattern analysis and machine intelligence 44.9 (2021):4555-4576.*

Curriculum learning

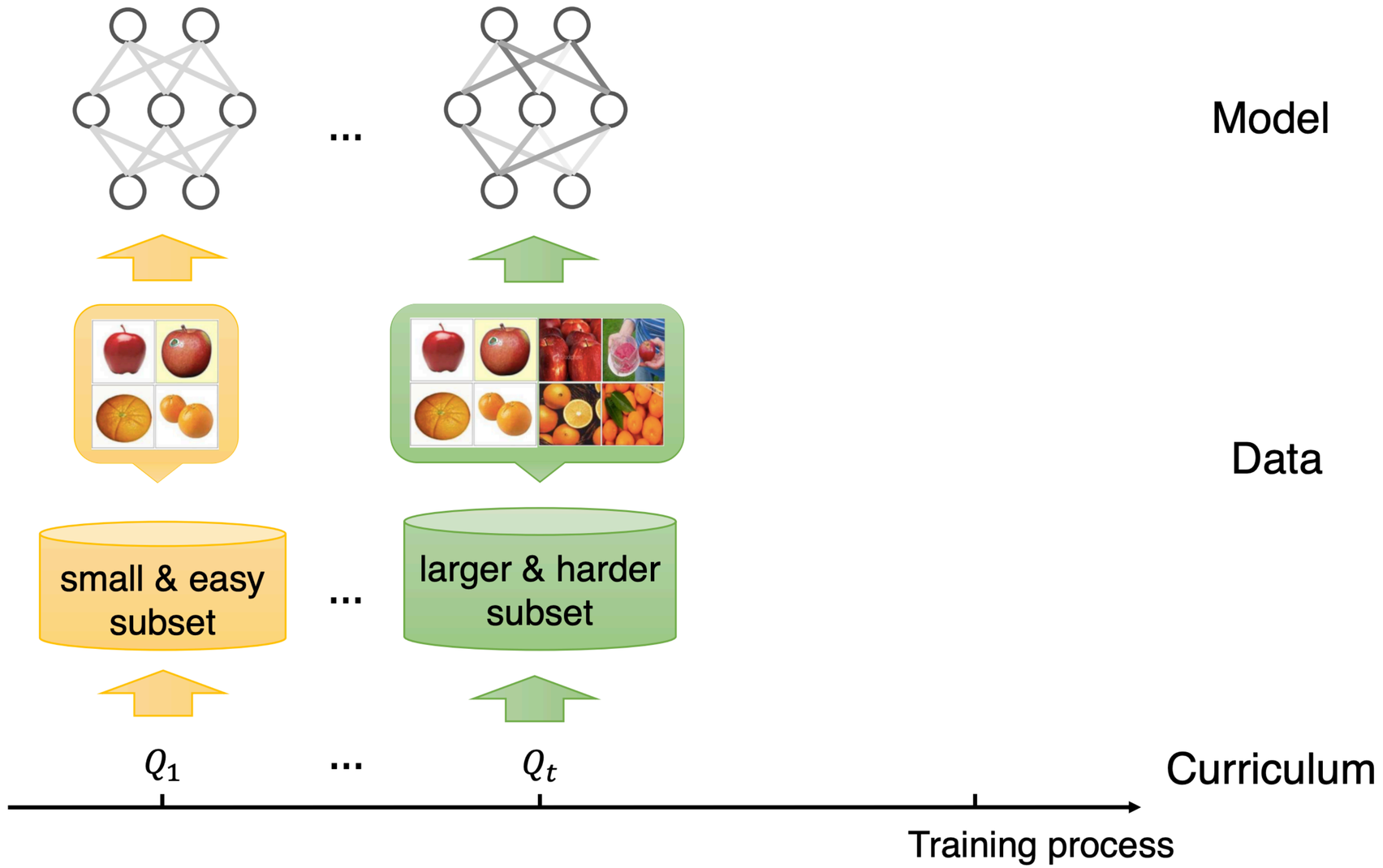


Image from: *Wang, Xin, Yudong Chen, and Wenwu Zhu. IEEE transactions on pattern analysis and machine intelligence 44.9 (2021):4555-4576.*

Curriculum learning

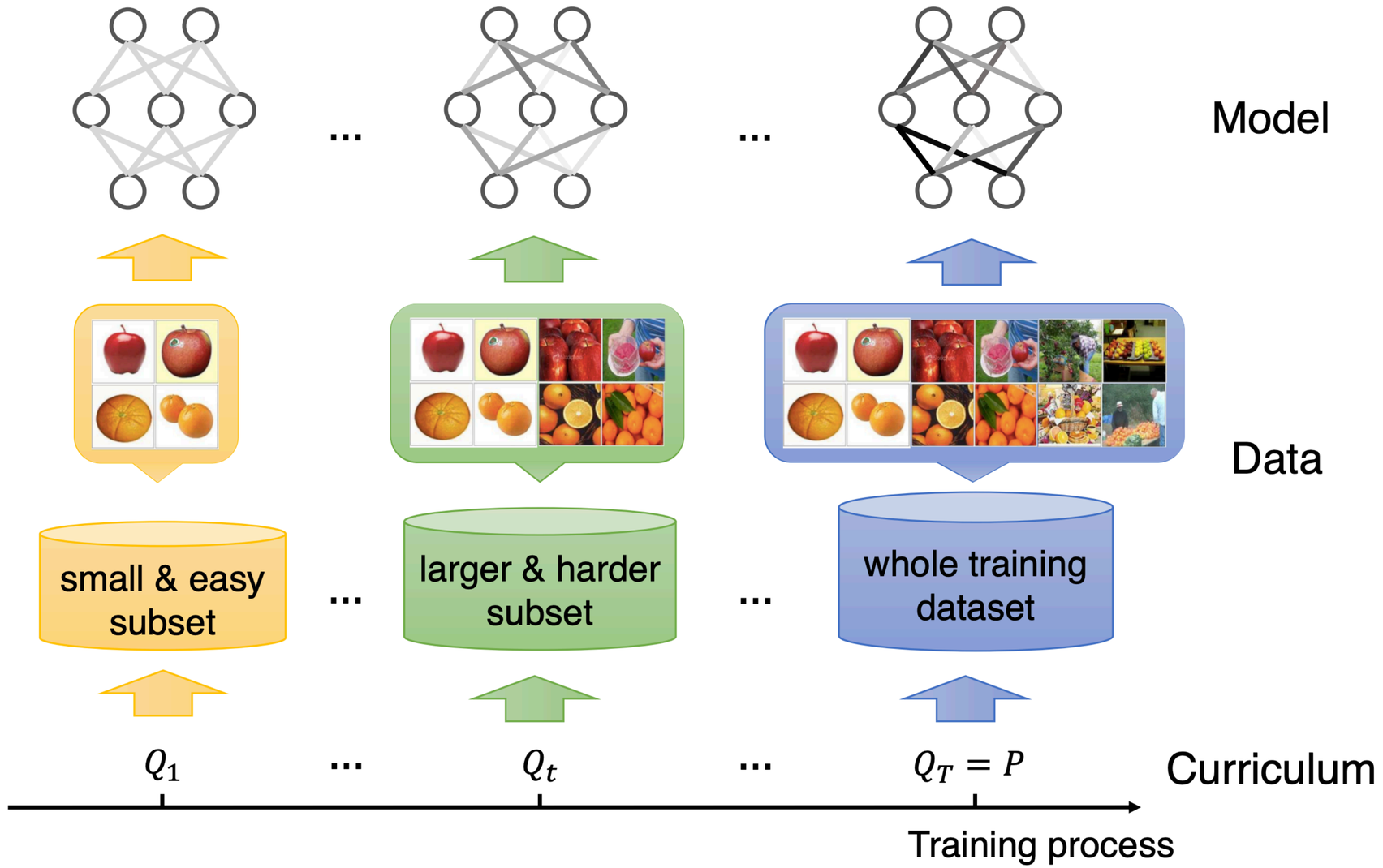


Image from: *Wang, Xin, Yudong Chen, and Wenwu Zhu. IEEE transactions on pattern analysis and machine intelligence 44.9 (2021):4555-4576.*

Curriculum learning

(non-exhaustive list)

Animals:

- Conditional reflexes (dogs) [Pavlov (1927)]
- Shaping (rats, pigeons) [Skinner (1938)]
- Discrimination along a continuum (rats) [Lawrence (1952)]
- Cross-species auditory identification (rats, humans) [Liu et al. (2008)]

Humans:

- Discrimination along a continuum [Baker, Stanley (1954)]
- Past tense [Plunkett et al (1990; 1991)]
- Fading with auditory and visual stimuli [Pashler, Mozer (2013)]
- Eureka effect [Ahissar, Hochstein (1997)]

ML (empirical):

- Easy-to-hard training [Bengio et al (2009]
- Anti-curriculum [Zhang et al (2019); Hacohen & Weinshall (2019)]
- No effect of CL in vision benchmarks [Wu et al (2020)]
- Convincing results for LLMs and RL [Brown et al (2020); Narvekar et al (2020)]

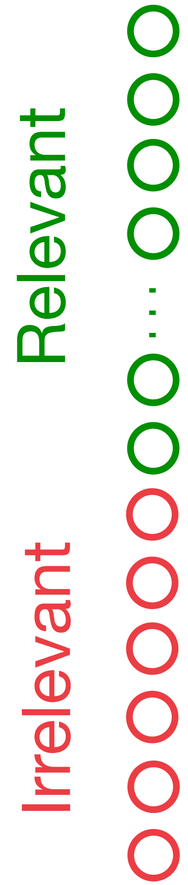
ML (theory):

- Single-update advantage of easy samples [Weinshall et al (2020)]
- Speed benefit but limited performance upgrade in convex problems [Saglietti et al (2021); Lee et al. (2024)]
- Computational benefit in parity machines [Abbe et al. (2023); Cornacchia et al (2023)]
- Asymptotic benefit in non-convex models [Mannelli et al (2024)]

A teacher-student model of curriculum learning

Introduced in: *Bengio, et al. (ICML 2009); Saglietti, et al. (NeurIPS 2022)*

Input: $\mathbf{x} = (\mathbf{x}_r, \mathbf{x}_i) \in \mathbb{R}^N$



$\mathbf{x}_r \in \mathbb{R}^{\rho N}$

$\mathbf{x}_r \sim \mathcal{N}(\mathbf{0}, \mathrm{Id})$

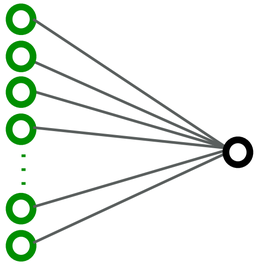
Unit variance

$\mathbf{x}_i \in \mathbb{R}^{(1-\rho)N}$

$\mathbf{x}_i \sim \mathcal{N}(\mathbf{0}, \sqrt{\Delta}\,\mathrm{Id})$

Variance Δ

Teacher



$y = \mathrm{sign}(\mathbf{w}^* \cdot \mathbf{x}_r)$

A teacher-student model of curriculum learning

Introduced in: *Bengio, et al. (ICML 2009); Saglietti, et al. (NeurIPS 2022)*

Input: $\mathbf{x} = (\mathbf{x}_r, \mathbf{x}_i) \in \mathbb{R}^N$

Relevant

Irrelevant

$\mathbf{x}_r \in \mathbb{R}^{\rho N}$

$\mathbf{x}_r \sim \mathcal{N}(\mathbf{0}, \mathrm{Id})$

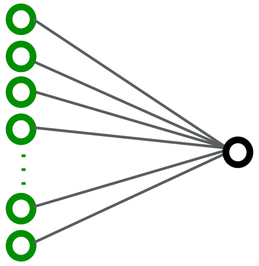
Unit variance

$\mathbf{x}_i \in \mathbb{R}^{(1-\rho)N}$

$\mathbf{x}_i \sim \mathcal{N}(\mathbf{0}, \sqrt{\Delta}\,\mathrm{Id})$

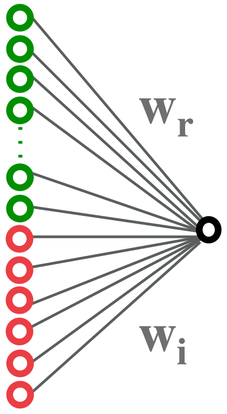
Variance Δ

Teacher



$$y = \mathrm{sign}(\mathbf{w}^* \cdot \mathbf{x}_r)$$

Student



$$y = \mathrm{erf}\left(\frac{\mathbf{w} \cdot \mathbf{x}}{\sqrt{2}}\right)$$

Ridge-regularized squared loss:

$$\mathscr{L} = \frac{1}{2}(y - \hat{y})^2 + \lambda \|\mathbf{w}\|_2^2$$

Online SGD

$$\mathbf{w}^{\mu+1} = \mathbf{w}^{\mu} - \eta \nabla_{\mathbf{w}} \mathscr{L}(u_\mu)$$

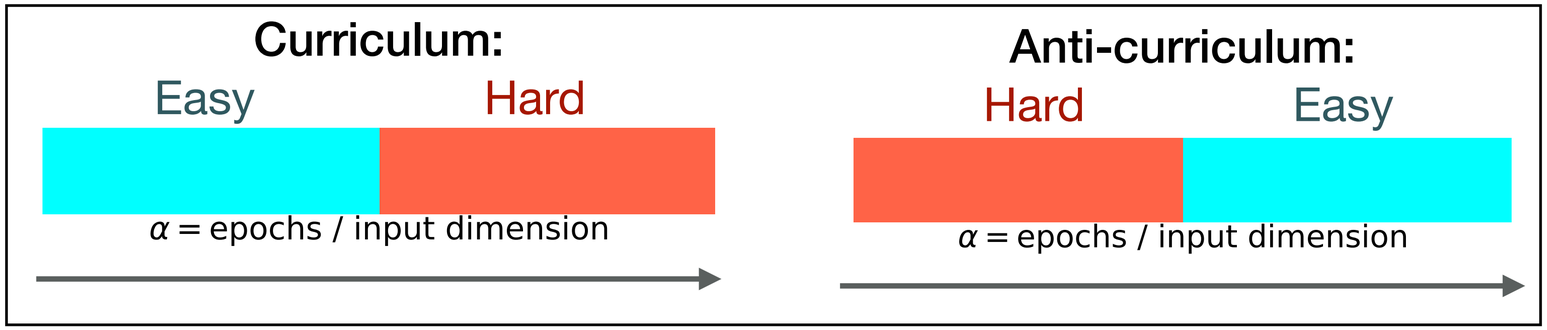
$$u^\mu = \Delta^\mu$$ (Higher = harder task)

A teacher-student model of curriculum learning

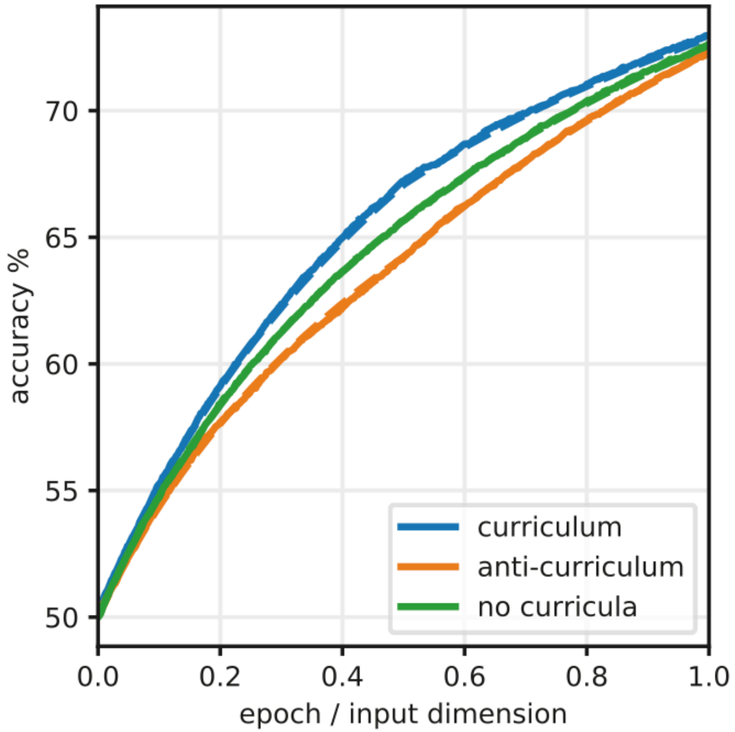
An Analytical Theory of Curriculum Learning in Teacher-Student Networks

Luca Saglietti[†,*], **Stefano Sarao Mannelli**[‡,*], **and Andrew Saxe**[‡,§]

[Saglietti, et al. (NeurIPS 2022)]



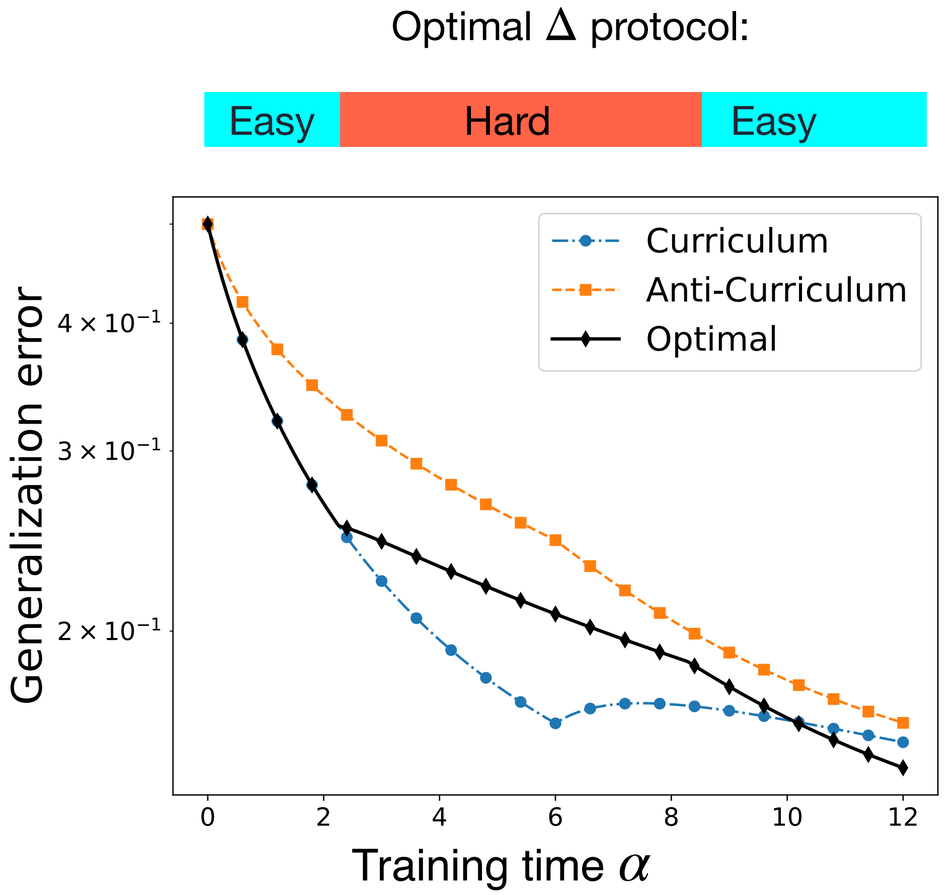
$\Delta \in \{\Delta_1, \Delta_2\}$, $\Delta_1 < \Delta_2$



Optimal curriculum protocol

- Easy-to-hard curriculum is often *suboptimal*.

Control: $u = \Delta$



$\rho = 0.5,\ \eta = 3\ \Delta_1 = 0\ ,\Delta_2 = 2, \lambda = 0$

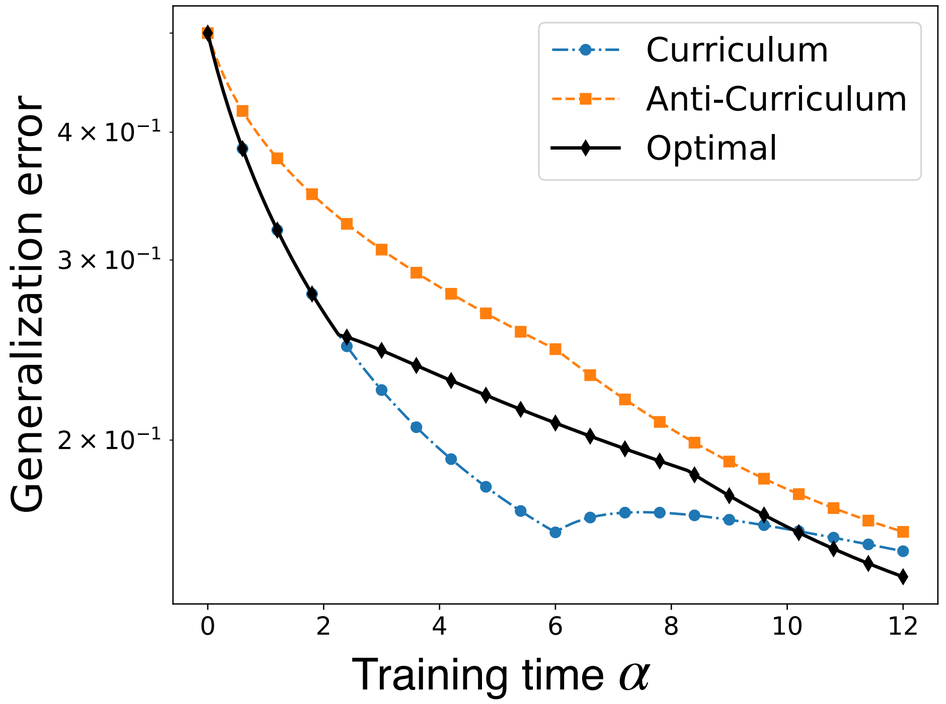
Optimal curriculum protocol

- Easy-to-hard curriculum is often *suboptimal*.

Control: $u = \Delta$

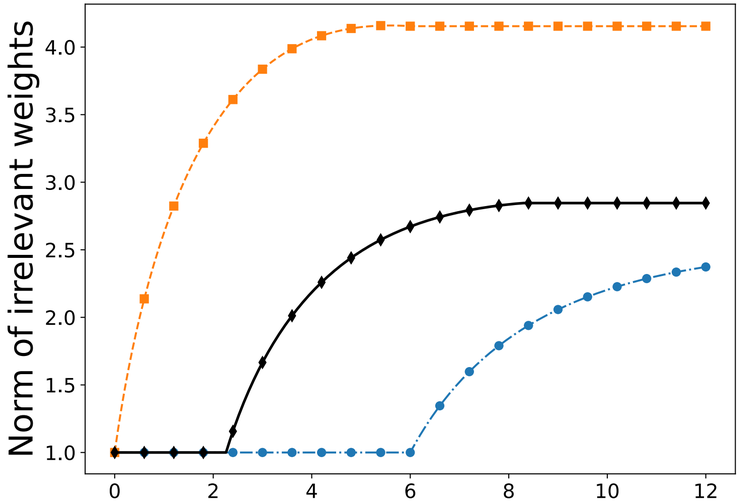
Optimal Δ protocol:

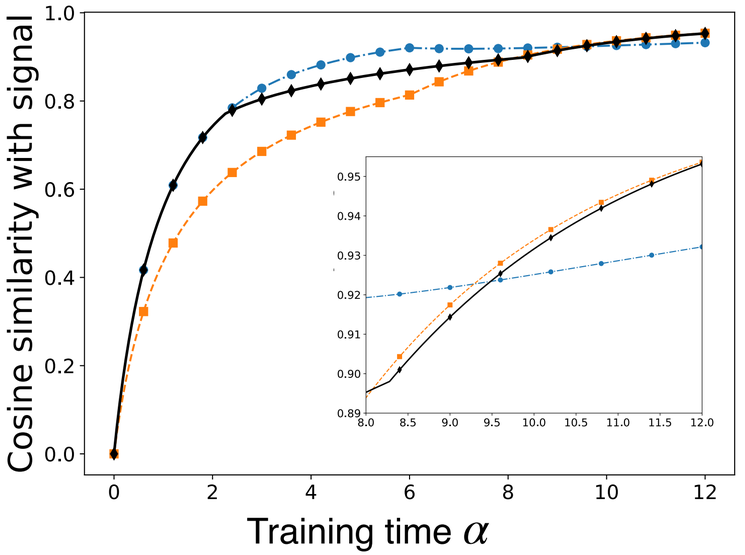




Don't overfit the irrelevant directions

Recover the signal along the relevant directions



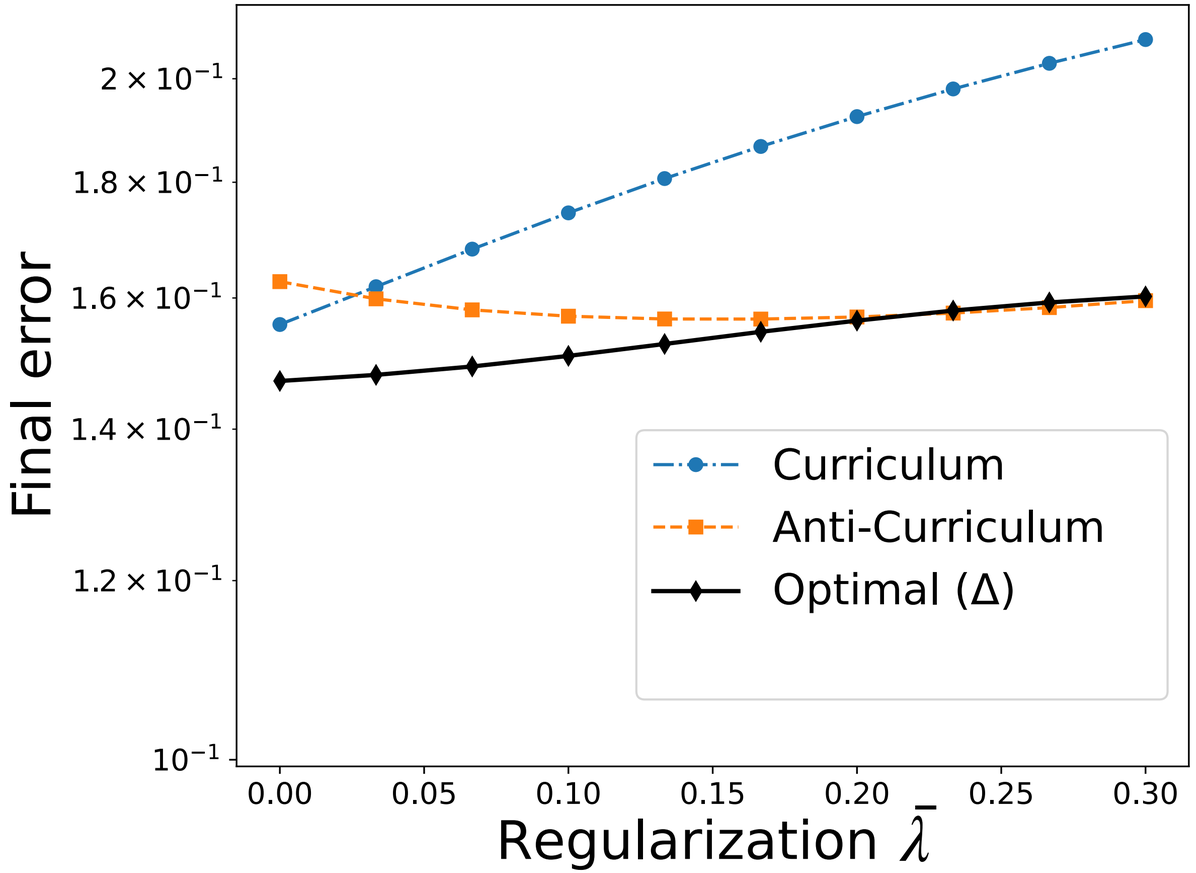


$\rho = 0.5,\ \eta = 3\ \Delta_1 = 0\ ,\ \Delta_2 = 2, \lambda = 0$

Optimal curriculum protocol

- Easy-to-hard curriculum is often *suboptimal*.

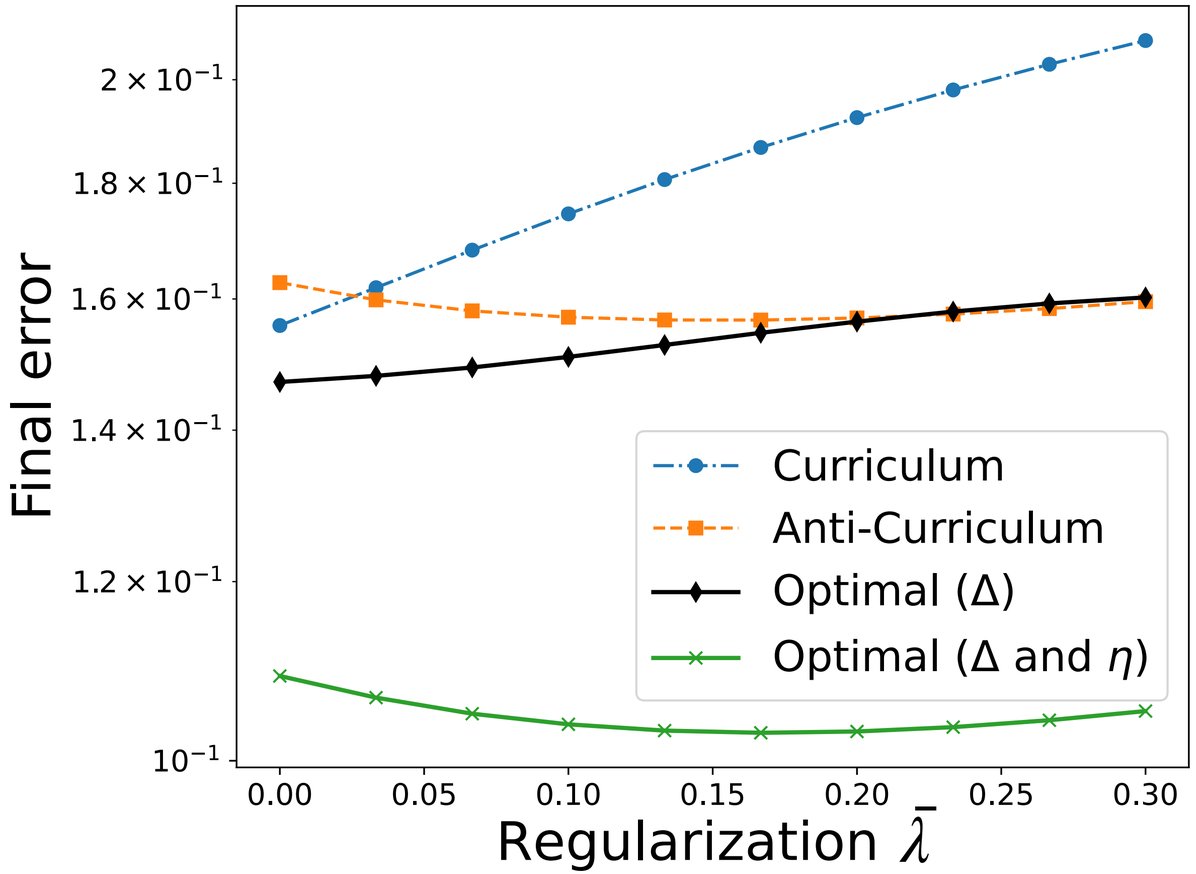
Control: $u = \Delta$



$\rho = 0.5,\ \eta = 3\ \Delta_1 = 0\ ,\ \Delta_2 = 2,$

Optimal curriculum protocol

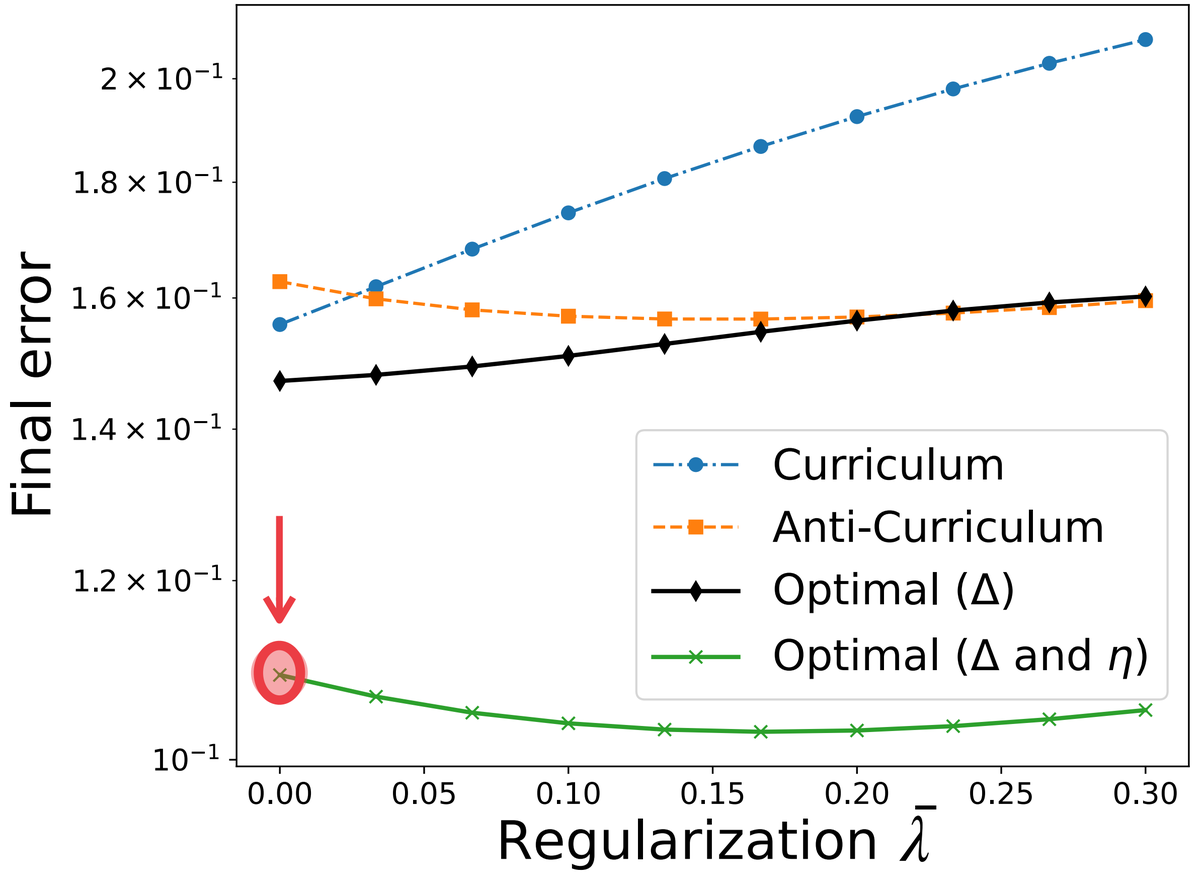
- Easy-to-hard curriculum becomes *optimal if we also optimize over the learning rate schedule.*

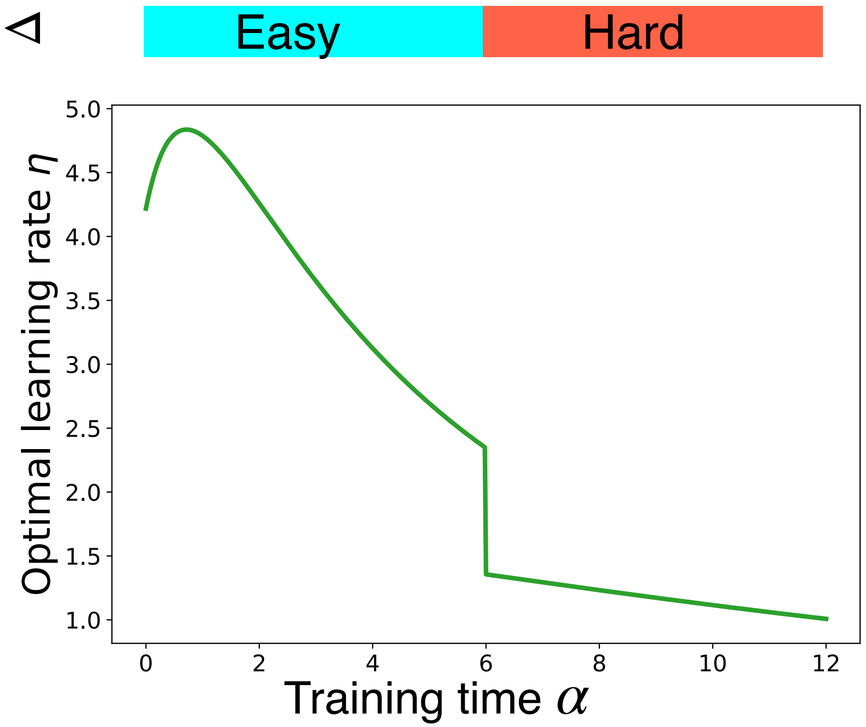


$\rho = 0.5,\ \eta = 3\ \Delta_1 = 0\ ,\ \Delta_2 = 2,$

Optimal curriculum protocol

- Easy-to-hard curriculum becomes *optimal if we also optimize over the learning rate schedule.*





$\lambda = 0$

$\rho = 0.5,\ \eta = 3\ \Delta_1 = 0\ ,\ \Delta_2 = 2,$

More applications

The impact of learning rate schedules on scaling laws

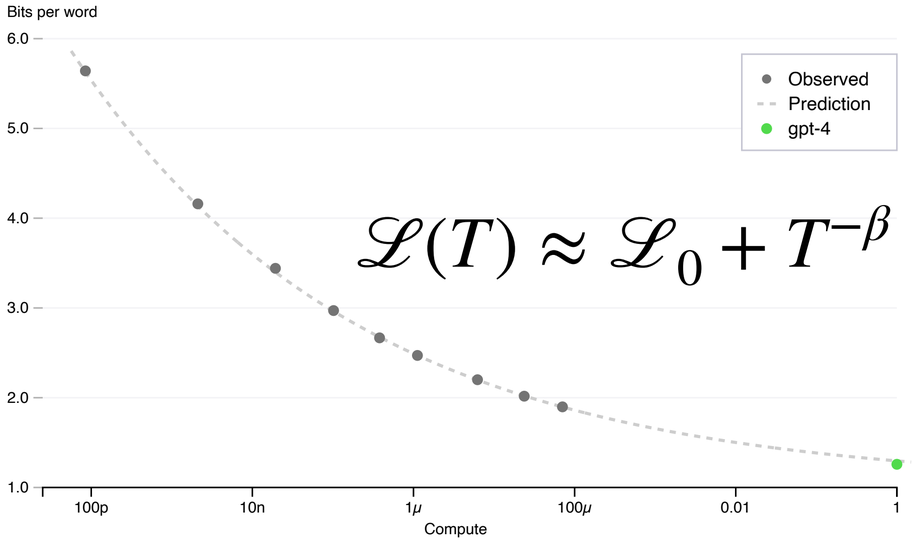
GPT-4 Technical Report

OpenAI*

Abstract

We report the development of GPT-4, a large-scale, multimodal model which can accept image and text inputs and produce text outputs. While less capable than humans in many real-world scenarios, GPT-4 exhibits human-level performance on various professional and academic benchmarks, including passing a simulated bar exam with a score around the top 10% of test takers. GPT-4 is a Transformer-based model pre-trained to predict the next token in a document. The post-training alignment process results in improved performance on measures of factuality and adherence to desired behavior. A core component of this project was developing infrastructure and optimization methods that behave predictably across a wide range of scales. This allowed us to accurately predict some aspects of GPT-4's performance based on models trained with no more than 1/1,000th the compute of GPT-4.

OpenAI codebase next word prediction



[B. Bordelon, F. Mori, *NeurIPS* 2026]

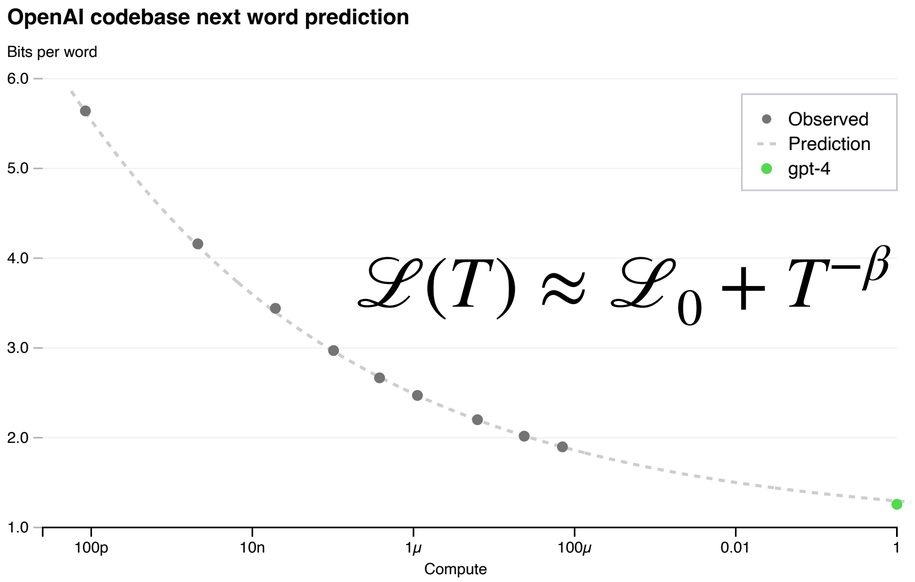
The impact of learning rate schedules on scaling laws

GPT-4 Technical Report

OpenAI*

Abstract

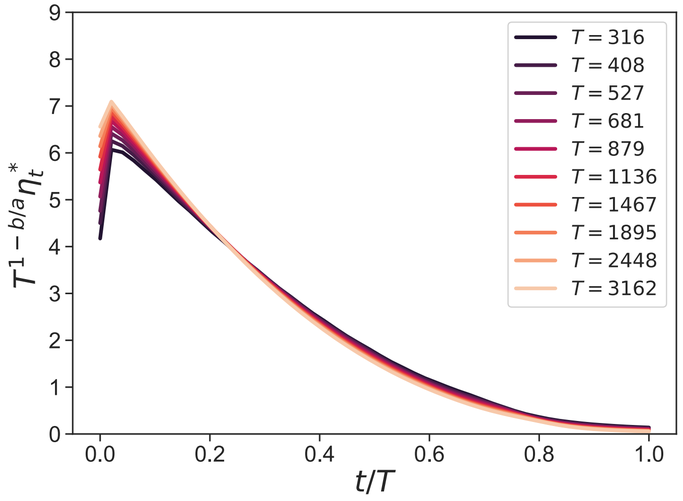
We report the development of GPT-4, a large-scale, multimodal model which can accept image and text inputs and produce text outputs. While less capable than humans in many real-world scenarios, GPT-4 exhibits human-level performance on various professional and academic benchmarks, including passing a simulated bar exam with a score around the top 10% of test takers. GPT-4 is a Transformer-based model pre-trained to predict the next token in a document. The post-training alignment process results in improved performance on measures of factuality and adherence to desired behavior. A core component of this project was developing infrastructure and optimization methods that behave predictably across a wide range of scales. This allowed us to accurately predict some aspects of GPT-4's performance based on models trained with no more than 1/1,000th the compute of GPT-4.



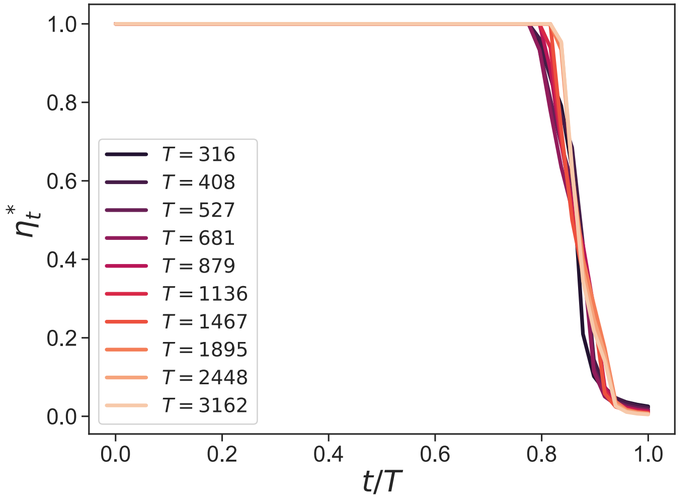
[B. Bordelon, F. Mori, *NeurIPS* 2026]

$$\eta_t^* = \text{argmin}_{\eta_t} \mathscr{L}(T) \qquad 0 < t < T$$

Easy phase ($b < a$)



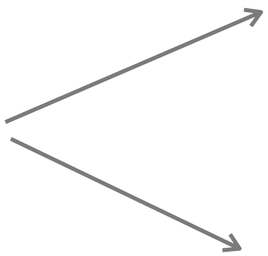
Hard phase ($b > a$)



Optimal schedules depend on the data structure.

Optimal selection schedules for online active learning

Stream of unlabeled inputs x



Query an expert to get the label $y(x)$
and use it to update the network

Fixed cost / total query budget

Skip the query and predict (risking mistake)

Which samples are worth labeling, and when?

Optimal selection schedules for online active learning

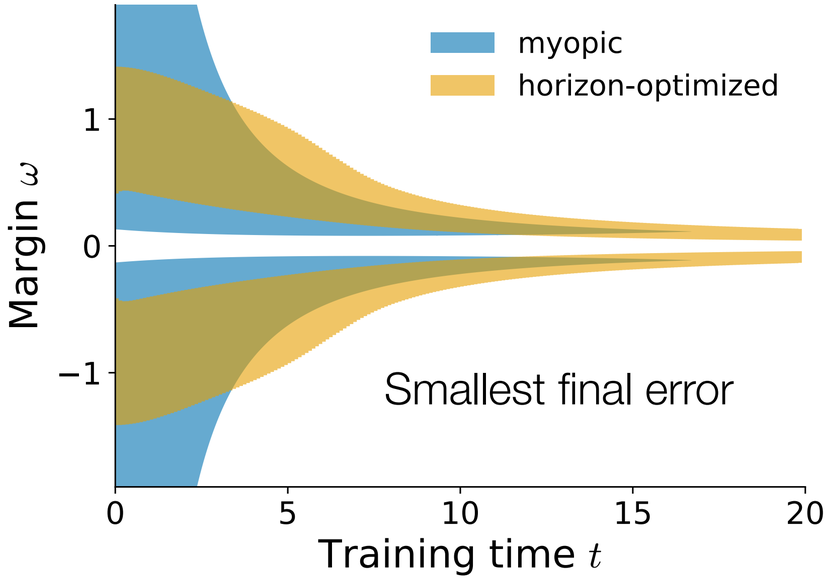
Stream of unlabeled inputs x

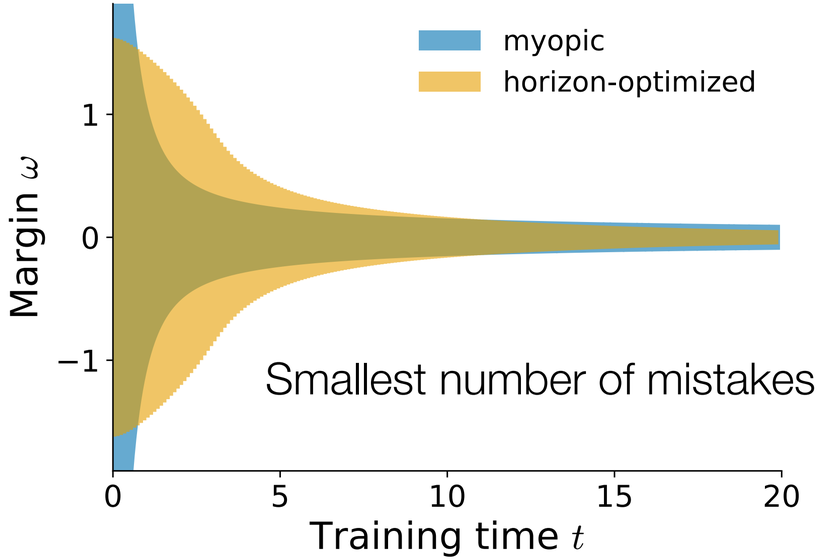
Query an expert to get the label $y(x)$ and use it to update the network

Fixed cost / total query budget

Skip the query and predict (risking mistake)

Which samples are worth labeling, and when?





[H. Cui, **FM**, F. Mori, arXiv:2610.23477]

The **optimal dynamic selection policy** depends on the query constraints and optimization objective.

Conclusions

- The design of learning protocols can be formulated as an optimal control problem on the low-dimensional dynamics of the order parameters to find nontrivial yet interpretable strategies.
- Joint protocol optimization highlights synergies between meta-parameters and learning trade-offs.

Many open directions, e.g.:

- Extend the theory to more realistic data models and architectures.
- Extend to other learning rules (Adam, momentum SGD, …)
- Optimal control of batch learning (memorization effects).
- Extend to different learning objectives (e.g., fairness metrics, robustness to distributional shifts, …).